BỘ LAO ĐỘNG – THƯƠNG BINH VÀ XÃ HỘI

TRƯỜNG CAO ĐẲNG CÔNG NGHỆ THÔNG TIN TP.HCM

KHOA CÔNG NGHỆ THÔNG TIN – ĐIỆN TỬ

BÁO CÁO ĐỒ ÁN MÔN HỌC

CHUYÊN ĐỀ BACK-END

ĐỀ TÀI:

XÂY DỰNG HỆ THỐNG BACK-END CHO WEBSITE THƯƠNG MẠI ĐIỆN TỬ BÁN LAPTOP LAPTOPNEW

Express.js · MongoDB Atlas · JWT · PayOS · Telegram Bot · Gemini AI

Giảng viên hướng dẫn: **ThS. Nguyễn Minh Hải**

Sinh viên thực hiện: **Lê Minh Trí – 501240045**

Âu Lê Thành Tài – 501250054

Lớp / Khóa: .....

TP. Hồ Chí Minh, tháng 9 năm 2026

LỜI MỞ ĐẦU

Thương mại điện tử đang thay đổi cách người Việt mua sắm thiết bị công nghệ. Với mặt hàng có giá trị cao và nhiều thông số kỹ thuật như máy tính xách tay, khách hàng cần một hệ thống vừa cung cấp thông tin cấu hình chính xác, vừa đặt hàng và thanh toán nhanh, lại vừa được tư vấn đúng nhu cầu. Phía sau mọi trải nghiệm đó là tầng Back-End: nơi lưu trữ dữ liệu, xử lý nghiệp vụ, xác thực người dùng và kết nối với các dịch vụ bên ngoài.

Xuất phát từ yêu cầu của môn **Chuyên đề Back-End**, nhóm chúng em thực hiện đề tài **"Xây dựng hệ thống Back-End cho website thương mại điện tử bán laptop LaptopNew bằng Express.js và MongoDB"**. Hệ thống được viết bằng Node.js và Express 5 theo mô hình phân tầng Route – Middleware – Controller – Service – Model, lưu trữ trên MongoDB Atlas thông qua Mongoose, xác thực bằng JSON Web Token và mã hóa mật khẩu bằng bcrypt.

Ngoài các chức năng quản lý cơ bản (sản phẩm, danh mục, đơn hàng, người dùng, mã giảm giá, đánh giá, tin tức), hệ thống còn tích hợp nhiều dịch vụ thực tế: gửi mã OTP và hóa đơn qua email (Brevo API và Nodemailer), thanh toán chuyển khoản tự động qua PayOS và VietQR, thông báo và duyệt đơn qua Telegram Bot, và Trợ lý AI tư vấn laptop dựa trên Google Gemini.

Báo cáo này trình bày toàn bộ quá trình phân tích, thiết kế và hiện thực hệ thống: từ kiến trúc tổng thể, thiết kế cơ sở dữ liệu, đặc tả API, luồng xử lý của từng chức năng, cơ chế xử lý lỗi và bảo mật, cho đến kết quả kiểm thử thực tế và hướng phát triển. Nhóm hy vọng đây vừa là sản phẩm hoàn thành yêu cầu học phần, vừa là bước chuẩn bị cho công việc phát triển phần mềm sau này.

LỜI CẢM ƠN

Trước hết, chúng em xin gửi lời cảm ơn chân thành đến Ban Giám hiệu và quý Thầy Cô Khoa Công nghệ Thông tin – Điện tử, Trường Cao đẳng Công nghệ Thông tin TP.HCM đã tạo môi trường học tập và truyền đạt nền tảng kiến thức trong suốt thời gian qua.

Đặc biệt, chúng em xin bày tỏ lòng biết ơn đến **Thầy Nguyễn Minh Hải**, giảng viên phụ trách môn Chuyên đề Back-End. Những bài giảng về Node.js, Express, MongoDB, cách tổ chức mã nguồn và bảo mật API của Thầy là nền tảng để nhóm hoàn thành đồ án này.

Chúng em cũng cảm ơn các thành viên trong nhóm đã hỗ trợ nhau trong quá trình thực hiện, cùng gia đình và bạn bè đã luôn động viên.

Do thời gian và kinh nghiệm còn hạn chế, báo cáo khó tránh khỏi thiếu sót. Nhóm rất mong nhận được nhận xét và góp ý của quý Thầy Cô để hoàn thiện hệ thống tốt hơn.

Chúng em xin chân thành cảm ơn!

TP. Hồ Chí Minh, tháng 9 năm 2026

Nhóm sinh viên thực hiện

NHẬN XÉT CỦA GIẢNG VIÊN HƯỚNG DẪN

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

Điểm đánh giá: .....

TP. Hồ Chí Minh, ngày ..... tháng ..... năm 2026

Giảng viên hướng dẫn

(Ký và ghi rõ họ tên)

MỤC LỤC

LỜI MỞ ĐẦU ..... **i**

LỜI CẢM ƠN ..... **ii**

NHẬN XÉT CỦA GIẢNG VIÊN HƯỚNG DẪN ..... **iii**

DANH MỤC TỪ VIẾT TẮT VÀ THUẬT NGỮ ..... **viii**

DANH MỤC HÌNH ẢNH ..... **ix**

DANH MỤC BẢNG ..... **xii**

CHƯƠNG 1. TỔNG QUAN ĐỀ TÀI **1**

1.1. Lý do chọn đề tài 1

1.2. Mục tiêu dự án 1

1.2.1. Các chức năng cơ bản 1

1.2.2. Mục đích phát triển ứng dụng Back-End 2

1.3. Phạm vi và đối tượng sử dụng 2

1.4. Công nghệ sử dụng 3

1.4.1. Node.js 3

1.4.2. Express.js 4

1.4.3. MongoDB Atlas và Mongoose 4

1.4.4. JSON Web Token (JWT) 4

1.4.5. Bcrypt 5

1.4.6. Nodemailer và Brevo Email API 5

1.4.7. Dotenv và CORS 5

1.4.8. PayOS và VietQR 5

1.4.9. Telegram Bot API 5

1.4.10. Google Gemini và 9Router Gateway 6

1.4.11. Công cụ phát triển 6

1.5. Lý do chọn công nghệ 6

CHƯƠNG 2. PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG **7**

2.1. Kiến trúc ứng dụng 7

2.1.1. Client (Front-End) 7

2.1.2. Server (Back-End) 7

2.1.3. Database (MongoDB Atlas) 8

2.1.4. Middleware 8

2.1.5. Vòng đời một request 8
2.2. Cấu trúc thư mục 10
2.2.1. config/ 10
2.2.2. routes/ 10
2.2.3. middleware/ 10
2.2.4. controllers/ 11
2.2.5. services/ 11
2.2.6. models/, utils/, scripts/ 11
2.3. Sơ đồ Use Case 11
2.4. Thiết kế cơ sở dữ liệu 12
2.4.1. Tổng quan các collection 12
2.4.2. Sơ đồ quan hệ dữ liệu 13
2.4.3. Collection san_pham 14
2.4.4. Collection don_hang 16
2.4.5. Collection nguoi_dung và dang_ky_tam 16
2.4.6. Collection ma_giam_gia 17
2.4.7. Collection danh_gia 18
2.4.8. Collection danh-muc, tin_tuc, lien_he, cai_dat 18
2.4.9. Chỉ mục (Index) và ràng buộc dữ liệu 19
CHƯƠNG 3. ĐẶC TẢ HỆ THỐNG API 20
3.1. Quy ước chung 20
3.1.1. Các mức quyền truy cập 20
3.2. Bảng tổng hợp API 20
3.3. Đặc tả chi tiết từng nhóm API 21
3.3.1. Nhóm Xác thực và Tài khoản — /api/auth 21
3.3.2. Nhóm Sản phẩm — /api/san-pham 22
3.3.3. Nhóm Danh mục — /api/danh-muc 23
3.3.4. Nhóm Đơn hàng — /api/don-hang 24
3.3.5. Nhóm Thanh toán — /api/thanh-toan 24
3.3.6. Nhóm Mã giảm giá — /api/ma-giam-gia 25
3.3.7. Nhóm Đánh giá — /api/danh-gia 25
3.3.8. Nhóm Người dùng — /api/nguoi-dung 26
3.3.9. Nhóm Tin tức — /api/tin-tuc 26
3.3.10. Nhóm Bảo hành — /api/bao-hanh 26
3.3.11. Nhóm Liên hệ — /api/lien-he 26

3.3.12. Nhóm Cài đặt showroom — /api/cai-dat 27
3.3.13. Nhóm Trợ lý AI — /api/tro-ly-ai 27
3.3.14. Nhóm Giám sát máy chủ 27
CHƯƠNG 4. HIỆN THỰC CÁC CHỨC NĂNG CHÍNH 29
4.1. Chức năng đăng ký và kích hoạt tài khoản bằng OTP 29
4.2. Chức năng đăng nhập, xác thực JWT và phân quyền 32
4.3. Chức năng đăng nhập bằng Google 35
4.4. Chức năng quên mật khẩu và đổi mật khẩu 36
4.5. Chức năng quản lý sản phẩm 39
4.5.1. Tìm kiếm, lọc và sắp xếp 39
4.5.2. Thêm, sửa, xóa sản phẩm (Admin) 41
4.6. Chức năng giỏ hàng và mã giảm giá 41
4.7. Chức năng đặt hàng và quản lý trạng thái đơn 44
4.7.1. Tạo đơn hàng 44
4.7.2. Phân quyền xem và cập nhật đơn 45
4.7.3. Trạng thái đơn và đồng bộ tồn kho 48
4.8. Chức năng thanh toán PayOS và VietQR 49
4.9. Chức năng gửi email tự động 52
4.10. Chức năng quản trị đơn hàng qua Telegram Bot 53
4.11. Chức năng Trợ lý AI tư vấn laptop 55
4.12. Chức năng tra cứu bảo hành điện tử 57
4.13. Các chức năng bổ trợ 59
4.14. Một số giao diện kết quả 59
CHƯƠNG 5. XỬ LÝ LỖI VÀ BẢO MẬT 64
5.1. Chuẩn hóa mã trạng thái HTTP 64
5.2. Xử lý từng loại lỗi 64
5.2.1. Lỗi 400 – Bad Request 64
5.2.2. Lỗi 401 – Unauthorized 65
5.2.3. Lỗi 403 – Forbidden 66
5.2.4. Lỗi 404 – Not Found 67
5.2.5. Lỗi 500 – Internal Server Error 68
5.2.6. Lỗi 429 – Too Many Requests 68
5.3. Các biện pháp bảo mật đã áp dụng 69
5.4. Đánh giá rủi ro còn tồn tại và hướng khắc phục 70

CHƯƠNG 6. TRIỂN KHAI VÀ KIỂM THỬ 71
6.1. Yêu cầu môi trường 71
6.2. Cấu hình biến môi trường 71
6.3. Cài đặt và khởi chạy 72
6.4. Kiểm thử API 73
6.5. Minh chứng kiểm thử API bằng Postman 75
6.5.1. Nhóm Xác thực và Tài khoản 76
6.5.2. Nhóm Sản phẩm và Danh mục 77
6.5.3. Nhóm Đơn hàng và Thanh toán 77
6.5.4. Nhóm Mã giảm giá, Đánh giá, Bảo hành và Liên hệ 78
6.5.5. Nhóm Trợ lý AI, Cài đặt và Giám sát máy chủ 79
CHƯƠNG 7. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN 81
7.1. Kết luận 81
7.2. Hướng phát triển 82
TÀI LIỆU THAM KHẢO 83

DANH MỤC TỪ VIẾT TẮT VÀ THUẬT NGỮ

Viết tắt / Thuật ngữ	Tên đầy đủ	Ý nghĩa trong đồ án
API	Application Programming Interface	Giao diện lập trình để Front-End gọi tới Back-End
REST / RESTful	Representational State Transfer	Phong cách thiết kế API dựa trên URL và phương thức HTTP
CRUD	Create – Read – Update – Delete	Bốn thao tác cơ bản trên dữ liệu
JSON	JavaScript Object Notation	Định dạng dữ liệu trao đổi giữa Client và Server
JWT	JSON Web Token (RFC 7519)	Chuỗi token ký số dùng để xác thực người dùng
RBAC	Role-Based Access Control	Phân quyền theo vai trò: khach_hang / admin
OTP	One-Time Password	Mã 6 số dùng một lần, hết hạn sau 10 phút
ODM	Object Document Mapping	Thư viện ánh xạ document ↔ đối tượng (Mongoose)
NoSQL	Not only SQL	Cơ sở dữ liệu phi quan hệ, lưu dạng document
BSON	Binary JSON	Định dạng nhị phân MongoDB dùng để lưu document
TTL Index	Time-To-Live Index	Chỉ mục tự xóa document khi hết hạn
CORS	Cross-Origin Resource Sharing	Cơ chế cho phép domain Front-End gọi API
MVC	Model – View – Controller	Mô hình tách lớp dữ liệu, xử lý và hiển thị
HMAC-SHA256	Hash-based Message Authentication Code	Thuật toán tạo/kiểm tra chữ ký PayOS
Webhook	HTTP Callback	Dịch vụ ngoài chủ động gọi vào API khi có sự kiện
NDJSON	Newline Delimited JSON	Mỗi dòng một JSON — dùng để stream câu trả lời AI
SKU	Stock Keeping Unit	Mã quản lý hàng hóa (ma_san_pham)
COD	Cash On Delivery	Thanh toán khi nhận hàng
CSKH	Chăm sóc khách hàng	Bộ phận xử lý yêu cầu liên hệ

DANH MỤC HÌNH ẢNH

Hình 2.1. Sơ đồ kiến trúc tổng thể hệ thống LaptopNew 7
Hình 2.2. Vòng đời xử lý một request qua các tầng của server.....9
Hình 2.3. Cấu trúc thư mục mã nguồn Back-End.....10
Hình 2.4. Sơ đồ use case của hệ thống Back-End LaptopNew 12
Hình 2.5. Sơ đồ quan hệ giữa các collection (ERD logic).....14
Hình 3.1. Kiểm thử GET /api/san-pham bằng Postman: trả về danh sách sản phẩm (200 OK).....28
Hình 4.1. Sơ đồ tuần tự đăng ký và kích hoạt tài khoản bằng OTP 30
Hình 4.2. Giao diện form đăng ký tài khoản 31
Hình 4.3. Email chứa mã OTP kích hoạt tài khoản 31
Hình 4.4. Kiểm thử POST /api/auth/kich-hoat bằng Postman: kích hoạt thành công, nhận JWT.....32
Hình 4.5. Sơ đồ tuần tự đăng nhập và gọi API được bảo vệ 33
Hình 4.6. Giao diện đăng nhập (email/mật khẩu và đăng nhập nhanh với Google) 34
Hình 4.7. Kiểm thử POST /api/auth/dang-nhap bằng Postman: đăng nhập thành công, nhận token JWT.....34
Hình 4.8. Chọn tài khoản Google khi đăng nhập 35
Hình 4.9. Google xác nhận chia sẻ tên, ảnh và email với website 36
Hình 4.10. Đăng nhập Google thành công, thanh điều hướng hiển thị tên người dùng 36
Hình 4.11. Sơ đồ tuần tự quên mật khẩu bằng OTP.....37
Hình 4.12. Trang quên mật khẩu: nhập email để nhận mã OTP 38
Hình 4.13. Bước nhập mã OTP xác thực.....38
Hình 4.14. Email chứa mã OTP đặt lại mật khẩu.....38
Hình 4.15. Kiểm thử POST /api/auth/xac-nhan-otp bằng Postman: OTP hợp lệ.....39
Hình 4.16. Trang danh sách sản phẩm với bộ lọc thương hiệu, khoảng giá, CPU, card đồ họa 40
Hình 4.17. Trang chi tiết sản phẩm: thư viện ảnh, phiên bản cấu hình, giá khuyến mãi 41
Hình 4.18. Trang quản trị: danh sách và form thêm/sửa sản phẩm.....41
Hình 4.19. Lưu đồ kiểm tra và tính tiền mã giảm giá.....42
Hình 4.20. Giỏ hàng áp dụng mã giảm giá TNTP3TR thành công 43
Hình 4.21. Kiểm thử POST /api/ma-giam-gia/kiem-tra bằng Postman: mã hợp lệ, giảm 3.000.000 đ 43

Hình 4.22. Lưu đồ xử lý tạo đơn hàng 45
Hình 4.23. Trang tra cứu tiến độ đơn hàng: khách nhập mã đơn và số điện thoại 46
Hình 4.24. Kiểm thử GET /api/don-hang/:id?sdt=… khi chưa đăng nhập: thông tin cá nhân đã được che 47
Hình 4.25. Kiểm thử GET /api/don-hang/:id với token chủ đơn: trả đầy đủ thông tin đơn hàng 47
Hình 4.26. Kiểm thử tra cứu đơn thiếu hoặc sai số điện thoại bị từ chối (403) 47
Hình 4.27. Vòng đời trạng thái đơn hàng và quy tắc trừ/hoàn tồn kho 48
Hình 4.28. Trang thanh toán: thông tin người nhận, địa chỉ hành chính, phương thức thanh toán 48
Hình 4.29. Email xác nhận đặt hàng thành công gửi tới khách hàng 49
Hình 4.30. Trang quản trị đơn hàng (Admin): lọc đơn và đổi trạng thái 49
Hình 4.31. Sơ đồ tuần tự thanh toán PayOS / VietQR với Webhook và polling 50
Hình 4.32. Màn hình mã QR thanh toán PayOS / VietQR 51
Hình 4.33. Thanh toán thành công — đơn tự chuyển trạng thái 52
Hình 4.34. Email xác nhận đơn hàng (hóa đơn HTML) 53
Hình 4.35. Sơ đồ tuần tự thông báo và duyệt đơn qua Telegram Bot 54
Hình 4.36. Tin nhắn đơn hàng mới kèm nút duyệt trên Telegram 55
Hình 4.37. Kết quả lệnh /thongke trên Telegram 55
Hình 4.38. Luồng xử lý của Trợ lý AI với cơ chế dự phòng nhiều tầng 56
Hình 4.39. Hộp chat Trợ lý AI tư vấn và gợi ý sản phẩm trên website 57
Hình 4.40. Kiểm thử POST /api/tro-ly-ai/chat bằng Postman: câu trả lời kèm sản phẩm gợi ý 57
Hình 4.41. Lưu đồ tra cứu bảo hành điện tử 58
Hình 4.42. Giao diện tra cứu bảo hành điện tử 58
Hình 4.43. Trang chủ website 59
Hình 4.44. Trang tin tức công nghệ 60
Hình 4.45. Kiểm thử GET /api/tin-tuc bằng Postman: danh sách bài viết 60
Hình 4.46. Trang liên hệ và hệ thống showroom có bản đồ 60
Hình 4.47. Trang so sánh: danh sách laptop đã chọn để so sánh 61
Hình 4.48. Bảng đối chiếu nhanh thương hiệu, giá và đánh giá 61
Hình 4.49. So sánh chi tiết bộ xử lý (CPU) giữa các máy 61
Hình 4.50. Trang tổng quan quản trị: thẻ thống kê doanh thu, đơn hàng, khách hàng, kho 62
Hình 4.51. Biểu đồ xu hướng doanh thu và cơ cấu ngành hàng 62

Hình 4.52. Kiểm thử GET / bằng Postman: thông tin server và danh sách endpoint chính 63
Hình 4.53. Dữ liệu JSON thực tế trả về từ GET /api/san-pham 63
Hình 4.54. Dữ liệu các collection trên MongoDB Atlas 63
Hình 5.1. Kiểm thử lỗi 400 bằng Postman: đăng ký thiếu thông tin 65
Hình 5.2. Kiểm thử lỗi 401 bằng Postman: gọi /api/auth/toi không gửi token.....66
Hình 5.3. Kiểm thử lỗi 403 bằng Postman: tài khoản khách gọi API thêm sản phẩm .67
Hình 5.4. Kiểm thử lỗi 404 bằng Postman: sản phẩm không tồn tại 68
Hình 5.5. Kiểm thử lỗi 429 bằng Postman: tra cứu đơn vượt giới hạn 69
Hình 6.1. Terminal khởi chạy Back-End thành công 73
Hình 6.2. Bộ kiểm thử chạy bằng Postman Runner 75
Hình 6.3. Kiểm thử POST /api/auth/dang-ky bằng Postman: đăng ký hợp lệ, gửi OTP 76
Hình 6.4. Kiểm thử GET /api/auth/toi với token hợp lệ 76
Hình 6.5. Kiểm thử PUT /api/auth/doi-mat-khau bằng Postman 77
Hình 6.6. Kiểm thử lọc sản phẩm theo nhiều hãng và danh mục 77
Hình 6.7. Kiểm thử POST /api/san-pham bằng tài khoản Admin (201 Created).....77
Hình 6.8. Kiểm thử GET /api/danh-muc?loai=thuong_hieu 77
Hình 6.9. Kiểm thử POST /api/don-hang: tạo đơn thành công (201 Created) 77
Hình 6.10. Kiểm thử PUT /api/don-hang/:id: Admin cập nhật trạng thái đơn 78
Hình 6.11. Kiểm thử POST /api/thanh-toan/tao-link: tạo mã QR thanh toán 78
Hình 6.12. Kiểm thử GET /api/thanh-toan/kiem-tra/:orderCode 78
Hình 6.13. Kiểm thử Webhook PayOS với chữ ký giả mạo bị từ chối (400) 78
Hình 6.14. Kiểm thử lưu voucher và xem ví voucher của tôi.....78
Hình 6.15. Kiểm thử POST /api/danh-gia: gửi đánh giá sản phẩm 79
Hình 6.16. Kiểm thử GET /api/bao-hanh/tra-cuu: tra cứu bảo hành điện tử.....79
Hình 6.17. Kiểm thử POST /api/lien-he: gửi phiếu yêu cầu tư vấn 79
Hình 6.18. Kiểm thử GET /api/tro-ly-ai/trang-thai.....79
Hình 6.19. Kiểm thử GET /api/cai-dat/showroom 79
Hình 6.20. Kiểm thử GET /api/health: kiểm tra sức khỏe máy chủ 80

DANH MỤC BẢNG

Bảng 1.1. Các nhóm đối tượng sử dụng hệ thống....2
Bảng 1.2. Công nghệ và thư viện sử dụng ở Back-End....3
Bảng 1.3. Lý do lựa chọn các công nghệ chính....6
Bảng 2.1. Vai trò các tầng trong server Express....8
Bảng 2.2. Danh sách collection trong cơ sở dữ liệu....12
Bảng 2.3. Cấu trúc collection san_pham....14
Bảng 2.4. Cấu trúc collection don_hang....16
Bảng 2.5. Cấu trúc collection nguoi_dung....16
Bảng 2.6. Cấu trúc collection ma_giam_gia....17
Bảng 2.7. Cấu trúc collection danh_gia....18
Bảng 2.8. Cấu trúc các collection danh-muc, tin_tuc, lien_he, cai_dat....18
Bảng 2.9. Các chỉ mục chính trong cơ sở dữ liệu....19
Bảng 3.1. Quy ước chung của API....20
Bảng 3.2. Ký hiệu mức quyền dùng trong các bảng đặc tả....20
Bảng 3.3. Thống kê số route theo nhóm chức năng....21
Bảng 3.4. Đặc tả API nhóm /api/auth....21
Bảng 3.5. Đặc tả API nhóm /api/san-pham....22
Bảng 3.6. Giá trị của tham số sap_xep....23
Bảng 3.7. Đặc tả API nhóm /api/danh-muc....23
Bảng 3.8. Đặc tả API nhóm /api/don-hang....24
Bảng 3.9. Đặc tả API nhóm /api/thanh-toan....24
Bảng 3.10. Đặc tả API nhóm /api/ma-giam-gia....25
Bảng 3.11. Đặc tả API nhóm /api/danh-gia....25
Bảng 3.12. Đặc tả API nhóm /api/nguoi-dung....26
Bảng 3.13. Đặc tả API nhóm /api/tin-tuc....26
Bảng 3.14. Đặc tả API nhóm /api/bao-hanh....26
Bảng 3.15. Đặc tả API nhóm /api/lien-he....26
Bảng 3.16. Đặc tả API nhóm /api/cai-dat....27
Bảng 3.17. Đặc tả API nhóm /api/tro-ly-ai....27
Bảng 3.18. Đặc tả API giám sát máy chủ....27
Bảng 4.1. Các chức năng bổ trợ khác....59
Bảng 5.1. Các mã trạng thái HTTP được sử dụng....64
Bảng 5.2. Tổng hợp biện pháp bảo mật....69

Bảng 5.3. Các rủi ro bảo mật phát hiện khi rà soát mã nguồn 70
Bảng 6.1. Danh sách biến môi trường của Back-End 71
Bảng 6.2. Kết quả kiểm thử API thực tế 73
Bảng 6.3. Tổng hợp minh chứng kiểm thử Postman theo nhóm API 75

CHƯƠNG 1. TỔNG QUAN ĐỀ TÀI

1.1. Lý do chọn đề tài

Laptop là mặt hàng có nhiều biến thể cấu hình (CPU, RAM, ổ cứng, card đồ họa, màn hình), giá trị cao và thay đổi nhanh theo từng thế hệ. Một website bán laptop vì vậy phải giải quyết nhiều bài toán Back-End điển hình:

- **Dữ liệu sản phẩm phức tạp:** mỗi máy có hàng chục thông số, nhiều phiên bản, quà tặng, tồn kho theo từng chi nhánh — rất phù hợp để khai thác mô hình document của MongoDB.
- **Xác thực và phân quyền:** khách hàng và quản trị viên có quyền khác nhau; tài khoản mới cần xác minh email để tránh đăng ký rác.
- **Đơn hàng và tồn kho:** trạng thái đơn thay đổi nhiều bước, tồn kho phải trừ và hoàn trả chính xác.
- **Thanh toán trực tuyến:** chuyển khoản thủ công dễ nhầm lẫn; cần đối soát tự động qua cổng thanh toán.
- **Tích hợp dịch vụ ngoài:** email, bot thông báo, trí tuệ nhân tạo — đòi hỏi kỹ năng gọi API bên thứ ba và xử lý lỗi, dự phòng.

Đề tài vừa bám sát nội dung môn học (Node.js, Express, MongoDB, JWT, Nodemailer), vừa đủ độ phức tạp để nhóm rèn luyện kỹ năng thiết kế một hệ thống Back-End hoàn chỉnh như trong thực tế.

1.2. Mục tiêu dự án

1.2.1. Các chức năng cơ bản

- **Đăng ký tài khoản** có xác thực mã OTP gửi qua email; **đăng nhập** bằng email/mật khẩu hoặc **Google**; **đăng xuất**.
- **Quên mật khẩu, đổi mật khẩu** bằng mã OTP; xem và cập nhật hồ sơ cá nhân, sổ địa chỉ.
- **Quản lý sản phẩm:** xem danh sách, tìm kiếm, lọc theo hãng/danh mục/khoảng giá, sắp xếp; xem chi tiết; admin thêm, sửa, xóa.
- **Quản lý danh mục, thương hiệu, tin tức, showroom** phục vụ giao diện.

- **Mã giảm giá:** kiểm tra và tính số tiền giảm; lưu voucher vào ví cá nhân; admin quản lý voucher.
- **Đặt hàng:** tạo đơn, xem lịch sử, tra cứu đơn, hủy đơn; admin cập nhật trạng thái, hệ thống tự trừ/hoàn tồn kho.
- **Thanh toán:** tạo mã QR PayOS/VietQR, kiểm tra trạng thái, nhận Webhook xác nhận tiền về.
- **Đánh giá sản phẩm**, **tra cứu bảo hành**, **gửi liên hệ tư vấn**, **chat với Trợ lý AI**.
- **Quản trị người dùng** và **nhận thông báo, duyệt đơn qua Telegram Bot**.

1.2.2. Mục đích phát triển ứng dụng Back-End

- **Cung cấp API RESTful:** 13 nhóm tài nguyên với 69 route để Front-End Next.js gọi tới.
- **Xử lý nghiệp vụ:** kiểm tra dữ liệu, tính tiền giảm giá, quản lý trạng thái đơn, tồn kho, thời hạn bảo hành.
- **Lưu trữ dữ liệu:** thiết kế 9 collection chính trên MongoDB Atlas và 1 collection tạm cho đăng ký.
- **Bảo mật:** mã hóa mật khẩu, xác thực JWT, phân quyền RBAC, kiểm tra chữ ký Webhook, giới hạn CORS.
- **Tích hợp:** kết nối Brevo/Gmail, PayOS, VietQR, Telegram, Google OAuth, Google Gemini.

1.3. Phạm vi và đối tượng sử dụng

Phạm vi đồ án tập trung vào **phần Back-End** (thư mục `BE/`). Phần Front-End Next.js (thư mục `FE/`) được dùng để minh họa kết quả và kiểm thử luồng nghiệp vụ. Hệ thống phục vụ bốn nhóm đối tượng:

Bảng 1.1. *Các nhóm đối tượng sử dụng hệ thống*

Đối tượng	Cách nhận diện ở Back-End	Quyền chính
Khách vãng lai	Không gửi token	Xem sản phẩm, tin tức, đánh giá; đăng ký, đăng nhập; tra cứu bảo hành; gửi liên hệ; chat AI
Thành viên	Token JWT hợp lệ, `vaiTro = khach_hang`	Mọi quyền của khách + đặt hàng, thanh toán, xem đơn của mình, ví voucher, hồ sơ

Đối tượng	Cách nhận diện ở Back-End	Quyền chính
Quản trị viên	Token JWT hợp lệ, `vaiTro = admin`	Mọi quyền của thành viên + CRUD sản phẩm, danh mục, voucher, người dùng; xem và xử lý toàn bộ đơn
Hệ thống ngoài	Webhook có chữ ký / Bot token	PayOS báo thanh toán; Telegram gửi thao tác duyệt đơn của admin

1.4. Công nghệ sử dụng

Bảng dưới đây tổng hợp các thư viện chính khai báo trong `BE/package.json` và các dịch vụ được tích hợp.

Bảng 1.2. *Công nghệ và thư viện sử dụng ở Back-End*

Thành phần	Phiên bản	Vai trò trong hệ thống
Node.js	v24	Môi trường chạy JavaScript phía server
express	^5.2.1	Framework xây dựng REST API, router, middleware
mongoose	^9.9.4	ODM: định nghĩa Schema, kiểm tra dữ liệu, truy vấn MongoDB
MongoDB Atlas	Cloud	Cơ sở dữ liệu NoSQL lưu toàn bộ dữ liệu (database `laptop_store`)
jsonwebtoken	^9.0.3	Tạo và xác minh JWT (hạn 7 ngày)
bcryptjs	^3.0.3	Băm mật khẩu với salt 10 vòng
nodemailer	^10.0.9	Gửi email qua Gmail SMTP khi chạy local
Brevo REST API v3	—	Gửi email khi triển khai cloud (SMTP bị chặn)
google-auth-library	^11.0.2	Xác minh Google ID Token khi đăng nhập Google
cors	^2.8.6	Chỉ cho phép domain Front-End khai báo trong `CLIENT_URL`
dotenv	^17.4.2	Nạp biến môi trường từ tệp `.env`
nodemon	^3.1.14 (dev)	Tự khởi động lại server khi sửa mã nguồn
PayOS + VietQR	REST API	Tạo link/QR thanh toán, Webhook đối soát
Telegram Bot API	REST API	Thông báo đơn mới, nút duyệt đơn, lệnh thống kê
Google Gemini / 9Router	REST API	Mô hình ngôn ngữ cho Trợ lý AI tư vấn laptop

1.4.1. Node.js

Node.js là môi trường thực thi JavaScript mã nguồn mở chạy trên V8 Engine của Google Chrome. Node.js hoạt động theo mô hình **hướng sự kiện, vào/ra không chặn**

(non-blocking I/O): khi chờ truy vấn cơ sở dữ liệu hoặc gọi API bên ngoài, luồng chính vẫn tiếp tục phục vụ yêu cầu khác. Điều này rất phù hợp với hệ thống của nhóm, vốn phải gọi nhiều dịch vụ ngoài (PayOS, Telegram, Gemini) cùng lúc.

Công cụ hỗ trợ — Nodemon: tự động khởi động lại server mỗi khi mã nguồn thay đổi (lệnh `npm run dev`), giúp tăng tốc quá trình phát triển.

1.4.2. Express.js

Express là framework web tối giản và phổ biến nhất của Node.js. Dự án dùng **Express 5**, phiên bản tự động chuyển lỗi của hàm `async` sang middleware xử lý lỗi. Các tính năng được khai thác:

- `express.Router()` để tách 13 nhóm route thành 13 tệp riêng.
- Chuỗi **middleware** (CORS → parse JSON → log → xác thực → phân quyền) trước khi vào controller.
- Middleware xử lý lỗi 404 và 500 tập trung ở cuối `server.js`.

1.4.3. MongoDB Atlas và Mongoose

MongoDB là cơ sở dữ liệu NoSQL hướng tài liệu: dữ liệu được lưu thành các **document** dạng BSON (tương tự JSON) trong các **collection**. Nhờ đó, thông số kỹ thuật, danh sách phiên bản, quà tặng, tồn kho chi nhánh của một laptop có thể nằm gọn trong một document mà không cần nhiều bảng và phép JOIN như SQL. Dự án dùng **MongoDB Atlas** — dịch vụ MongoDB trên đám mây, có sẵn Replica Set sao lưu dữ liệu.

Mongoose là thư viện ODM giúp định nghĩa Schema (kiểu dữ liệu, bắt buộc, duy nhất, giá trị mặc định, enum), tạo index, viết hook `pre('validate')` để tự sinh slug và thêm phương thức cho model (ví dụ `soSanhMatKhau`).

1.4.4. JSON Web Token (JWT)

JWT là tiêu chuẩn mở (RFC 7519) để truyền thông tin xác thực dưới dạng chuỗi đã ký số gồm ba phần *Header.Payload.Signature*. Sau khi đăng nhập, server ký token chứa `userId`, `email`, `vaiTro` bằng khóa `JWT_SECRET`, thời hạn 7 ngày. Client gửi token ở header `Authorization: Bearer <token>`; server chỉ cần xác minh chữ ký mà không phải lưu phiên (stateless).

1.4.5. Bcrypt

Bcrypt là hàm băm mật khẩu được thiết kế **chậm có chủ đích** và có **salt ngẫu nhiên**, khiến tấn công dò mật khẩu (brute-force) và bảng tra sẵn (rainbow table) trở nên rất tốn kém. Hệ thống dùng `bcrypt.genSalt(10)` và `bcrypt.hash()` khi lưu, `bcrypt.compare()` khi đăng nhập; mật khẩu gốc không bao giờ được lưu.

1.4.6. Nodemailer và Brevo Email API

Nodemailer là module gửi email phổ biến của Node.js, hỗ trợ nội dung HTML. Tuy nhiên, nhiều nền tảng cloud (như Railway) chặn cổng SMTP, nên dịch vụ email của nhóm được thiết kế **hai tầng**: ưu tiên gọi **Brevo REST API v3** qua HTTPS; nếu chưa cấu hình Brevo và đang chạy local thì dùng Nodemailer với Gmail. Hệ thống gửi bốn loại email: OTP kích hoạt, OTP quên mật khẩu, OTP đổi mật khẩu và hóa đơn xác nhận đơn hàng.

1.4.7. Dotenv và CORS

Dotenv nạp cấu hình nhạy cảm (chuỗi kết nối MongoDB, khóa JWT, khóa PayOS, token Telegram, khóa Gemini) từ tệp `.env` vào `process.env`, giúp tách bí mật khỏi mã nguồn và cấu hình khác nhau cho từng môi trường. **CORS** được cấu hình chỉ chấp nhận các origin liệt kê trong `CLIENT_URL`.

1.4.8. PayOS và VietQR

PayOS là cổng thanh toán cho phép tạo yêu cầu thanh toán, sinh mã QR chuyển khoản có sẵn số tiền và nội dung, sau đó gọi **Webhook** về server khi tiền vào tài khoản. Mọi yêu cầu và Webhook đều được ký bằng **HMAC-SHA256** với `checksumKey`. Song song, hệ thống dựng sẵn ảnh **VietQR** (chuẩn Napas) để khách luôn có QR, kể cả khi PayOS phản hồi chậm.

1.4.9. Telegram Bot API

Telegram Bot API cho phép server gửi tin nhắn có **bàn phím nút bấm (Inline Keyboard)** tới quản trị viên và nhận lại thao tác bằng cơ chế **Long Polling** (`getUpdates`). Nhờ đó admin có thể xác nhận, giao hàng hoặc hủy đơn ngay trên điện thoại, và dùng các lệnh `/thongke`, `/donmoi`, `/tatcadon`, `/timdon`.

1.4.10. Google Gemini và 9Router Gateway

Trợ lý AI dùng mô hình ngôn ngữ lớn để hiểu câu hỏi tiếng Việt và tư vấn laptop dựa trên **dữ liệu sản phẩm thật** trong MongoDB. Yêu cầu được gửi trước tới cổng trung gian **9Router** (chuẩn OpenAI `/chat/completions`, hỗ trợ stream); nếu lỗi sẽ chuyển sang gọi trực tiếp **Google Gemini** với cơ chế **xoay vòng nhiều API key**, và cuối cùng là câu trả lời dự phòng.

1.4.11. Công cụ phát triển

- **Visual Studio Code:** soạn thảo mã nguồn.
- **Postman:** gửi request và kiểm thử API.
- **MongoDB Compass / Atlas Web UI:** xem và kiểm tra dữ liệu.
- **Git & GitHub:** quản lý phiên bản mã nguồn và làm việc nhóm.

1.5. Lý do chọn công nghệ

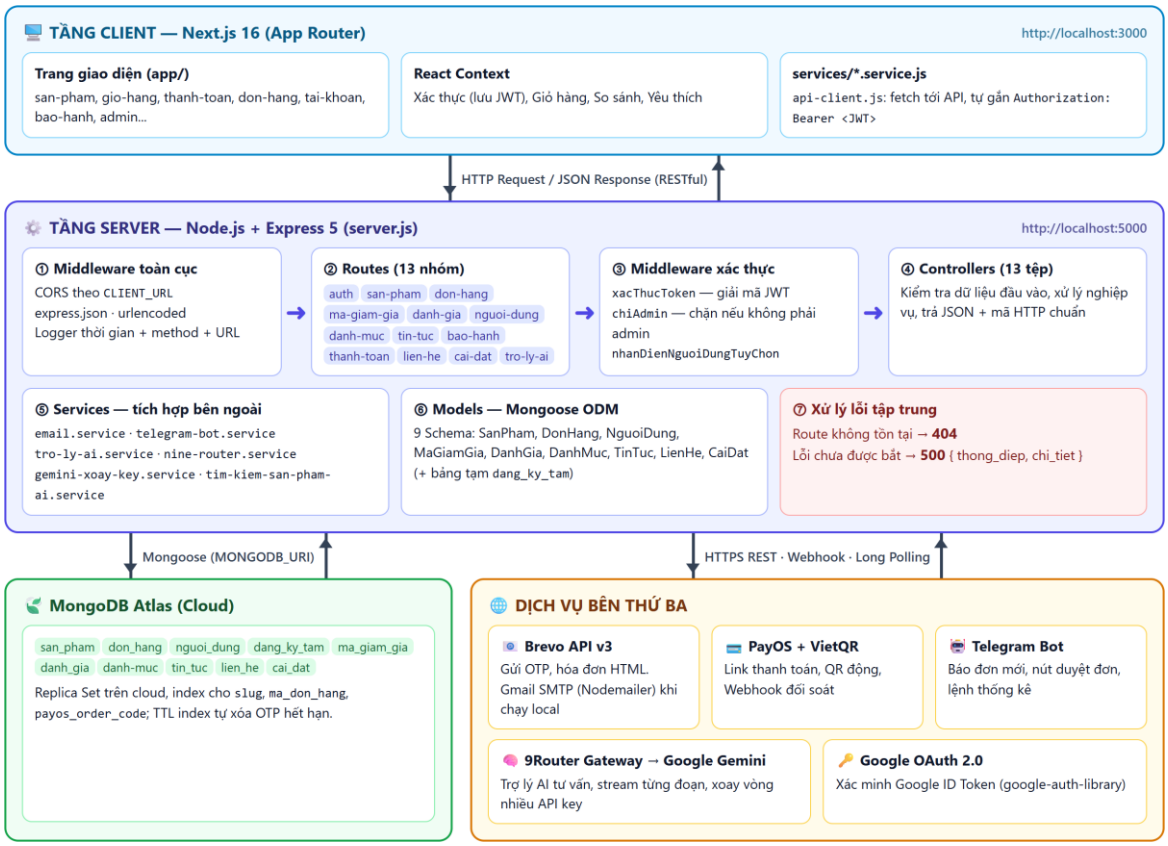
Bảng 1.3. *Lý do lựa chọn các công nghệ chính*

Công nghệ	Lý do lựa chọn
Node.js	Dùng chung JavaScript cho cả Front-End và Back-End; xử lý bất đồng bộ tốt cho ứng dụng nhiều I/O; hệ sinh thái npm rất lớn.
Express	Đơn giản, linh hoạt, cộng đồng lớn; cơ chế middleware giúp tách xác thực, phân quyền, xử lý lỗi rõ ràng.
MongoDB	Schema linh hoạt, phù hợp dữ liệu laptop nhiều thông số lồng nhau; dễ mở rộng; Atlas miễn phí cho môi trường học tập.
Mongoose	Ràng buộc dữ liệu ngay ở tầng ứng dụng, hỗ trợ hook, index, phương thức model.
JWT	Xác thực không lưu phiên, dễ dùng cho SPA/Next.js, mang theo vai trò để phân quyền.
Bcrypt	Chuẩn công nghiệp cho lưu mật khẩu, có salt và độ trễ điều chỉnh được.
Nodemailer / Brevo	Gửi email OTP và hóa đơn; Brevo khắc phục việc cloud chặn SMTP.
PayOS	Đối soát chuyển khoản tự động qua Webhook, không cần kiểm tra thủ công.

CHƯƠNG 2. PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG

2.1. Kiến trúc ứng dụng

Hệ thống được tổ chức theo mô hình **Client – Server** ba tầng: tầng giao diện Next.js, tầng API Express và tầng dữ liệu MongoDB Atlas, cùng với các dịch vụ bên thứ ba. Bên trong server, mã nguồn được chia theo **kiến trúc phân tầng** (biến thể của MVC cho Web API, trong đó "View" chính là dữ liệu JSON trả về).



Hình 2.1. *Sơ đồ kiến trúc tổng thể hệ thống LaptopNew*

2.1.1. Client (Front-End)

- Là nơi người dùng tương tác: xem sản phẩm, đặt hàng, đăng nhập, quản trị.
- Giao tiếp với server qua các endpoint `/api/...`; token JWT được lưu ở client và gắn vào header `Authorization` bởi `services/api-client.js`.
- Dữ liệu trả về ở định dạng JSON để hiển thị.

2.1.2. Server (Back-End)

Server Express nhận request, xử lý nghiệp vụ và trả JSON. Mã nguồn được chia thành các tầng có trách nhiệm riêng:

Bảng 2.1. *Vai trò các tầng trong server Express*

Tầng	Thư mục	Trách nhiệm
Routes	`routes/`	Khai báo URL + phương thức HTTP, gắn middleware và hàm controller tương ứng.
Middleware	`middleware/`	Xử lý trước controller: giải mã JWT, kiểm tra quyền admin, nhận diện người dùng tùy chọn.
Controllers	`controllers/`	Đọc tham số, kiểm tra dữ liệu, thực hiện nghiệp vụ, trả mã HTTP và JSON.
Services	`services/`	Đóng gói việc gọi dịch vụ ngoài: email, Telegram, AI; controller không cần biết chi tiết.
Models	`models/`	Định nghĩa Schema Mongoose, ràng buộc dữ liệu và truy cập MongoDB.
Config / Utils	`config/`, `utils/`	Kết nối CSDL, cấu hình AI; hàm tiện ích như tạo slug tiếng Việt.

2.1.3. Database (MongoDB Atlas)

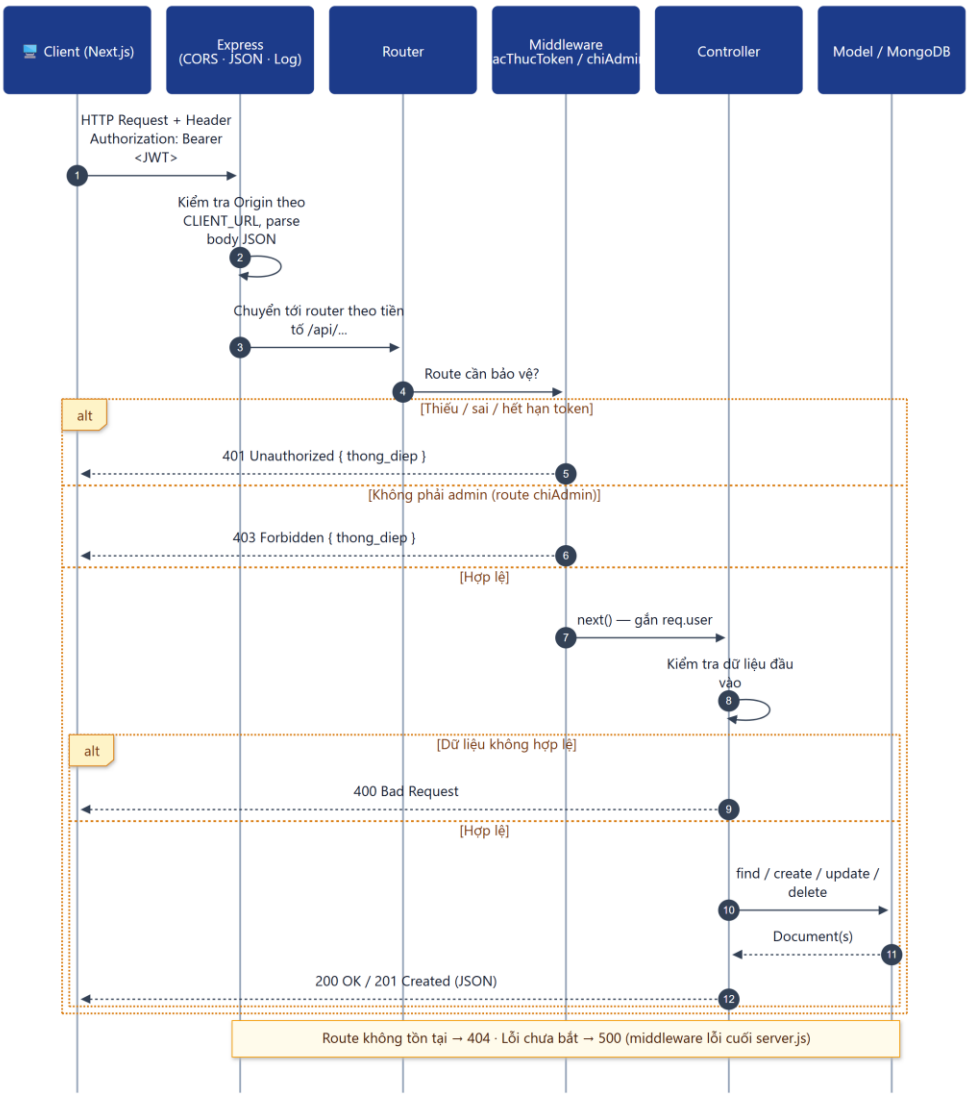
- Lưu toàn bộ dữ liệu: sản phẩm, đơn hàng, người dùng, voucher, đánh giá, danh mục, tin tức, liên hệ, cấu hình showroom.
- Hỗ trợ đầy đủ thao tác CRUD thông qua Mongoose; kết nối bằng biến `MONGODB_URI` với thời gian chờ chọn server 6 giây.

2.1.4. Middleware

- **Toàn cục:** `cors` (kiểm tra origin), `express.json()`, `express.urlencoded()`, middleware ghi log `[giờ] METHOD URL`.
- **Xác thực:** `xacThucToken` bắt buộc có JWT hợp lệ; `nhanDienNguoiDungTuyChon` gắn `req.user` nếu có token nhưng không chặn khách.
- **Phân quyền:** `chiAdmin` chỉ cho `vaiTro = admin` đi tiếp.
- **Xử lý lỗi:** middleware 404 cho route không tồn tại và middleware 4 tham số `(err, req, res, next)` trả lỗi 500.

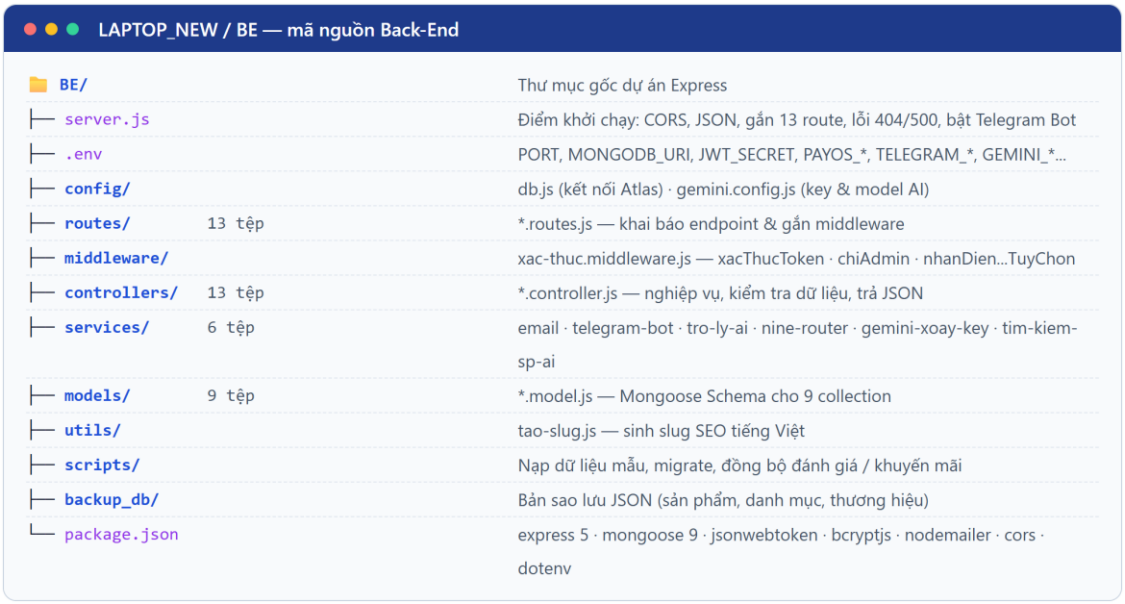
2.1.5. Vòng đời một request

Sơ đồ tuần tự sau mô tả đường đi của một request tới route cần bảo vệ và các nhánh lỗi có thể xảy ra ở từng tầng.



Hình 2.2. *Vòng đời xử lý một request qua các tầng của server*

2.2. Cấu trúc thư mục



Hình 2.3. *Cấu trúc thư mục mã nguồn Back-End*

2.2.1. config/

- `db.js`: hàm `ketNoiCoSoDuLieu()` kết nối MongoDB Atlas qua Mongoose, in hướng dẫn mở Network Access nếu lỗi.
- `gemini.config.js`: đọc danh sách API key từ `GEMINI_API_KEYS`, thứ tự mô hình ưu tiên và tham số sinh nội dung (temperature 0.35, tối đa 2048 token).

2.2.2. routes/

Gồm 13 tệp `*.routes.js`, mỗi tệp ứng với một nhóm tài nguyên và được gắn vào tiền tố tương ứng trong `server.js`: `/api/auth`, `/api/san-pham`, `/api/don-hang`, `/api/ma-giam-gia`, `/api/danh-gia`, `/api/nguoi-dung`, `/api/danh-muc`, `/api/tin-tuc`, `/api/bao-hanh`, `/api/thanh-toan`, `/api/lien-he`, `/api/cai-dat`, `/api/tro-ly-ai`.

Ví dụ khai báo route có gắn middleware phân quyền

```
// routes/san-pham.routes.js
router.get('/', layTatCaSanPham);                          // Công khai
router.get('/:id', laySanPhamTheoId);                      // Công khai
router.post('/', xacThucToken, chiAdmin, themSanPham);     // Chỉ Admin
router.put('/:id', xacThucToken, chiAdmin, capNhatSanPham); // Chỉ Admin
router.delete('/:id', xacThucToken, chiAdmin, xoaSanPham);  // Chỉ Admin
```

2.2.3. middleware/

`xac-thuc.middleware.js` chứa ba middleware `xacThucToken`, `chiAdmin`, `nhanDienNguoiDungTuyChon` (chi tiết ở mục 4.2).

2.2.4. controllers/

13 tệp controller chứa logic nghiệp vụ, ví dụ `xac-thuc.controller.js` (đăng ký, OTP, đăng nhập, Google, hồ sơ), `don-hang.controller.js` (tạo đơn, phân quyền xem đơn, trừ/hoàn kho), `thanh-toan.controller.js` (PayOS, VietQR, Webhook).

2.2.5. services/

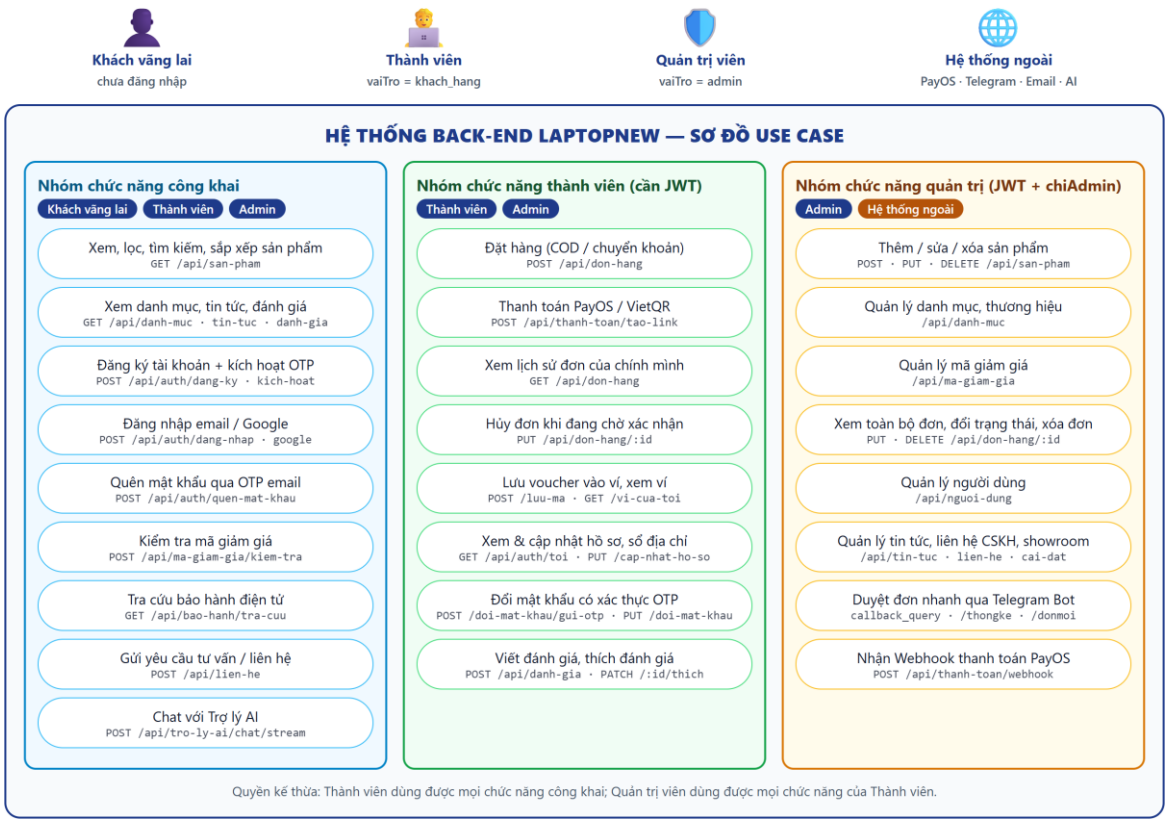
- `email.service.js`: gửi email qua Brevo API hoặc Gmail SMTP, dựng nội dung HTML cho OTP và hóa đơn.
- `telegram-bot.service.js`: gửi thông báo đơn, xử lý nút bấm, lệnh quản trị, vòng Long Polling.
- `tro-ly-ai.service.js`: điều phối Trợ lý AI; `nine-router.service.js` và `gemini-xoay-key.service.js`: gọi mô hình; `tim-kiem-san-pham-ai.service.js`: phân tích ý định và tìm sản phẩm phù hợp.

2.2.6. models/, utils/, scripts/

- `models/`: 9 tệp Schema Mongoose (mục 2.4).
- `utils/tao-slug.js`: chuyển tiêu đề tiếng Việt có dấu thành slug URL, ví dụ "ASUS Zenbook 14" → `asus-zenbook-14`.
- `scripts/`: các tập lệnh nạp dữ liệu mẫu, migrate và đồng bộ dữ liệu (chạy thủ công bằng `node scripts/<tên>.js`).

2.3. Sơ đồ Use Case

Sơ đồ use case thể hiện các chức năng chính theo ba nhóm quyền. Dưới mỗi use case là endpoint hiện thực chức năng đó.



Hình 2.4. *Sơ đồ use case của hệ thống Back-End LaptopNew*

2.4. Thiết kế cơ sở dữ liệu

2.4.1. Tổng quan các collection

Cơ sở dữ liệu `laptop_store` trên MongoDB Atlas gồm 9 collection được quản lý bằng Mongoose và 1 collection tạm `dang_ky_tam` được thao tác trực tiếp qua driver. Số lượng bản ghi dưới đây được lấy từ API thực tế ngày 29/09/2026.

Bảng 2.2. *Danh sách collection trong cơ sở dữ liệu*

Collection	Model / Tệp	Nội dung lưu trữ	Số bản ghi
`san_pham`	SanPham · san-pham.model.js	Laptop, phụ kiện: giá, thông số, phiên bản, tồn kho	61 (35 hãng)
`don_hang`	DonHang · don-hang.model.js	Đơn hàng, giao hàng, thanh toán, lịch trình	—
`nguoi_dung`	NguoiDung · nguoi-dung.model.js	Tài khoản, vai trò, OTP, sổ địa chỉ, ví voucher	—
`dang_ky_tam`	(driver trực tiếp)	Thông tin đăng ký chờ xác thực OTP, tự xóa khi hết hạn	tạm thời
`ma_giam_gia`	MaGiamGia · ma-giam-gia.model.js	Voucher giảm tiền mặt hoặc phần trăm	14

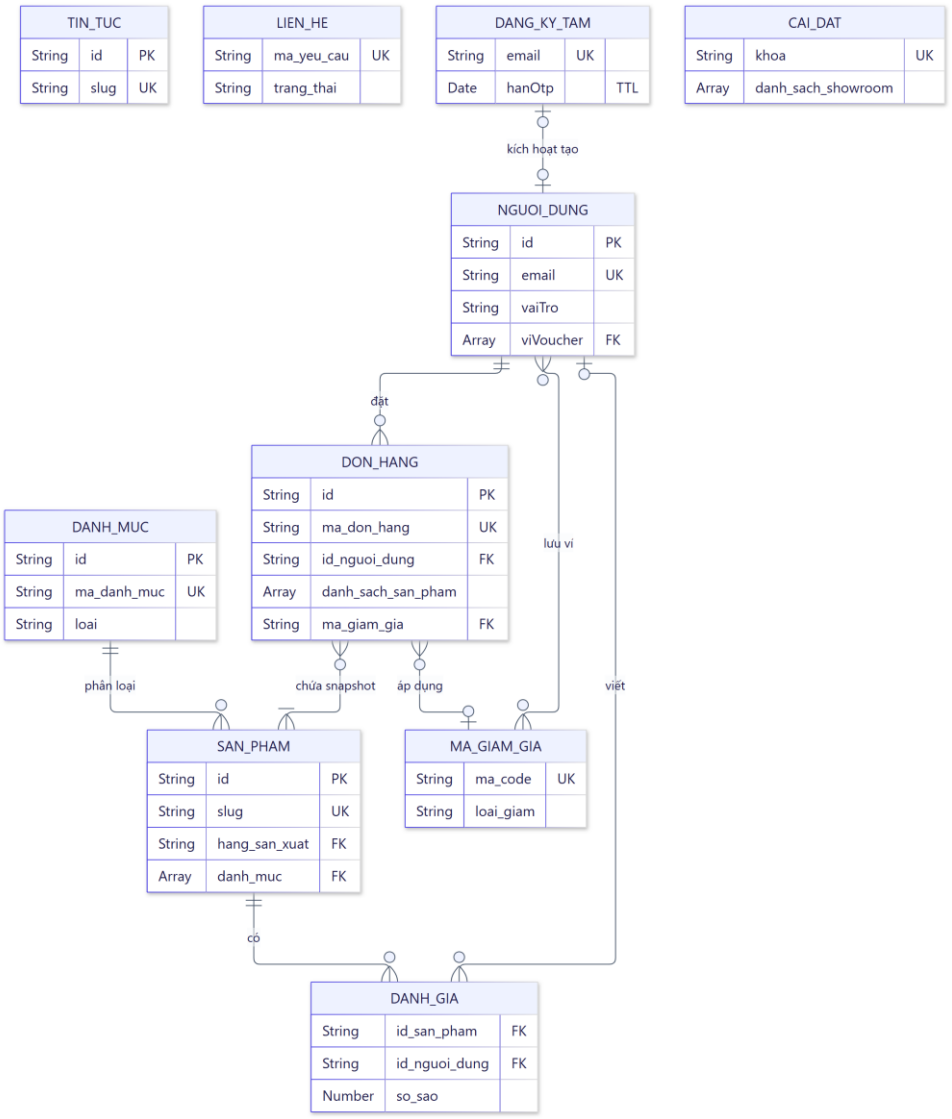
Collection	Model / Tệp	Nội dung lưu trữ	Số bản ghi
`danh_gia`	DanhGia · danh-gia.model.js	Đánh giá sao và bình luận sản phẩm	30
`danh-muc`	DanhMuc · danh-muc.model.js	Thương hiệu, nhu cầu, phân khúc	21
`tin_tuc`	TinTuc · tin-tuc.model.js	Bài viết tin tức công nghệ	10
`lien_he`	LienHe · lien-he.model.js	Phiếu yêu cầu tư vấn của khách	—
`cai_dat`	CaiDat · cai-dat.model.js	Cấu hình hệ thống: danh sách showroom, tọa độ	1 (3 showroom)

> **Ghi chú:** "—" là các collection chứa dữ liệu cá nhân hoặc chỉ admin mới xem được qua API, nên nhóm không thống kê trong báo cáo.

2.4.2. Sơ đồ quan hệ dữ liệu

MongoDB không có khóa ngoại như SQL; quan hệ được thể hiện bằng **tham chiếu theo mã** (lưu `id` của document khác) hoặc **nhúng (embedding)** dữ liệu vào trong document. Nhóm lựa chọn như sau:

- **Nhúng** thông số kỹ thuật, phiên bản cấu hình, quà tặng, tồn kho chi nhánh vào `san_pham` — vì luôn được đọc cùng sản phẩm.
- **Nhúng bản chụp (snapshot)** sản phẩm vào `danh_sach_san_pham` của đơn hàng — để giá và tên trong đơn không đổi khi sản phẩm được sửa sau này.
- **Tham chiếu** `id_nguoi_dung` trong đơn hàng và đánh giá; `id_san_pham` trong đánh giá; mảng mã `viVoucher` trong người dùng.



Hình 2.5. *Sơ đồ quan hệ giữa các collection (ERD logic)*

2.4.3. Collection san_pham

Bảng 2.3. *Cấu trúc collection san_pham*

Tên trường	Kiểu	Ràng buộc	Mô tả
`id`	String	Bắt buộc, duy nhất	Mã định danh, ví dụ `asus-zenbook-14-oled-ai`
`slug`	String	Duy nhất, index, tự sinh	Đường dẫn SEO sinh từ tên bằng hook `pre('validate')`
`ma_san_pham`	String	Bắt buộc	Mã part-number / SKU, ví dụ `UX3405MA-PP152W`
`ten_san_pham`	String	Bắt buộc	Tên thương mại kèm cấu hình nổi bật
`hang_san_xuat`	String	Bắt buộc, lowercase	Hãng: asus, dell, lenovo, apple, msi...
`danh_muc`	[String]	Mặc định []	Nhu cầu: gaming, van-phong-mong-nhe, ai-pc...

Tên trường	Kiểu	Ràng buộc	Mô tả
`gia_goc`	Number	Mặc định 0	Giá niêm yết (VNĐ)
`gia_khuyen_mai`	Number	Bắt buộc	Giá bán thực tế — dùng để lọc và sắp xếp
`phan_tram_giam_gia`	Number	Mặc định 0	Phần trăm giảm hiển thị
`hinh_anh_chinh`	String	Bắt buộc	Ảnh đại diện
`thu_vien_hinh_anh`	[String]	Mặc định []	Các ảnh chi tiết
`video_demo`	String	Mặc định ""	Video giới thiệu
`thong_so`	Mixed	Mặc định {}	Object thông số tự do: cpu, card_do_hoa, ram, o_cung, man_hinh, dung_luong_pin, chip_ai_npu...
`tuy_chon_phien_ban`	[Object]	Nhúng	`ma_tuy_chon`, `ten_tuy_chon`, `ram`, `o_cung`, `chenh_lech_gia`, `hinh_anh`
`qua_tang`	[Object]	Nhúng	`ten_qua_tang`, `gia_tri`
`ton_kho`	[Object]	Nhúng	Theo chi nhánh: `ten_chi_nhanh`, `dia_chi`, `so_dien_thoai`, `so_luong_con`
`danh_gia`	[Object]	Nhúng	Đánh giá rút gọn hiển thị nhanh trên trang chi tiết
`dac_diem_noi_bat`	[String]	Mặc định []	Các điểm nổi bật
`mo_ta_chi_tiet`	String	Mặc định ""	Bài mô tả dài
`so_luong_ton_kho`	Number	Mặc định 10	Tổng tồn kho — bị trừ khi đơn giao thành công
`so_luong_da_ban`	Number	Mặc định 0	Số đã bán — dùng sắp xếp "bán chạy"
`con_hang`	Boolean	Mặc định true	Tự chuyển `false` khi tồn kho về 0
`diem_danh_gia_tb`, `so_luong_danh_gia`	Number	Mặc định 5 / 0	Điểm trung bình và số lượt đánh giá
`la_flash_sale`, `la_ban_chay`, `la_san_pham_moi`	Boolean	Mặc định false	Cờ phục vụ các khối trên trang chủ
`ho_tro_tra_gop_0_phan_tram`	Boolean	Mặc định true	Hỗ trợ trả góp 0%
`createdAt, updatedAt`	Date	Tự động	Do tùy chọn `timestamps: true`

Hook tự sinh slug trước khi kiểm tra dữ liệu

```
// models/san-pham.model.js (trích)
SanPhamSchema.pre('validate', function () {
    if (!this.slug || this.isModified('ten_san_pham')) {
        this.slug = taoSlug(this.ten_san_pham) || this.id;
    }
});
const SanPham = mongoose.model('SanPham', SanPhamSchema, 'san_pham');
```

2.4.4. Collection don_hang

Bảng 2.4. *Cấu trúc collection don_hang*

Tên trường	Kiểu	Ràng buộc	Mô tả
`id`	String	Bắt buộc, duy nhất	Tự sinh `dh-<timestamp>` nếu client không gửi
`ma_don_hang`	String	Bắt buộc, duy nhất, index	Mã hiển thị, dạng `LPN-2026-xxxx`
`id_nguoi_dung`	String	Index	Chủ đơn — lấy từ token khi có
`ngay_tao`	String	Bắt buộc	Thời điểm đặt theo định dạng vi-VN
`trang_thai`	String	Enum	`cho_xac_nhan` · `da_xac_nhan` · `dang_giao` · `da_giao` · `da_huy`
`thong_tin_giao_hang`	Object	SĐT bắt buộc	`ho_ten`, `so_dien_thoai`, `email`, `dia_chi_chi_tiet`, `tinh_thanh`, `quan_huyen`, `phuong_xa`, `ghi_chu`
`danh_sach_san_pham`	[Object]	Nhúng	`san_pham_id`, `ten_san_pham`, `hinh_anh`, `so_luong`, `gia`, `tong_tien_muc`, `tuy_chon_chon`, snapshot `san_pham`
`tam_tinh`	Number	Bắt buộc	Tổng tiền hàng trước giảm giá
`tien_giam_gia / ma_giam_gia`	Number / String	Mặc định 0 / ""	Số tiền giảm và mã voucher đã áp dụng
`phi_van_chuyen`	Number	Mặc định 0	Phí giao hàng
`tong_tien_thanh_toan`	Number	Bắt buộc	Số tiền khách phải trả
`hinh_thuc_thanh_toan`	String	Mặc định `tien_mat_cod`	COD, chuyển khoản, PayOS
`da_thanh_toan`	Boolean	Mặc định false, index	Đã nhận tiền hay chưa
`trang_thai_thanh_toan`	String	Enum	`chua_thanh_toan` · `cho_thanh_toan` · `da_thanh_toan` · `that_bai` · `thanh_toan_khi_nhan_hang`
`payos_order_code`	Number	Index	Mã giao dịch PayOS để đối soát Webhook
`noi_dung_chuyen_khoan`	String		Nội dung CK, dạng `TRIKUN xxxxxx`
`da_tru_ton_kho`	Boolean	Mặc định false	Cờ chống trừ kho hai lần
`lich_trinh_giao_hang`	[Object]	Nhúng	`thoi_gian`, `tieu_de`, `mo_ta_chi_tiet`, `hoan_thanh`

2.4.5. Collection nguoi_dung và dang_ky_tam

Bảng 2.5. *Cấu trúc collection nguoi_dung*

Tên trường	Kiểu	Ràng buộc	Mô tả
`id`	String	Bắt buộc, duy nhất	Mã người dùng

Tên trường	Kiểu	Ràng buộc	Mô tả
`hoTen`	String	Bắt buộc	Họ và tên
`email`	String	Bắt buộc, duy nhất, lowercase	Dùng làm tên đăng nhập
`matKhau`	String	Bắt buộc	Chuỗi băm bcrypt; bị loại khỏi JSON trả về nhờ `toJSON()`
`soDienThoai, avatar`	String	Mặc định ""	Liên lạc, ảnh đại diện
`vaiTro`	String	Enum	`khach_hang` (mặc định) hoặc `admin` — nền tảng RBAC
`hangThanhVien`	String	Mặc định	Hạng thành viên, ví dụ "Thành Viên Mới"
`viVoucher`	[String]	Mặc định []	Mã voucher đã lưu
`gioiTinh, ngaySinh`	String	Enum / ""	`nam` · `nu` · `khac`
`danhSachDiaChi`	[Object]	Nhúng	Sổ địa chỉ: người nhận, SĐT, tỉnh/quận/phường, `macDinh`, `loaiDiaChi`
`googleId, authProvider`	String	Enum	`local` hoặc `google`
`maOtp, hanOtp, loaiOtp`	String, Date, String	Mặc định null	OTP hiện hành, hạn 10 phút, loại: `quen_mat_khau` · `kich_hoat` · `doi_mat_khau`
`daKichHoat`	Boolean	Mặc định true	Tài khoản đã xác thực email
`trangThai, biKhoa, lyDoKhoa`	String, Boolean, String		Khóa tài khoản — đăng nhập bị từ chối với mã 403

Collection `dang_ky_tam` lưu thông tin đăng ký chưa xác thực gồm `hoTen`, `email`, `soDienThoai`, `matKhau` (đã băm), `maOtp`, `hanOtp`, `loaiOtp`. Collection có **TTL index** trên `hanOtp` với `expireAfterSeconds: 0`, nên MongoDB tự xóa bản ghi khi OTP hết hạn. Nhờ vậy, bảng `nguoi_dung` chỉ chứa tài khoản đã xác thực, không bị đầy tài khoản rác.

2.4.6. Collection ma_giam_gia

Bảng 2.6. *Cấu trúc collection ma_giam_gia*

Tên trường	Kiểu	Ràng buộc	Mô tả
`ma_code`	String	Bắt buộc, duy nhất, uppercase	Mã khách nhập, ví dụ `TNTP3TR`
`tieu_de, mo_ta`	String	tieu_de bắt buộc	Nội dung hiển thị
`loai_giam`	String	Enum	`tien_mat` (mặc định) hoặc `phan_tram`
`gia_tri_giam`	Number	Bắt buộc	Số tiền hoặc số phần trăm giảm
`giam_toi_da`	Number	Mặc định 0	Trần giảm cho loại phần trăm
`don_hang_toi_thieu`	Number	Mặc định 0	Tạm tính tối thiểu để áp dụng

Tên trường	Kiểu	Ràng buộc	Mô tả
`ngay_het_han`	String	Mặc định ""	Thông tin hạn dùng hiển thị
`kich_hoat`	Boolean	Mặc định true	Chỉ mã đang bật mới kiểm tra hợp lệ
`chuyen_muc, badge, hot_deal, mau_sac`	Mixed		Thông tin trình bày trên trang khuyến mãi
`so_luong_phat_hanh, so_luong_con_lai, phan_tram_da_dung, khung_gio_san`	Number / String		Thông tin "săn mã" theo khung giờ

2.4.7. Collection danh_gia

Bảng 2.7. *Cấu trúc collection danh_gia*

Tên trường	Kiểu	Ràng buộc	Mô tả
`id_san_pham`	String	Bắt buộc	Sản phẩm được đánh giá
`id_nguoi_dung`	String		Người viết (nếu đăng nhập)
`ho_ten`	String	Bắt buộc	Tên hiển thị
`so_sao`	Number	Bắt buộc, 1–5	Số sao
`tieu_de, noi_dung`	String	noi_dung bắt buộc	Nội dung nhận xét
`hinh_anh_dinh_kem`	[String]	Mặc định []	Ảnh khách gửi kèm
`luot_thich`	Number	Mặc định 0	Tăng qua `PATCH /:id/thich`
`da_mua_hang`	Boolean	Mặc định true	Nhãn "Đã mua hàng"
`ngheNghiep, avatar, diaDiemMua, thoi_gian_dung, ngay_danh_gia`	String		Thông tin bổ sung khi hiển thị

2.4.8. Collection danh-muc, tin_tuc, lien_he, cai_dat

Bảng 2.8. *Cấu trúc các collection danh-muc, tin_tuc, lien_he, cai_dat*

Collection	Trường chính	Ràng buộc nổi bật
`danh-muc`	`id`, `slug`, `ma_danh_muc`, `ten_danh_muc`, `loai`, `logo`, `mo_ta`, `xuat_xu`, `kich_hoat`, `thu_tu_hien_thi`	`ma_danh_muc` duy nhất, lowercase; `loai` ∈ {`thuong_hieu`, `nhu_cau`, `phan_khuc`}; slug tự sinh
`tin_tuc`	`id`, `slug`, `tieu_de`, `tom_tat`, `noi_dung`, `hinh_anh`, `ngay_dang`, `tac_gia`, `chuyen_muc`, `thoi_gian_doc`, `luot_xem`, `la_tieu_diem`, `xuat_ban`, `tags`	`tieu_de`, `tom_tat` bắt buộc; slug duy nhất tự sinh
`lien_he`	`ma_yeu_cau`, `ho_ten`, `so_dien_thoai`, `email`, `chu_de`, `noi_dung`, `trang_thai`, `ghi_chu_noi_bo`, `nguon_tiep_nhan`	`ma_yeu_cau` duy nhất; thông báo lỗi tiếng Việt cho trường bắt buộc; `trang_thai` ∈ {`chua_xu_ly`, `dang_xu_ly`, `da_hoan_thanh`, `da_huy`}; index {SĐT, ngày tạo}

Collection	Trường chính	Ràng buộc nổi bật
`cai_dat`	`khoa` (mặc định `he_thong`), `danh_sach_showroom[]`: `id`, `ten`, `dia_chi`, `hotline`, `gio_mo_cua`, `vi_do`, `kinh_do`, `la_mac_dinh`	`khoa` duy nhất (một bản ghi cấu hình); tọa độ bắt buộc để vẽ bản đồ

2.4.9. Chỉ mục (Index) và ràng buộc dữ liệu

Bảng 2.9. *Các chỉ mục chính trong cơ sở dữ liệu*

Collection	Chỉ mục	Mục đích
san_pham	`id` (unique), `slug` (unique, sparse)	Tra chi tiết theo id hoặc slug SEO
don_hang	`id`, `ma_don_hang` (unique); `payos_order_code`, `id_nguoi_dung`, `da_thanh_toan`	Tra đơn nhanh, đối soát Webhook, lọc đơn theo người dùng
nguoi_dung	`id`, `email` (unique)	Không cho trùng email khi đăng ký
dang_ky_tam	`hanOtp` (TTL, expireAfterSeconds = 0)	Tự xóa đăng ký chưa xác thực khi hết hạn
ma_giam_gia	`ma_code` (unique)	Mỗi mã chỉ tồn tại một lần
lien_he	`ma_yeu_cau` (unique), `trang_thai`, {`so_dien_thoai`, `ngay_tao`}	Lọc phiếu theo trạng thái, tra theo SĐT

CHƯƠNG 3. ĐẶC TẢ HỆ THỐNG API

3.1. Quy ước chung

Bảng 3.1. *Quy ước chung của API*

Hạng mục	Quy ước
Địa chỉ gốc (Base URL)	`http://localhost:5000` khi phát triển; biến `NEXT_PUBLIC_API_URL` ở Front-End trỏ tới domain triển khai
Tiền tố	Mọi tài nguyên nằm dưới `/api/<tên-nhóm>`; tên nhóm viết thường, không dấu, nối bằng gạch ngang
Định dạng dữ liệu	Request và response dùng JSON (`Content-Type: application/json`); riêng stream AI dùng NDJSON
Xác thực	Header `Authorization: Bearer <JWT>` nhận được sau khi đăng nhập / kích hoạt
Phương thức	`GET` đọc · `POST` tạo mới / hành động · `PUT` cập nhật toàn phần · `PATCH` cập nhật một phần · `DELETE` xóa
Thông báo lỗi	Trả kèm mã HTTP phù hợp và trường `thong_diep` bằng tiếng Việt (một số nhóm dùng `message` / `thongBao`)
Định danh linh hoạt	Các API `/:id` chấp nhận `id` chuỗi, `slug` hoặc `_id` ObjectId của MongoDB

3.1.1. Các mức quyền truy cập

Bảng 3.2. *Ký hiệu mức quyền dùng trong các bảng đặc tả*

Ký hiệu	Middleware	Ý nghĩa
Công khai	(không có)	Ai cũng gọi được
Tùy chọn	`nhanDienNguoiDungTuyChon`	Không bắt buộc token; nếu có token hợp lệ thì hệ thống biết người gọi là ai để lọc dữ liệu
JWT	`xacThucToken`	Bắt buộc token hợp lệ, nếu không trả 401
JWT + Admin	`xacThucToken` + `chiAdmin`	Bắt buộc token của tài khoản `vaiTro = admin`, nếu không trả 401 / 403

3.2. Bảng tổng hợp API

Hệ thống khai báo **69 route** trong 13 tệp route và `server.js`. Một số route là **bí danh (alias)** trỏ cùng một hàm để tương thích Front-End cũ (ví dụ `/tao-link` và `/tao_link`), nên số endpoint chức năng khác nhau là **63**.

Bảng 3.3. *Thống kê số route theo nhóm chức năng*

STT	Tiền tố	Nhóm chức năng	Số route	Tệp route
1	`/api/auth`	Xác thực & tài khoản	13	xac-thuc.routes.js
2	`/api/san-pham`	Sản phẩm	5	san-pham.routes.js
3	`/api/danh-muc`	Danh mục & thương hiệu	4	danh-muc.routes.js
4	`/api/don-hang`	Đơn hàng	5	don-hang.routes.js
5	`/api/thanh-toan`	Thanh toán PayOS / VietQR	6 (3 alias)	thanh-toan.routes.js
6	`/api/ma-giam-gia`	Mã giảm giá & ví voucher	7	ma-giam-gia.routes.js
7	`/api/danh-gia`	Đánh giá sản phẩm	4	danh-gia.routes.js
8	`/api/nguoi-dung`	Quản trị người dùng	4	nguoi-dung.routes.js
9	`/api/tin-tuc`	Tin tức	5	tin-tuc.routes.js
10	`/api/bao-hanh`	Tra cứu bảo hành	2 (1 alias)	bao-hanh.routes.js
11	`/api/lien-he`	Liên hệ / CSKH	5 (1 alias)	lien-he.routes.js
12	`/api/cai-dat`	Cấu hình showroom	3	cai-dat.routes.js
13	`/api/tro-ly-ai`	Trợ lý AI	4 (1 alias)	tro-ly-ai.routes.js
14	`/`, `/api/health`	Giám sát máy chủ	2	server.js
		Tổng cộng	**69**	

3.3. Đặc tả chi tiết từng nhóm API

3.3.1. Nhóm Xác thực và Tài khoản — /api/auth

Bảng 3.4. *Đặc tả API nhóm /api/auth*

Method	Endpoint	Quyền	Tham số / Body	Phản hồi	Chức năng
POST	/dang-ky	Công khai	`hoTen`, `email`, `soDienThoai`, `matKhau`	200 / 400 / 502	Kiểm tra dữ liệu, băm mật khẩu, lưu `dang_ky_tam`, gửi OTP kích hoạt
POST	/kich-hoat	Công khai	`email`, `otp`	200 / 400 / 404	Xác thực OTP, tạo tài khoản chính thức, trả JWT
POST	/dang-nhap	Công khai	`email`, `matKhau`	200 / 400 / 401 / 403	So khớp bcrypt, kiểm tra khóa, trả `token` + `nguoiDung`
POST	/dang-xuat	Công khai	—	200	Xác nhận đăng xuất (client xóa token)
POST	/google	Công khai	`credential` (Google ID Token), `email`, `hoTen`, `avatar`	200 / 400 / 401 / 403	Xác minh token Google, tạo hoặc liên kết tài khoản, trả JWT

Method	Endpoint	Quyền	Tham số / Body	Phản hồi	Chức năng
POST	/quen-mat-khau	Công khai	`email`	200 / 404 / 502	Sinh OTP loại `quen_mat_khau`, gửi email
POST	/gui-otp	Công khai	`email`	200 / 404	Gửi lại OTP (đăng ký hoặc quên mật khẩu)
POST	/xac-nhan-otp	Công khai	`email`, `otp`	200 / 400 / 404	Kiểm tra OTP còn hạn trước khi cho đặt mật khẩu mới
POST	/dat-lai-mat-khau	Công khai	`email`, `otp`, `matKhauMoi` (≥ 6 ký tự)	200 / 400 / 404	Băm và lưu mật khẩu mới, xóa OTP
GET	/toi	JWT	Header Bearer	200 / 401	Lấy hồ sơ người đang đăng nhập
PUT	/cap-nhat-ho-so	JWT	`hoTen`, `soDienThoai`, `avatar`, `gioiTinh`, `ngaySinh`, `danhSachDiaChi`	200 / 404	Cập nhật hồ sơ và sổ địa chỉ
POST	/doi-mat-khau/gui-otp	JWT	`matKhauCu` (nếu tài khoản có mật khẩu)	200 / 400 / 502	Kiểm tra mật khẩu cũ, gửi OTP loại `doi_mat_khau`
PUT	/doi-mat-khau	JWT	`matKhauCu`, `matKhauMoi`, `otp`	200 / 400	Đổi mật khẩu khi OTP hợp lệ

Ví dụ request / response đăng nhập

```
POST /api/auth/dang-nhap
Content-Type: application/json

{ "email": "khach@example.com", "matKhau": "matkhau123" }

HTTP/1.1 200 OK
{
  "thong_diep": "Đăng nhập thành công! Xin chào Nguyễn Văn A.",
  "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1c2VySWQiOi...",
  "nguoiDung": { "id": "...", "hoTen": "Nguyễn Văn A", "email": "khach@example.com",
"vaiTro": "khach_hang" }
}
```

3.3.2. Nhóm Sản phẩm — /api/san-pham

Bảng 3.5. *Đặc tả API nhóm /api/san-pham*

Method	Endpoint	Quyền	Tham số / Body	Phản hồi	Chức năng
GET	/	Công khai	Query: `tu_khoa`/`q`, `hang_san_xuat`, `danh_muc` (nhiều giá trị cách bằng dấu phẩy), `gia_min`, `gia_max`, `flash_sale`, `ban_chay`, `moi_nhat`, `sap_xep`, `limit`	200	Tìm kiếm, lọc, sắp xếp; có header cache 60 giây
GET	/:id	Công khai	Param: `id` / `slug` / `ma_san_pham` / `_id`	200 / 404	Chi tiết một sản phẩm

Method	Endpoint	Quyền	Tham số / Body	Phản hồi	Chức năng
POST	/	JWT + Admin	Document sản phẩm	201 / 409	Thêm sản phẩm; trùng id trả 409
PUT	/:id	JWT + Admin	Các trường cần sửa	200 / 404	Cập nhật sản phẩm
DELETE	/:id	JWT + Admin	Param: `id`	200 / 404	Xóa sản phẩm

Bảng 3.6. *Giá trị của tham số sap_xep*

Giá trị	Sắp xếp MongoDB	Ý nghĩa
`gia_tang_dan`	`{ gia_khuyen_mai: 1 }`	Giá thấp đến cao
`gia_giam_dan`	`{ gia_khuyen_mai: -1 }`	Giá cao đến thấp
`ban_chay_nhat`	`{ so_luong_da_ban: -1 }`	Bán chạy nhất
`danh_gia_cao`	`{ diem_danh_gia_tb: -1 }`	Đánh giá cao nhất
`moi_nhat`	`{ la_san_pham_moi: -1, createdAt: -1 }`	Hàng mới về

Kết quả thực tế khi lọc laptop ASUS dưới 30 triệu

```
GET /api/san-pham?hang_san_xuat=asus&gia_max=30000000&sap_xep=gia_tang_dan&limit=1

HTTP/1.1 200 OK
[{
  "id": "asus-zenbook-14-oled-ai",
  "ma_san_pham": "UX3405MA-PP152W",
  "ten_san_pham": "ASUS Zenbook 14 OLED (2026) Intel Core Ultra 7 155H | 32GB | 1TB ...",
  "hang_san_xuat": "asus",
  "gia_goc": 34990000, "gia_khuyen_mai": 29990000, "phan_tram_giam_gia": 14,
  "thong_so": { "cpu": "Intel® Core™ Ultra 7 155H (16 Nhân 22 Luồng...)", "ram": "32GB
LPDDR5X 7467MHz", ... },
  ...
}]
```

3.3.3. Nhóm Danh mục — /api/danh-muc

Bảng 3.7. *Đặc tả API nhóm /api/danh-muc*

Method	Endpoint	Quyền	Tham số / Body	Phản hồi	Chức năng
GET	/	Công khai	Query: `loai` (thuong_hieu / nhu_cau / phan_khuc)	200	Danh sách danh mục, sắp theo thứ tự hiển thị
POST	/	JWT + Admin	`ma_danh_muc`, `ten_danh_muc`, ...	201 / 400 / 409	Tạo danh mục; thiếu mã/tên trả 400, trùng mã trả 409
PUT	/:id	JWT + Admin	Các trường cần sửa	200 / 404	Cập nhật danh mục
DELETE	/:id	JWT + Admin	Param: `id`	200 / 404	Xóa danh mục

3.3.4. Nhóm Đơn hàng — /api/don-hang

Bảng 3.8. *Đặc tả API nhóm /api/don-hang*

Method	Endpoint	Quyền	Tham số / Body	Phản hồi	Chức năng
POST	/	Tùy chọn	`thong_tin_giao_hang`, `danh_sach_san_pham`, `tam_tinh`, `tien_giam_gia`, `ma_giam_gia`, `phi_van_chuyen`, `tong_tien_thanh_toan`, `hinh_thuc_thanh_toan`	201 / 400 / 401	Tạo đơn (bắt buộc xác định được người dùng), gửi email và thông báo Telegram
GET	/	Tùy chọn	Query: `trang_thai`, `id_nguoi_dung`, `email`, `sdt`	200	Admin: tất cả đơn · Thành viên: chỉ đơn của mình · Khách: phải truyền điều kiện, nếu không trả `[]`
GET	/:id	Tùy chọn + giới hạn	Param: `id` / `ma_don_hang` / orderCode / `_id`; Query: `sdt` (bắt buộc với khách)	200 / 403 / 404 / 429	Admin, chủ đơn: xem đầy đủ. Khách: phải nhập đúng mã đơn + SĐT nhận hàng, dữ liệu cá nhân được che; tối đa 30 lần / 10 phút / IP
PUT	/:id	JWT	Admin: mọi trường · Chủ đơn: trường thanh toán, hoặc `trang_thai = da_huy` khi đơn chờ xác nhận	200 / 403 / 404	Cập nhật trạng thái, tự trừ/hoàn tồn kho
DELETE	/:id	JWT + Admin	Param: `id`	200 / 404	Xóa đơn; hoàn kho nếu đơn đã trừ kho

3.3.5. Nhóm Thanh toán — /api/thanh-toan

Bảng 3.9. *Đặc tả API nhóm /api/thanh-toan*

Method	Endpoint	Quyền	Tham số / Body	Phản hồi	Chức năng
POST	/tao-link (alias /tao_link)	Công khai	`ma_don_hang`, `id_don_hang`, `tong_tien`, `ten_khach_hang`	200 / 500	Tạo yêu cầu PayOS + ảnh VietQR; trả `qrImageUrl`, `checkoutUrl`, `orderCode`, thông tin tài khoản
GET	/kiem-tra/:orderCode (alias /kiem_tra/:orderCode)	Công khai	Param: `orderCode`	200 / 400	Kiểm tra đã thanh toán chưa (DB → cache 3 giây → PayOS)
POST	/webhook (alias /payos-webhook)	PayOS	`{ data, signature }`	200 / 400	Xác minh chữ ký HMAC, đánh dấu đơn đã thanh toán

```
Ví dụ tạo yêu cầu thanh toán
POST /api/thanh-toan/tao-link
{ "ma_don_hang": "LPN-2026-4821", "tong_tien": 29990000, "ten_khach_hang": "Nguyen Van
A" }

HTTP/1.1 200 OK
{
  "success": true,
  "orderCode": 20264821,
  "amount": 29990000,
  "description": "TRIKUN 6-4821",
  "qrImageUrl": "https://img.vietqr.io/image/<BIN>-<STK>-
compact2.png?amount=29990000&addInfo=...",
  "checkoutUrl": "https://pay.payos.vn/web/...",
  "thongTinNganHang": { "nganHang": "KienlongBank", "soTien": 29990000, "noiDung":
"TRIKUN 6-4821", ... }
}
```

3.3.6. Nhóm Mã giảm giá — /api/ma-giam-gia

Bảng 3.10. *Đặc tả API nhóm /api/ma-giam-gia*

Method	Endpoint	Quyền	Tham số / Body	Phản hồi	Chức năng
GET	/	Công khai	Query: `kich_hoat`	200	Danh sách voucher
POST	/kiem-tra	Công khai	`ma_code`, `tong_tien_tam_tinh`	200 / 400 / 404	Kiểm tra mã và tính `so_tien_giam`
POST	/luu-ma	JWT	`ma_code`	200 / 400 / 404	Lưu voucher vào `viVoucher`
GET	/vi-cua-toi	JWT	—	200 / 404	Danh sách voucher trong ví
POST	/	JWT + Admin	Document voucher	201 / 400	Thêm voucher; trùng mã trả 400
PUT	/:id	JWT + Admin	Các trường cần sửa	200 / 404	Cập nhật, bật/tắt voucher
DELETE	/:id	JWT + Admin	Param: `id`	200 / 404	Xóa voucher

3.3.7. Nhóm Đánh giá — /api/danh-gia

Bảng 3.11. *Đặc tả API nhóm /api/danh-gia*

Method	Endpoint	Quyền	Tham số / Body	Phản hồi	Chức năng
GET	/ và /:productId	Công khai	Param `productId` hoặc query `id_san_pham`	200	Danh sách đánh giá của sản phẩm (hoặc tất cả)
POST	/	Công khai	`id_san_pham`, `ho_ten`, `so_sao`, `noi_dung`, ...	201 / 400	Thêm đánh giá, tính lại điểm trung bình của sản phẩm
PATCH	/:id/thich	Công khai	Param: `id`	200 / 404	Tăng lượt thích

3.3.8. Nhóm Người dùng — /api/nguoi-dung

Bảng 3.12. *Đặc tả API nhóm /api/nguoi-dung*

Method	Endpoint	Quyền	Tham số / Body	Phản hồi	Chức năng
GET	/	Công khai (*)	Query: `email`	200	Danh sách / tìm người dùng theo email
POST	/	Công khai (*)	Thông tin tài khoản	201 / 400	Tạo người dùng; trùng email trả 400
PUT	/:id	JWT + Admin	Các trường cần sửa	200 / 404	Cập nhật, khóa/mở khóa tài khoản
DELETE	/:id	JWT + Admin	Param: `id`	200 / 404	Xóa người dùng

3.3.9. Nhóm Tin tức — /api/tin-tuc

Bảng 3.13. *Đặc tả API nhóm /api/tin-tuc*

Method	Endpoint	Quyền	Tham số / Body	Phản hồi	Chức năng
GET	/	Công khai	Query: `chuyen_muc`, `q`, `la_tieu_diem`, `tat_ca`	200	Danh sách bài viết `{ thanhCong, duLieu }`
GET	/:id	Công khai	Param: `id` / `slug`	200 / 404	Chi tiết bài viết, tự tăng `luot_xem`
POST	/	Công khai (*)	Nội dung bài viết	201	Tạo bài viết
PUT	/:id	Công khai (*)	Các trường cần sửa	200 / 404	Cập nhật bài viết
DELETE	/:id	Công khai (*)	Param: `id`	200 / 404	Xóa bài viết

3.3.10. Nhóm Bảo hành — /api/bao-hanh

Bảng 3.14. *Đặc tả API nhóm /api/bao-hanh*

Method	Endpoint	Quyền	Tham số / Body	Phản hồi	Chức năng
GET	/ và /tra-cuu	Công khai	Query: `tu_khoa` (hoặc `q`) — mã đơn, SĐT, serial, mã sản phẩm	200	Tìm đơn khớp chính xác, tính hạn bảo hành 24 tháng, che SĐT

3.3.11. Nhóm Liên hệ — /api/lien-he

Bảng 3.15. *Đặc tả API nhóm /api/lien-he*

Method	Endpoint	Quyền	Tham số / Body	Phản hồi	Chức năng
POST	/	Công khai	`ho_ten`, `so_dien_thoai`, `email`, `chu_de`, `noi_dung`	201 / 400	Kiểm tra dữ liệu (SĐT Việt Nam), sinh `ma_yeu_cau`, lưu phiếu

Method	Endpoint	Quyền	Tham số / Body	Phản hồi	Chức năng
GET	/	Công khai (*)	Query: `trang`, `gioi_han`, `trang_thai`, `chu_de`, `tu_khoa`	200	Danh sách phiếu có phân trang và thống kê
PATCH / PUT	/:id	Công khai (*)	`trang_thai`, `ghi_chu_noi_bo`	200 / 404	Cập nhật tiến độ xử lý
DELETE	/:id	Công khai (*)	Param: `id`	200 / 404	Xóa phiếu

3.3.12. Nhóm Cài đặt showroom — /api/cai-dat

Bảng 3.16. *Đặc tả API nhóm /api/cai-dat*

Method	Endpoint	Quyền	Tham số / Body	Phản hồi	Chức năng
GET	/showroom	Công khai	—	200	Danh sách showroom và tọa độ GPS
PUT	/showroom	Công khai (*)	`danh_sach_showroom[]`	200 / 400	Lưu danh sách showroom (upsert bản ghi `he_thong`)
POST	/showroom/khoi-phuc	Công khai	—	410	Chức năng khôi phục mặc định đã ngừng hỗ trợ (410 Gone)

3.3.13. Nhóm Trợ lý AI — /api/tro-ly-ai

Bảng 3.17. *Đặc tả API nhóm /api/tro-ly-ai*

Method	Endpoint	Quyền	Tham số / Body	Phản hồi	Chức năng
POST	/chat/stream	Công khai	`tin_nhan`, `lich_su_chat[]`	200 (NDJSON) / 400	Trả từng đoạn văn bản, cuối cùng là kết quả đầy đủ kèm sản phẩm gợi ý
POST	/chat (alias /)	Công khai	`tin_nhan` (hoặc `message`), `lich_su_chat`	200 / 400 / 500	Trả toàn bộ câu trả lời một lần dạng JSON
GET	/trang-thai	Công khai	—	200	Trạng thái 9Router, số key Gemini, danh sách model, tình trạng từng key (đã che)

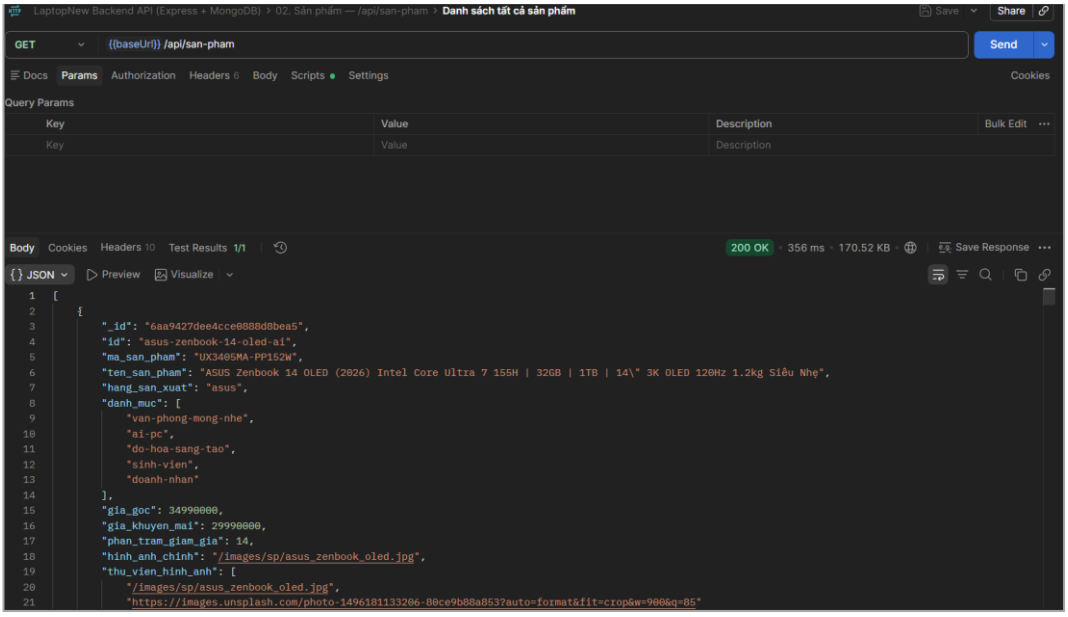
3.3.14. Nhóm Giám sát máy chủ

Bảng 3.18. *Đặc tả API giám sát máy chủ*

Method	Endpoint	Quyền	Tham số / Body	Phản hồi	Chức năng
GET	/	Công khai	—	200	Thông điệp server, phiên bản 1.1.0, mô hình, danh sách endpoint chính

Method	Endpoint	Quyền	Tham số / Body	Phản hồi	Chức năng
GET	/api/health	Công khai	—	200	`{ trang_thai: "OK", thoi_gian }` dùng cho health check khi triển khai

> **(*) Ghi chú:** các route đánh dấu (*) hiện chưa gắn middleware `xacThucToken` / `chiAdmin` dù đây là thao tác quản trị. Nhóm đã ghi nhận và đề xuất cách khắc phục ở mục 5.4.



Hình 3.1. *Kiểm thử GET /api/san-pham bằng Postman: trả về danh sách sản phẩm (200 OK)*

CHƯƠNG 4. HIỆN THỰC CÁC CHỨC NĂNG CHÍNH

Chương này trình bày cách hiện thực từng chức năng quan trọng theo cấu trúc thống nhất: **mục tiêu**, **luồng hoạt động**, **sơ đồ**, **đoạn mã tiêu biểu** và **hình ảnh kết quả**. Các đoạn mã được trích trực tiếp từ mã nguồn thư mục `BE/`, có lược bớt phần log để dễ đọc.

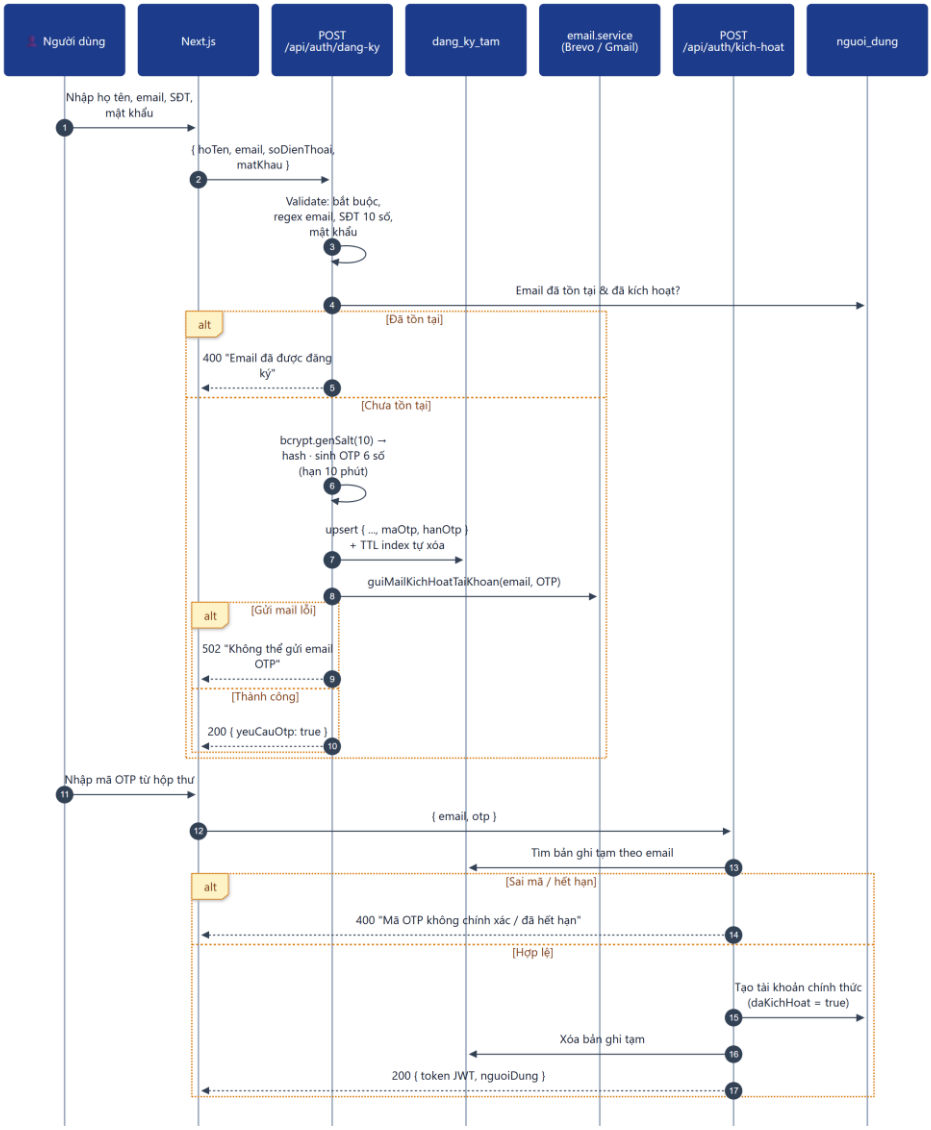
4.1. Chức năng đăng ký và kích hoạt tài khoản bằng OTP

❖ **Mục tiêu:**

- Thu thập họ tên, email, số điện thoại, mật khẩu và kiểm tra hợp lệ.
- Mã hóa mật khẩu trước khi lưu; chỉ tạo tài khoản chính thức sau khi người dùng chứng minh sở hữu email.
- Không để lại tài khoản rác trong `nguoi_dung` khi người dùng bỏ dở.

❖ **Luồng hoạt động:**

- **Bước 1:** Client gửi `POST /api/auth/dang-ky`. Server làm sạch dữ liệu (trim, lowercase email, bỏ khoảng trắng SĐT) và kiểm tra: đủ trường, họ tên ≥ 2 ký tự, email đúng định dạng, SĐT Việt Nam 10 số, mật khẩu hợp lệ.
- **Bước 2:** Nếu email đã có tài khoản kích hoạt → trả **400**.
- **Bước 3:** Băm mật khẩu bằng bcrypt (salt 10), sinh OTP 6 số hạn 10 phút, **upsert** vào collection tạm `dang_ky_tam` (có TTL index tự xóa).
- **Bước 4:** Gửi email kích hoạt chứa OTP. Nếu gửi thất bại → trả **502**, cho phép gửi lại qua `/gui-otp`.
- **Bước 5:** Người dùng nhập OTP → `POST /api/auth/kich-hoat`. Sai mã hoặc hết hạn → **400**; hợp lệ → tạo document trong `nguoi_dung` với `daKichHoat = true`, xóa bản ghi tạm và trả **JWT** để đăng nhập luôn.

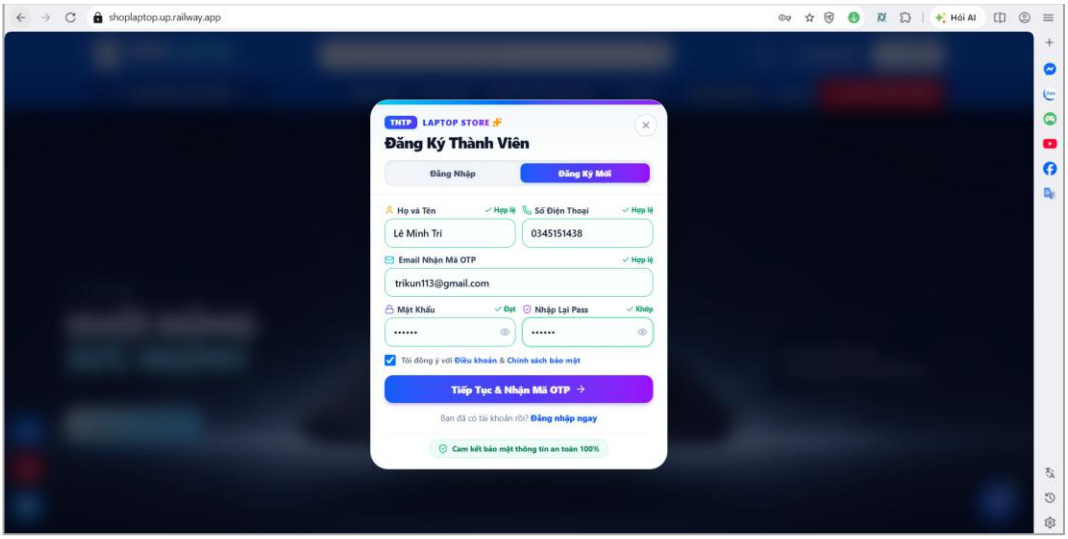


Hình 4.1. *Sơ đồ tuần tự đăng ký và kích hoạt tài khoản bằng OTP*

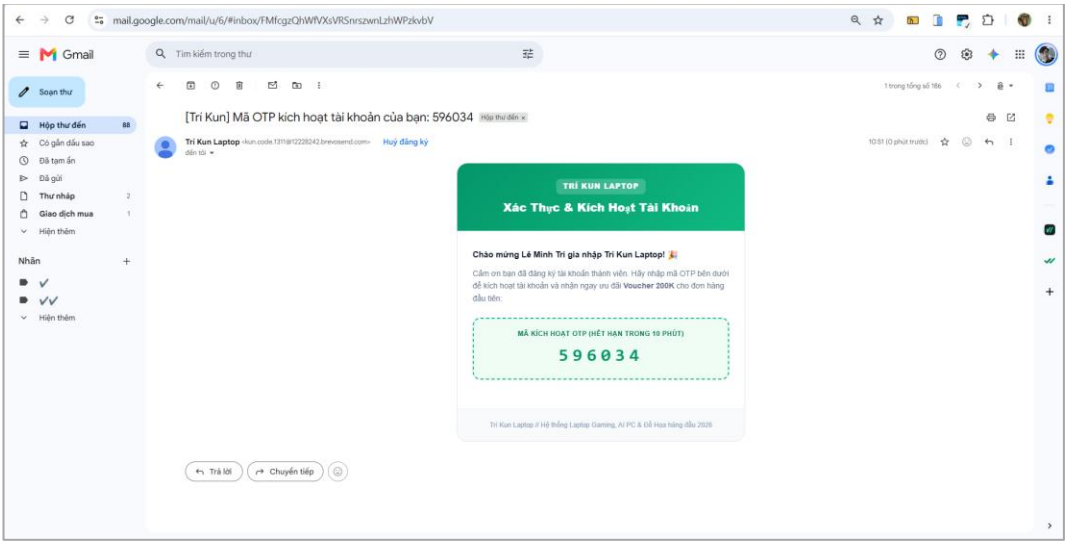
Lưu đăng ký tạm và gửi OTP

```
// controllers/xac-thuc.controller.js — dangKy (trích)
const salt = await bcrypt.genSalt(10);
const matKhauHash = await bcrypt.hash(mkClean, salt);
const maOtp = Math.floor(100000 + Math.random() * 900000).toString();
const hanOtp = new Date(Date.now() + 10 * 60 * 1000);            // 10 phút

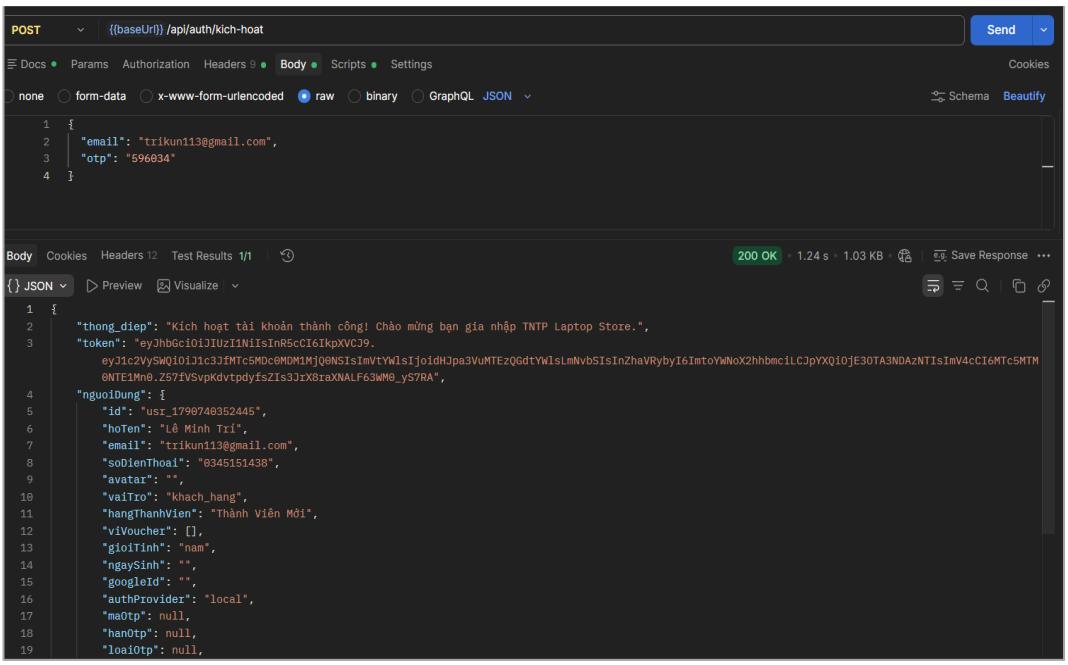
const bangTam = mongoose.connection.collection('dang_ky_tam');
bangTam.createIndex({ hanOtp: 1 }, { expireAfterSeconds: 0 });   // TTL: tự xóa khi hết
hạn
await bangTam.updateOne(
    { email: emailClean },
    { $set: { hoTen: hoTenClean, email: emailClean, soDienThoai: sdtClean,
              matKhau: matKhauHash, maOtp, hanOtp, loaiOtp: 'kich_hoat' } },
    { upsert: true }
);
const ketQuaGuiMail = await guiMailKichHoatTaiKhoan(emailClean, hoTenClean, maOtp);
if (!ketQuaGuiMail.thanhCong) return res.status(502).json({ thong_diep: 'Không thể gửi
email OTP...' });
```



Hình 4.2. *Giao diện form đăng ký tài khoản*



Hình 4.3. *Email chứa mã OTP kích hoạt tài khoản*



Hình 4.4. *Kiểm thử POST /api/auth/kich-hoat bằng Postman: kích hoạt thành công, nhận JWT*

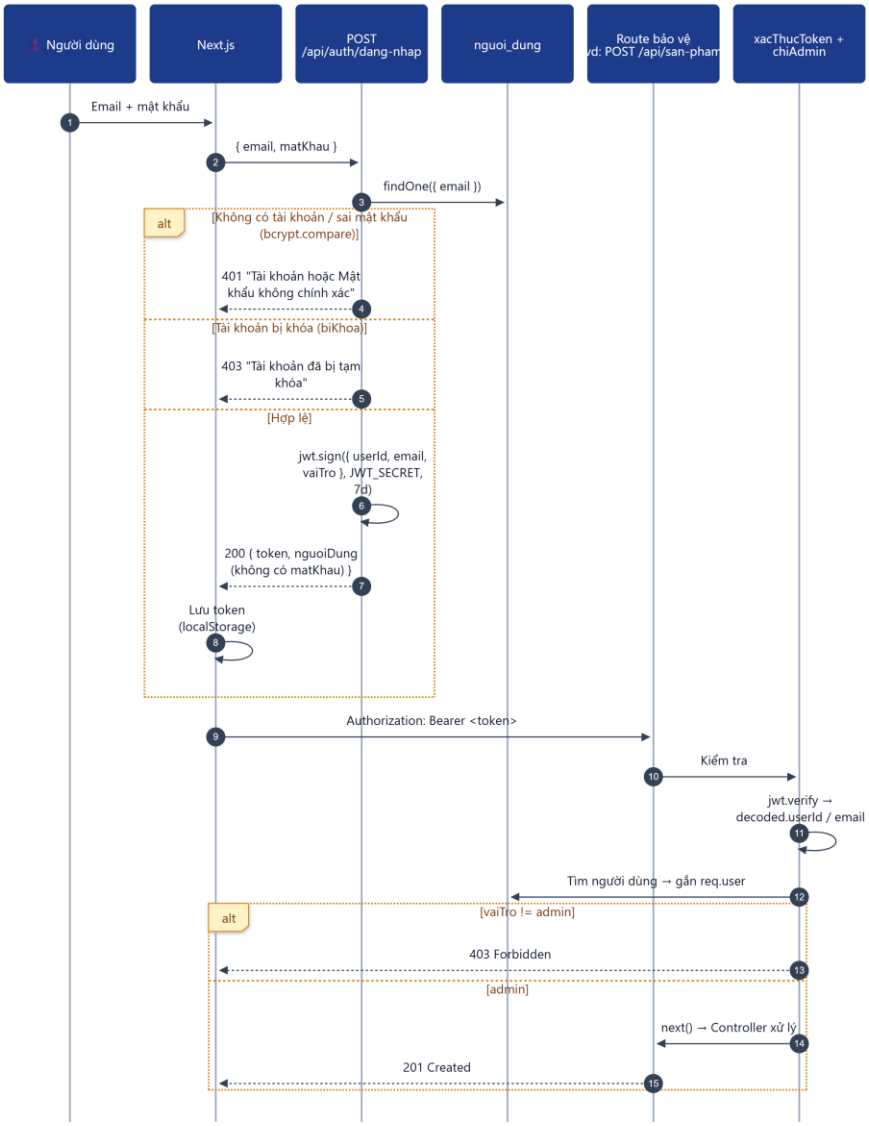
4.2. Chức năng đăng nhập, xác thực JWT và phân quyền

❖ Mục tiêu:

- Cho phép người dùng đăng nhập bằng email và mật khẩu, nhận JWT để gọi các API cần bảo vệ.
- Chặn tài khoản bị khóa; phân biệt quyền khách hàng và quản trị viên ở mọi route nhạy cảm.

❖ Luồng hoạt động:

- **Bước 1:** `POST /api/auth/dang-nhap` với `email`, `matKhau`. Thiếu trường → **400**.
- **Bước 2:** Tìm người dùng theo email; không có hoặc `bcrypt.compare` sai → **401** "Tài khoản hoặc Mật khẩu không chính xác!" (không tiết lộ trường nào sai).
- **Bước 3:** Tài khoản có `biKhoa` hoặc `trangThai = bi_khoa` → **403** kèm lý do khóa.
- **Bước 4:** Ký JWT chứa `userId`, `email`, `vaiTro`, hạn 7 ngày → trả **200** cùng thông tin người dùng (đã loại `matKhau`).
- **Bước 5:** Ở các lần gọi sau, `xacThucToken` giải mã token, tìm lại người dùng trong DB và gắn vào `req.user`; `chiAdmin` kiểm tra `req.user.vaiTro === 'admin'`.



Hình 4.5. *Sơ đồ tuần tự đăng nhập và gọi API được bảo vệ*

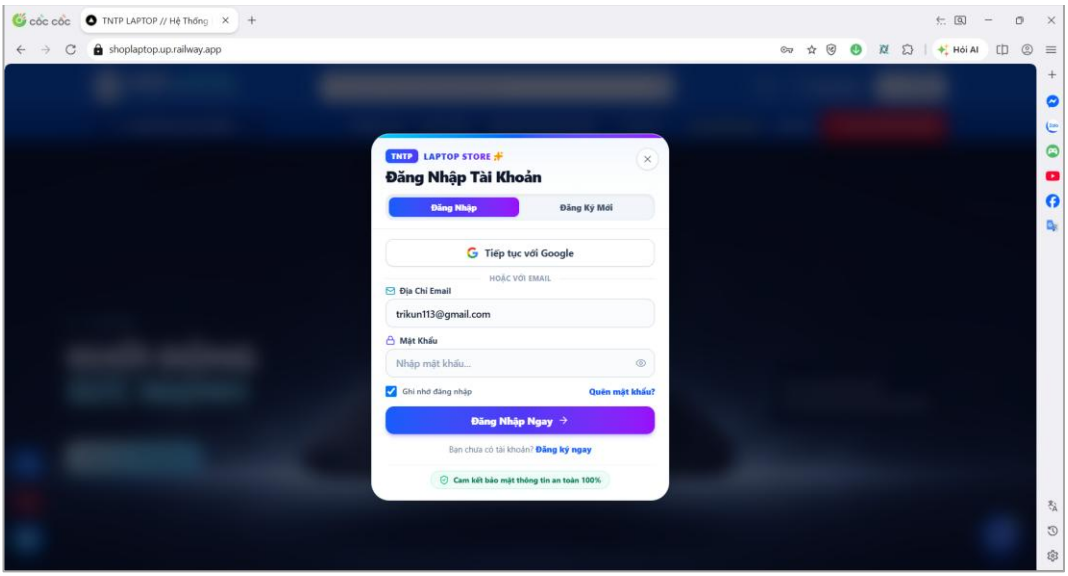
Tạo JWT và hai middleware xác thực, phân quyền

```
// Tạo token – controllers/xac-thuc.controller.js
const taoToken = (nguoiDung) => jwt.sign(
    { userId: nguoiDung.id, email: nguoiDung.email, vaiTro: nguoiDung.vaiTro },
    JWT_SECRET, { expiresIn: JWT_EXPIRES_IN }            // mặc định '7d'
);

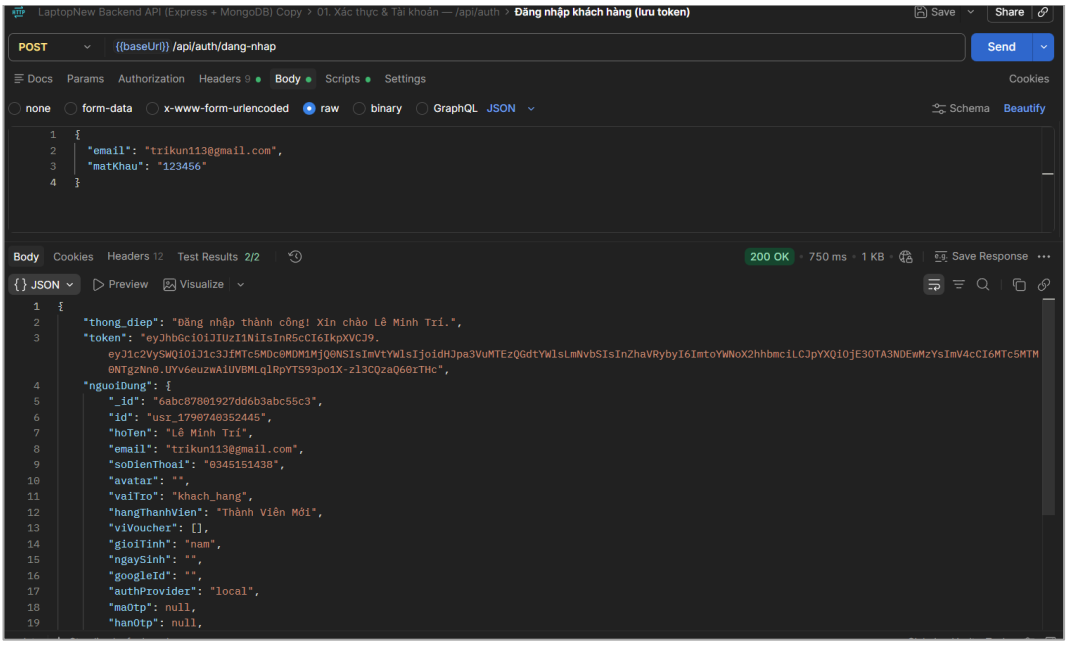
// middleware/xac-thuc.middleware.js
const xacThucToken = async (req, res, next) => {
    try {
        const header = req.headers.authorization;
        const token = header && header.startsWith('Bearer ') ? header.split(' ')[1] :
null;
        if (!token) return res.status(401).json({ thong_diep: 'Vui lòng đăng nhập ...
(Thiếu Token xác thực)!' });
        const decoded = jwt.verify(token, process.env.JWT_SECRET);
        const nguoiDung = await NguoiDung.findOne({ $or: [{ id: decoded.userId },
{ email: decoded.email }] });
        if (!nguoiDung) return res.status(401).json({ thong_diep: 'Tài khoản không tồn
tại hoặc đã bị khóa!' });
        req.user = nguoiDung;
        next();
```

```
    } catch (loi) {
        if (loi.name === 'TokenExpiredError')
            return res.status(401).json({ thong_diep: 'Phiên đăng nhập đã hết hạn, vui
lòng đăng nhập lại!' });
        return res.status(401).json({ thong_diep: 'Token xác thực không hợp lệ hoặc đã bị
thay đổi!' });
    }
};

const chiAdmin = (req, res, next) => {
    if (!req.user || req.user.vaiTro !== 'admin')
        return res.status(403).json({ thong_diep: 'Truy cập bị từ chối: Bạn không có
quyền Quản trị viên...' });
    next();
};
```



Hình 4.6. *Giao diện đăng nhập (email/mật khẩu và đăng nhập nhanh với Google)*



Hình 4.7. *Kiểm thử POST /api/auth/dang-nhap bằng Postman: đăng nhập thành công, nhận token JWT*

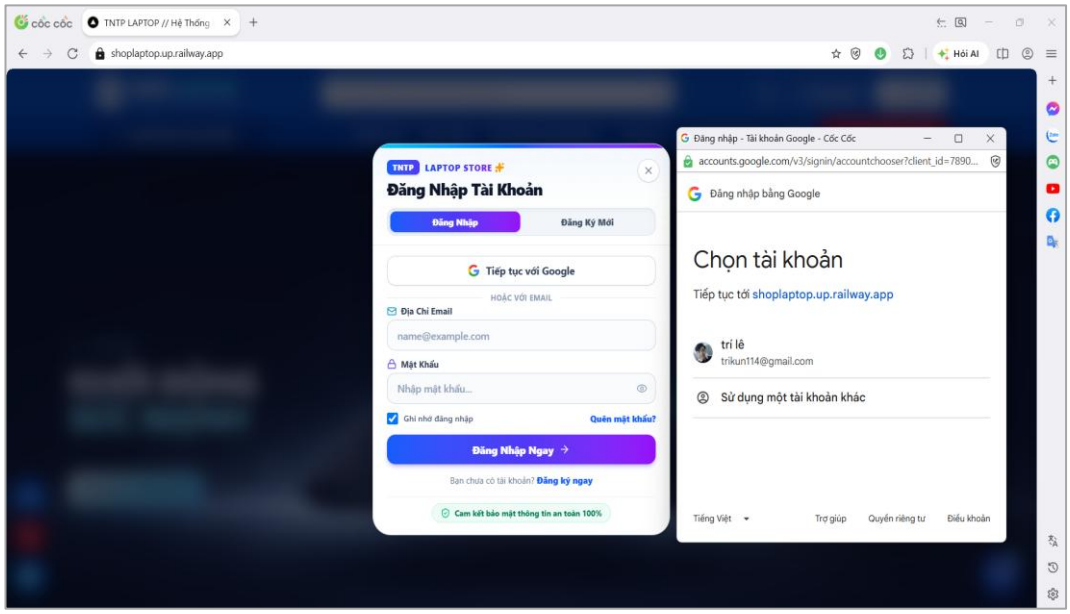
4.3. Chức năng đăng nhập bằng Google

❖ Mục tiêu:

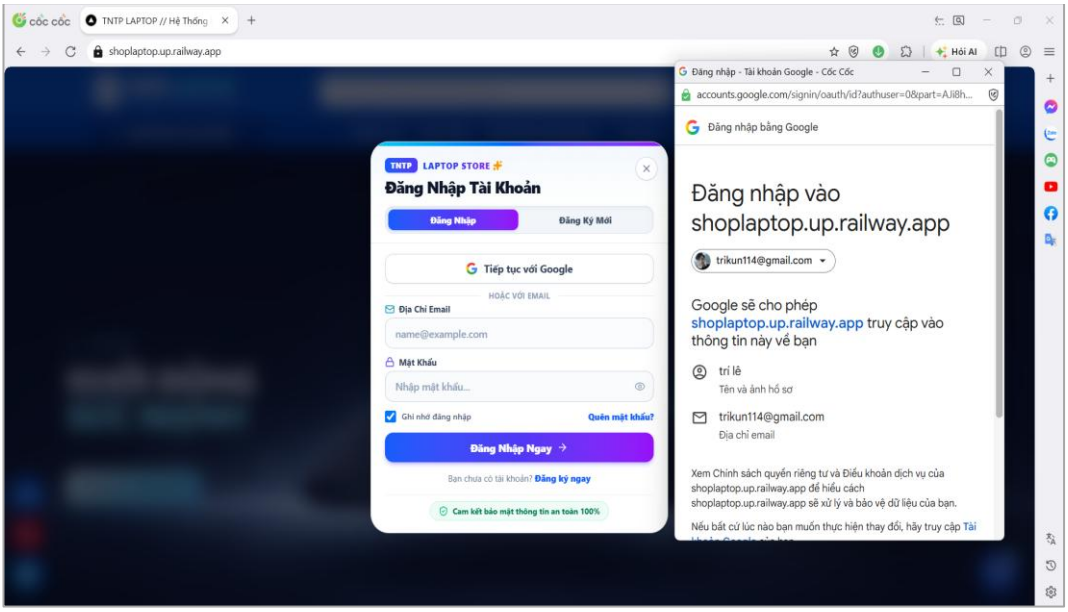
- Cho phép đăng nhập một chạm bằng tài khoản Google, tự tạo tài khoản nếu email chưa có.

❖ Luồng hoạt động:

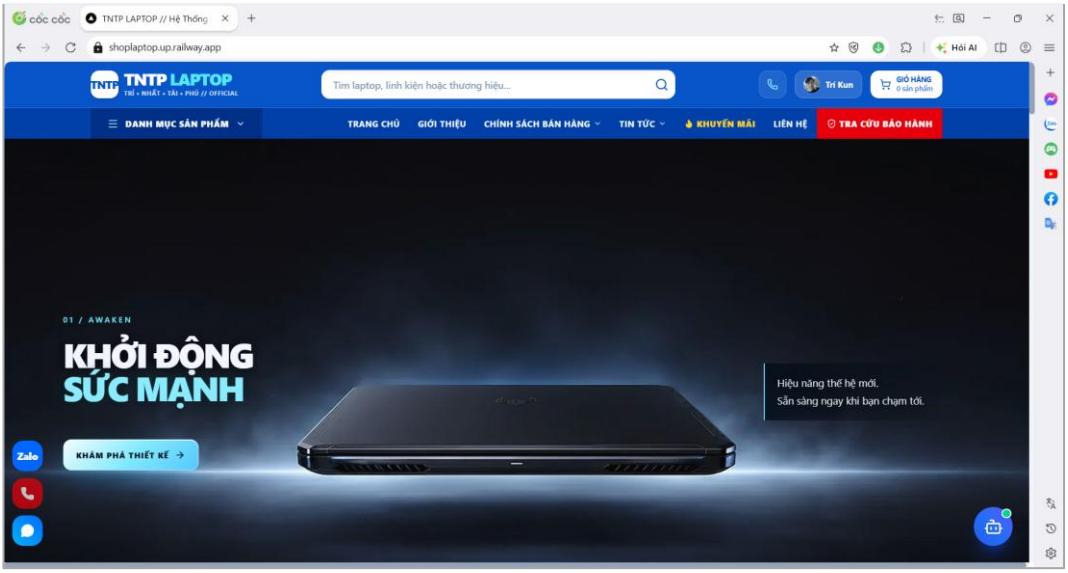
- Front-End dùng Google Sign-In, nhận **ID Token** (`credential`) và gửi tới `POST /api/auth/google`.
- Server gọi `OAuth2Client.verifyIdToken({ idToken, audience: GOOGLE_CLIENT_ID })` của `google-auth-library` để xác minh chữ ký và lấy `email`, `name`, `picture`, `sub`.
- Tìm người dùng theo `email` hoặc `googleId`. Chưa có → tạo mới với `authProvider = google`, mật khẩu ngẫu nhiên đã băm. Đã có → liên kết `googleId`.
- Kiểm tra khóa tài khoản (**403**), sau đó trả JWT như đăng nhập thường.



Hình 4.8. *Chọn tài khoản Google khi đăng nhập*



Hình 4.9. *Google xác nhận chia sẻ tên, ảnh và email với website*



Hình 4.10. *Đăng nhập Google thành công, thanh điều hướng hiển thị tên người dùng*

4.4. Chức năng quên mật khẩu và đổi mật khẩu

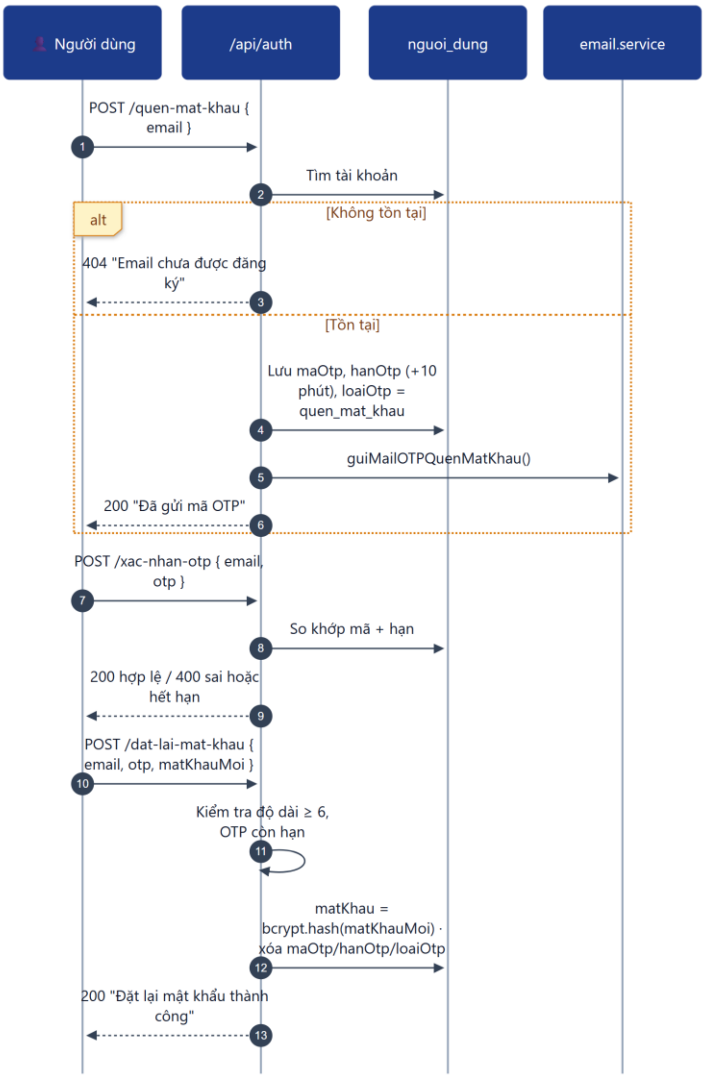
❖ Mục tiêu:

- Hỗ trợ người dùng đặt lại mật khẩu khi quên, bằng mã OTP gửi về email.
- Khi đổi mật khẩu lúc đã đăng nhập, yêu cầu cả mật khẩu cũ và OTP để chống chiếm quyền.

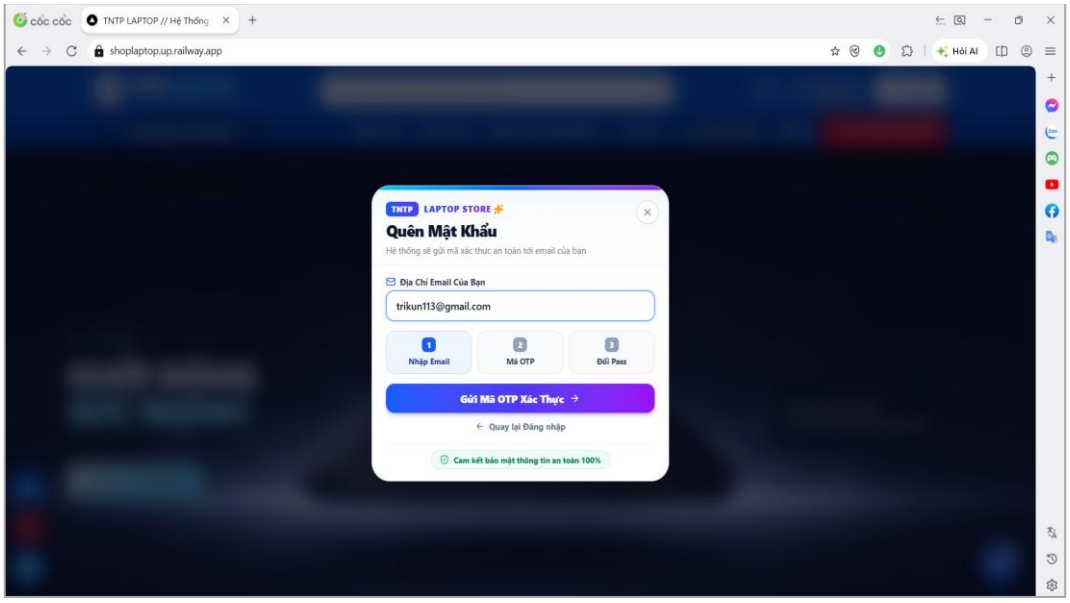
❖ Luồng hoạt động:

- **Quên mật khẩu:** `POST /quen-mat-khau` → email không tồn tại trả **404**; tồn tại thì lưu `maOtp`, `hanOtp` (+10 phút), `loaiOtp = quen_mat_khau` và gửi email.

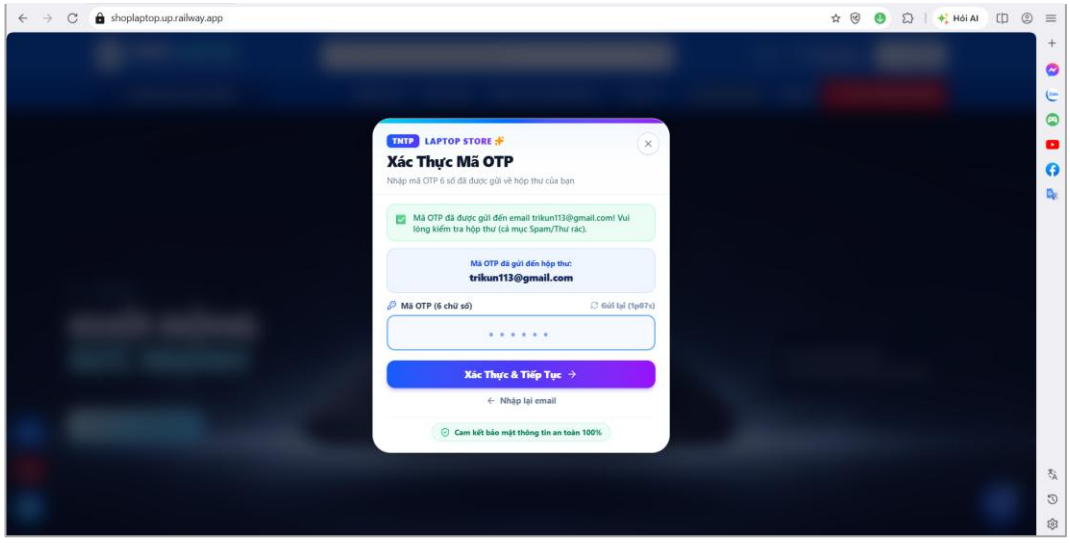
- `POST /xac-nhan-otp` kiểm tra mã và hạn; `POST /dat-lai-mat-khau` kiểm tra mật khẩu mới ≥ 6 ký tự, băm và lưu, **xóa OTP** để không dùng lại được.
- **Đổi mật khẩu:** `POST /doi-mat-khau/gui-otp` (JWT) kiểm tra `matKhauCu` rồi gửi OTP loại `doi_mat_khau`; `PUT /doi-mat-khau` chỉ chấp nhận khi đúng loại OTP, đúng mã và còn hạn.



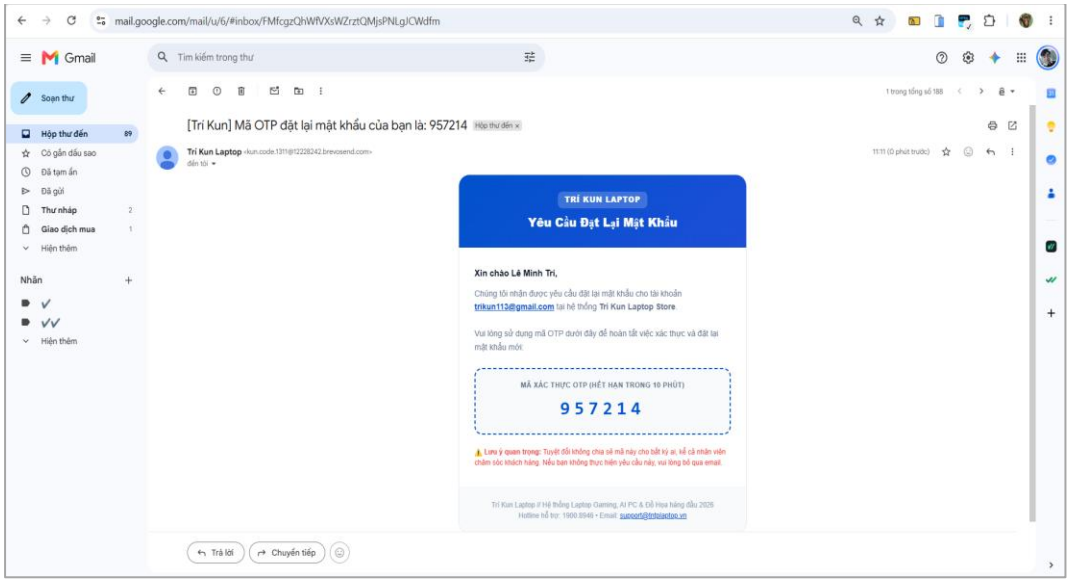
Hình 4.11. *Sơ đồ tuần tự quên mật khẩu bằng OTP*



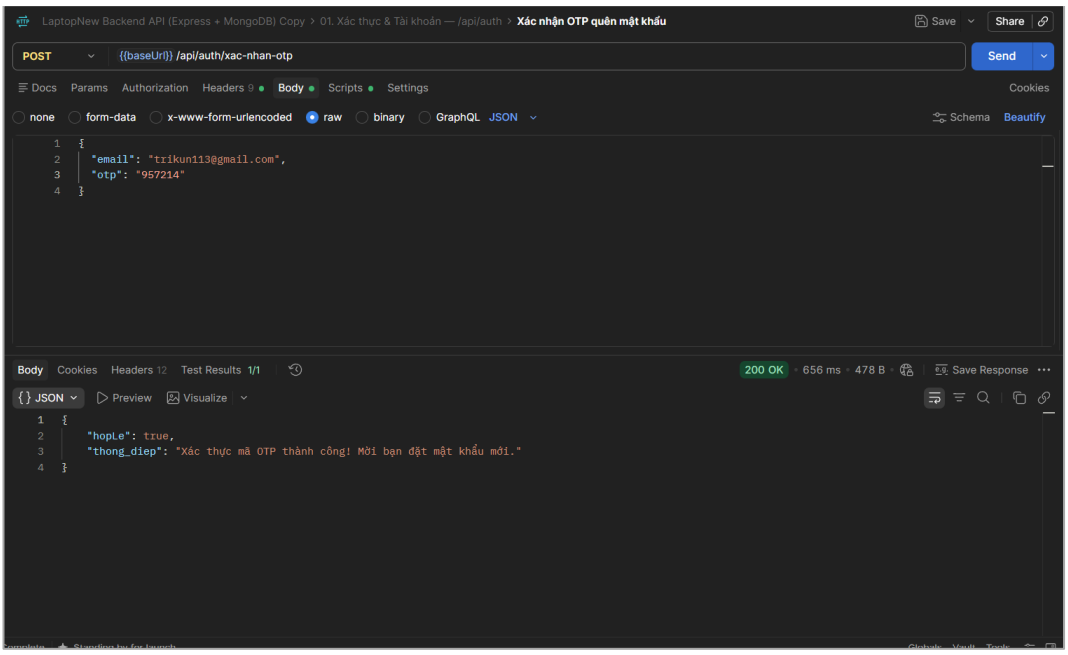
Hình 4.12. *Trang quên mật khẩu: nhập email để nhận mã OTP*



Hình 4.13. *Bước nhập mã OTP xác thực*



Hình 4.14. *Email chứa mã OTP đặt lại mật khẩu*



Hình 4.15. *Kiểm thử POST /api/auth/xac-nhan-otp bằng Postman: OTP hợp lệ*

4.5. Chức năng quản lý sản phẩm

4.5.1. Tìm kiếm, lọc và sắp xếp

❖ **Mục tiêu:**

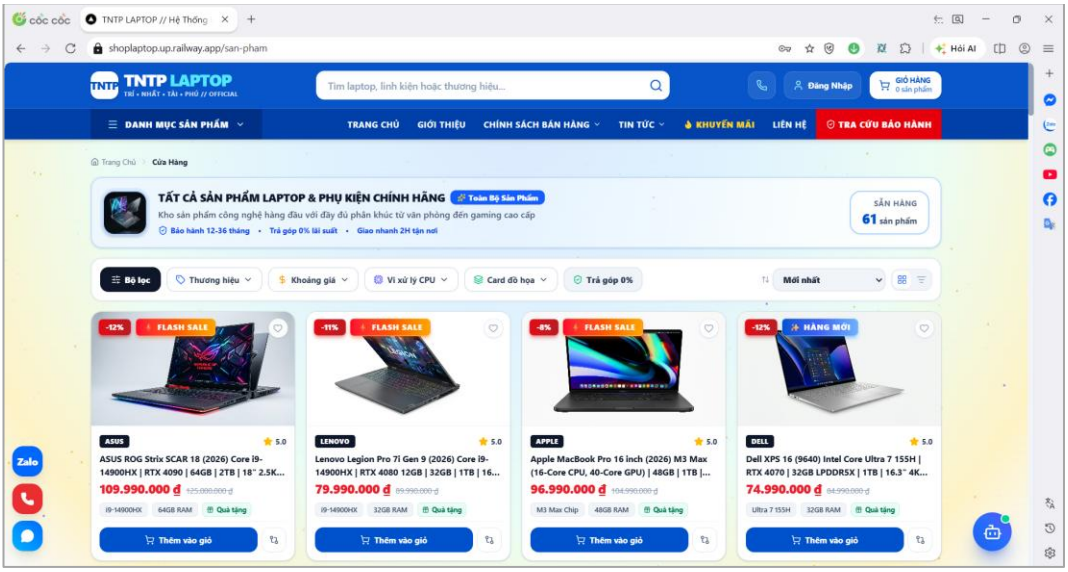
- Cho phép khách tìm laptop theo từ khóa, hãng, nhu cầu, khoảng giá và sắp xếp theo nhiều tiêu chí trong một API duy nhất.

❖ **Luồng hoạt động:**

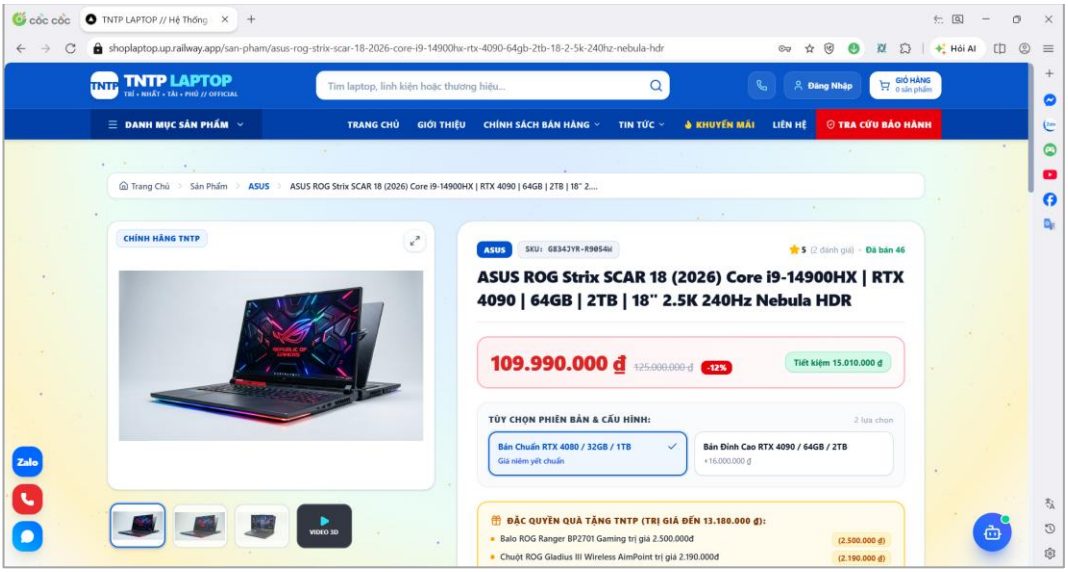
- Controller đọc các tham số query và **xây dựng dần đối tượng** `filter` của MongoDB.
- Từ khóa tìm trên 5 trường (tên, mã, hãng, CPU, card đồ họa) bằng `$regex` không phân biệt hoa thường.
- Hãng và danh mục cho phép nhiều giá trị (`asus,dell`) → toán tử `$in`; khoảng giá → `$gte` / `$lte`.
- Áp dụng `sort` theo `sap_xep`, `limit` nếu có; đặt header `Cache-Control: public, max-age=60` để trình duyệt/CDN lưu tạm 60 giây.

Xây dựng bộ lọc MongoDB từ tham số query

```
// controllers/san-pham.controller.js — layTatCaSanPham (trích)
const filter = {};
const keyword = tu_khoa || q;
if (keyword) filter.$or = [
    { ten_san_pham: { $regex: keyword, $options: 'i' } },
    { ma_san_pham:  { $regex: keyword, $options: 'i' } },
    { hang_san_xuat:{ $regex: keyword, $options: 'i' } },
    { 'thong_so.cpu':         { $regex: keyword, $options: 'i' } },
    { 'thong_so.card_do_hoa': { $regex: keyword, $options: 'i' } }
];
if (hang_san_xuat) filter.hang_san_xuat = { $in: hang_san_xuat.split(',').map(b =>
b.trim().toLowerCase()) };
if (danh_muc)      filter.danh_muc      = { $in: danh_muc.split(',').map(c =>
c.trim().toLowerCase()) };
if (gia_min !== undefined || gia_max !== undefined) {
    filter.gia_khuyen_mai = {};
    if (gia_min !== undefined) filter.gia_khuyen_mai.$gte = Number(gia_min);
    if (gia_max !== undefined) filter.gia_khuyen_mai.$lte = Number(gia_max);
}
let query = SanPham.find(filter).sort(sortOption);
if (limit) query = query.limit(Number(limit));
```



Hình 4.16. *Trang danh sách sản phẩm với bộ lọc thương hiệu, khoảng giá, CPU, card đồ họa*



Hình 4.17. *Trang chi tiết sản phẩm: thư viện ảnh, phiên bản cấu hình, giá khuyến mãi*

4.5.2. Thêm, sửa, xóa sản phẩm (Admin)

❖ Luồng hoạt động:

- Ba route `POST`, `PUT /:id`, `DELETE /:id` đều qua `xacThucToken` → `chiAdmin`.
- Khi thêm: tự sinh `id` (`sp-<timestamp>`) và `slug` nếu thiếu; kiểm tra trùng `id`/`slug`/`ma_san_pham` → **409 Conflict**.
- Khi sửa / xóa: tìm theo nhiều loại định danh; không thấy → **404**.

▣ CHÈN ẢNH CHỤP MÀN HÌNH

Đăng nhập admin@laptopnew.vn → vào /admin → mục **Kho sản phẩm**

Chụp bảng danh sách sản phẩm và form **Thêm sản phẩm**

(Xóa khung này, dán ảnh vào vị trí này — giữ nguyên dòng chú thích bên dưới)

Hình 4.18. *Trang quản trị: danh sách và form thêm/sửa sản phẩm*

4.6. Chức năng giỏ hàng và mã giảm giá

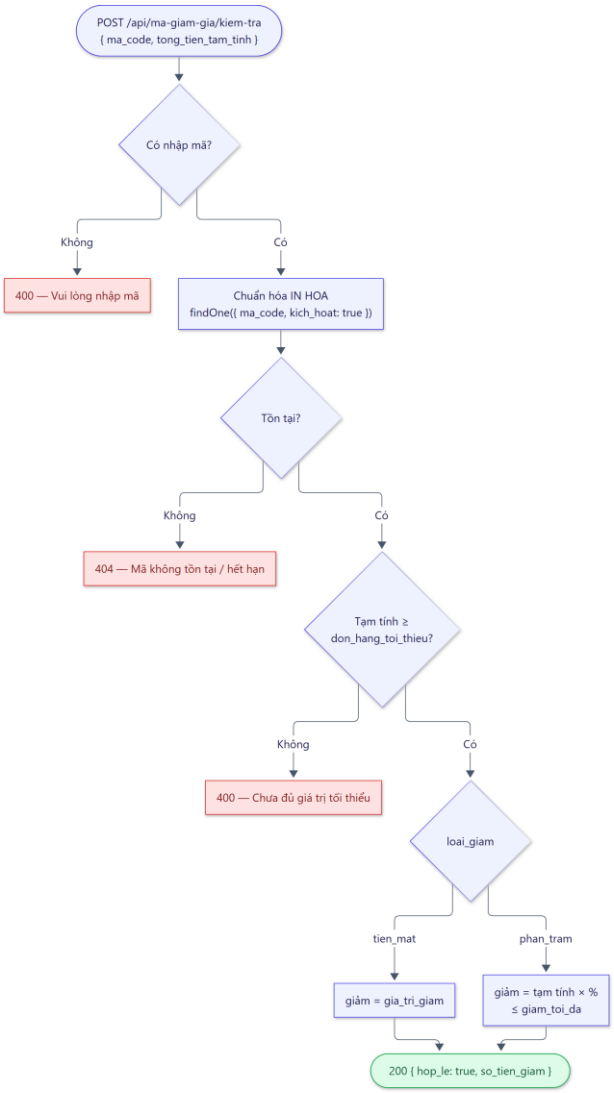
❖ Mục tiêu:

- Kiểm tra mã voucher khách nhập, tính đúng số tiền được giảm theo loại voucher và điều kiện đơn tối thiểu.

❖ Luồng hoạt động:

- Giỏ hàng được quản lý ở Front-End (React Context). Khi khách nhập mã, client gọi `POST /api/ma-giam-gia/kiem-tra` với `ma_code` và `tong_tien_tam_tinh`.

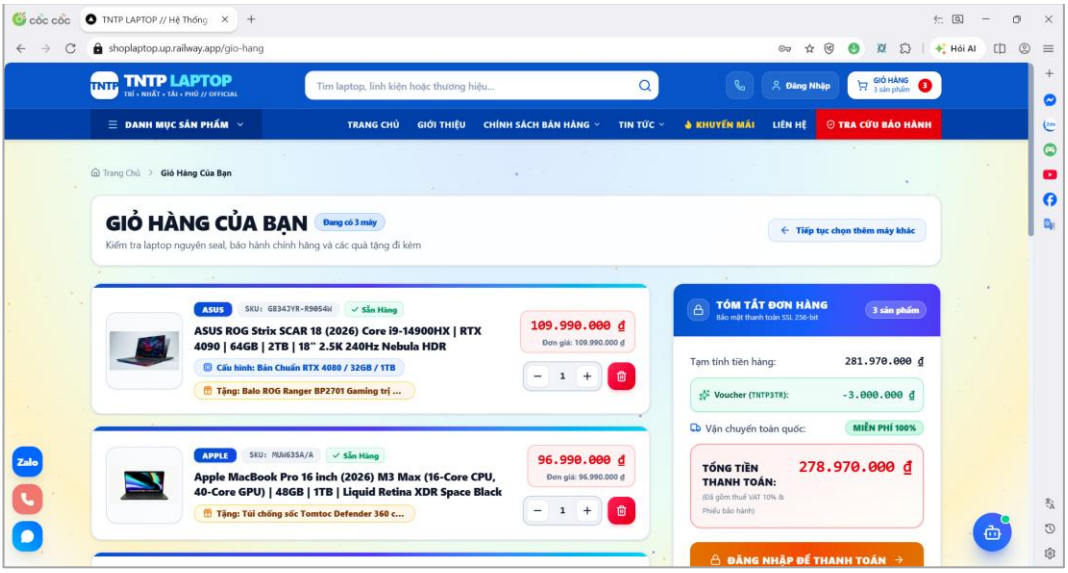
- Server chuẩn hóa mã IN HOA, chỉ tìm voucher đang `kich_hoat`; tính tiền giảm: loại `tien_mat` giảm cố định, loại `phan_tram` giảm theo % nhưng không vượt `giam_toi_da`.
- Thành viên có thể "săn mã" vào ví bằng `POST /luu-ma` và xem lại bằng `GET /vi-cua-toi`.



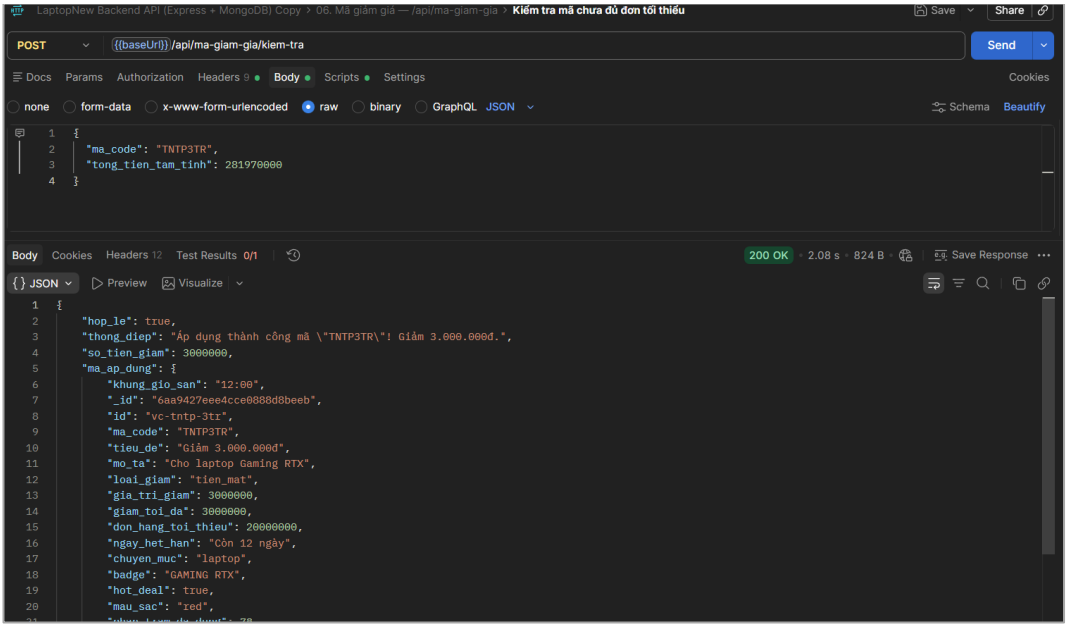
Hình 4.19. *Lưu đồ kiểm tra và tính tiền mã giảm giá*

Tính số tiền giảm của voucher

```
// controllers/ma-giam-gia.controller.js — kiemTraMaGiamGia (trích)
const voucher = await MaGiamGia.findOne({ ma_code: ma_code.trim().toUpperCase(),
kich_hoat: true });
if (!voucher) return res.status(404).json({ hop_le: false, thong_diep: 'Mã giảm giá không
tồn tại...' });
const tamTinh = Number(tong_tien_tam_tinh) || 0;
if (tamTinh < voucher.don_hang_toi_thieu) return res.status(400).json({ hop_le:
false, ... });
let soTienGiam = 0;
if (voucher.loai_giam === 'tien_mat') soTienGiam = voucher.gia_tri_giam;
else {
    soTienGiam = Math.round((tamTinh * voucher.gia_tri_giam) / 100);
    if (voucher.giam_toi_da && soTienGiam > voucher.giam_toi_da) soTienGiam =
voucher.giam_toi_da;
}
return res.status(200).json({ hop_le: true, so_tien_giam: soTienGiam, ma_ap_dung:
voucher });
```



Hình 4.20. *Giỏ hàng áp dụng mã giảm giá TNTP3TR thành công*



Hình 4.21. *Kiểm thử POST /api/ma-giam-gia/kiem-tra bằng Postman: mã hợp lệ, giảm 3.000.000 đ*

4.7. Chức năng đặt hàng và quản lý trạng thái đơn

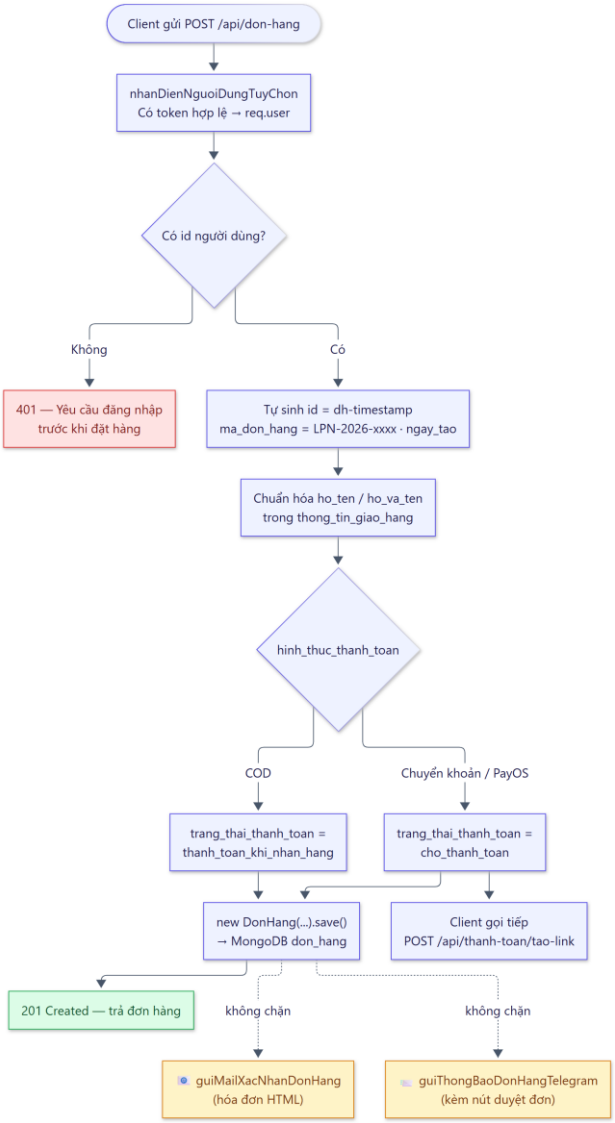
4.7.1. Tạo đơn hàng

❖ **Mục tiêu:**

- Ghi nhận đơn hàng hợp lệ của người dùng đã đăng nhập, chuẩn hóa dữ liệu và thông báo ngay cho khách và quản trị viên.

❖ **Luồng hoạt động:**

- `nhanDienNguoiDungTuyChon` gắn `req.user` nếu có token; không xác định được người đặt → **401** "Quý khách vui lòng đăng nhập...".
- Tự sinh `id`, `ma_don_hang` dạng `LPN-2026-xxxx`, `ngay_tao`; đồng bộ `ho_ten`/`ho_va_ten`.
- Chuẩn hóa trạng thái thanh toán: COD → `thanh_toan_khi_nhan_hang`; chuyển khoản → `cho_thanh_toan`.
- Lưu đơn, trả **201**, rồi chạy **không chặn** hai tác vụ: gửi email hóa đơn và gửi thông báo Telegram (lỗi ở hai tác vụ này không làm hỏng đơn).



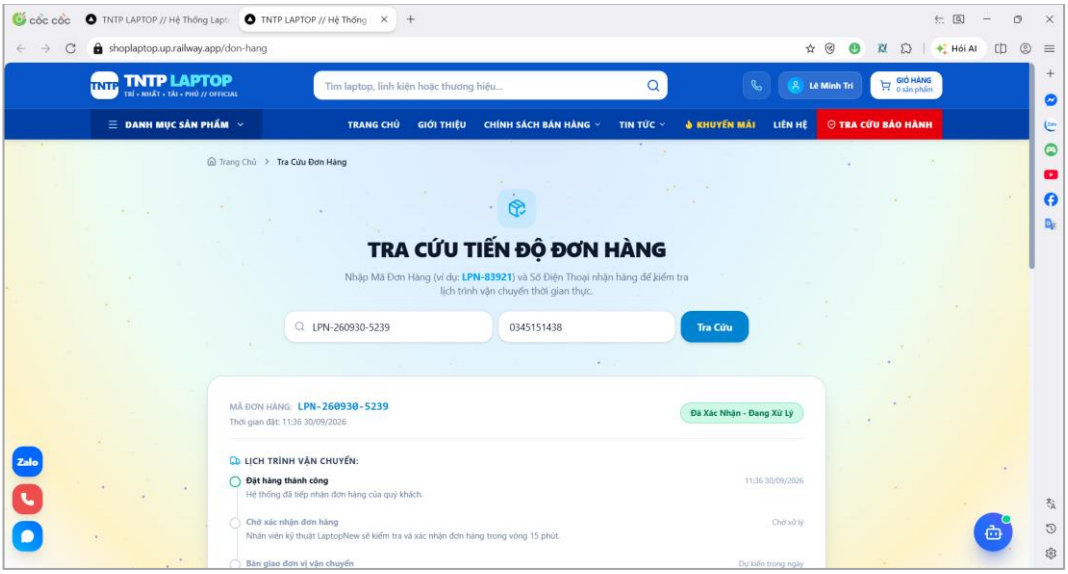
Hình 4.22. *Lưu đồ xử lý tạo đơn hàng*

4.7.2. Phân quyền xem và cập nhật đơn

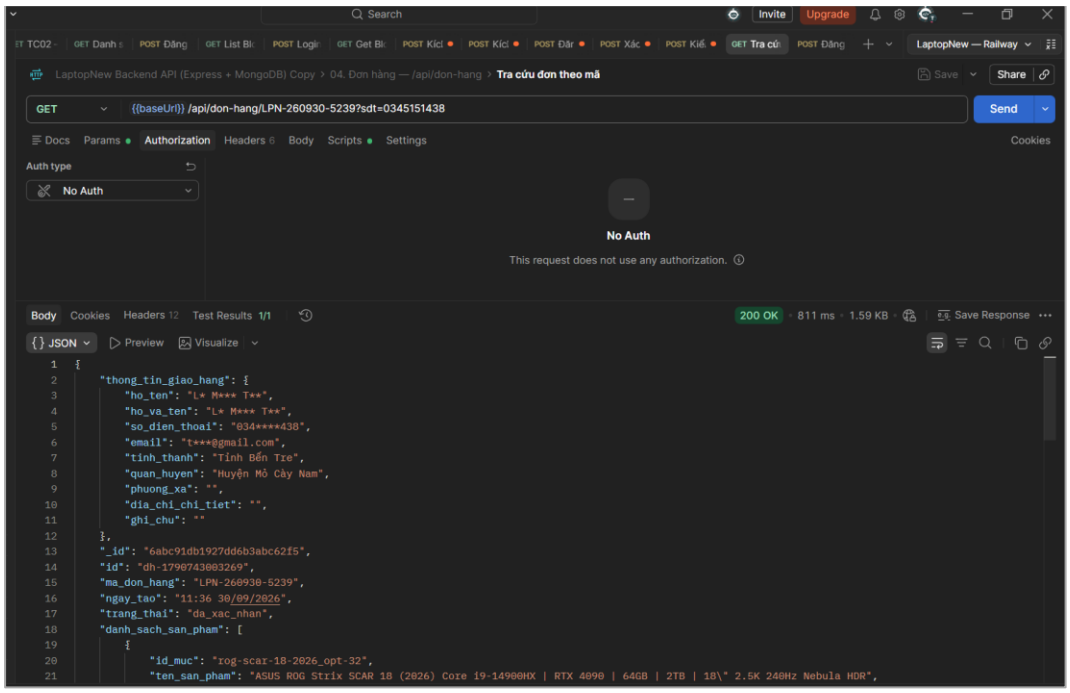
- **Admin** xem được mọi đơn, lọc theo trạng thái/người dùng; cập nhật mọi trường.
- **Thành viên** chỉ xem đơn có `id_nguoi_dung`, email hoặc SĐT trùng với tài khoản; chỉ được sửa các trường thanh toán và được **hủy** khi đơn còn `cho_xac_nhan`. Sửa đơn của người khác → **403**.
- **Khách chưa đăng nhập** gọi `GET /api/don-hang` mà không truyền điều kiện → trả mảng rỗng, không bao giờ lộ toàn bộ đơn.
- **Tra cứu công khai** (`GET /api/don-hang/:id`): khách chưa đăng nhập phải nhập đúng **mã đơn** và **số điện thoại nhận hàng** (`?sdt=`); thiếu hoặc sai → **403** kèm `yeu_cau_xac_minh_sdt: true`.

- Dữ liệu trả cho khách được che bởi `anThongTinDonHangCongKhai()`: họ tên `L* M*** T**`, SĐT `034****438`, email `t***@gmail.com`, bỏ địa chỉ chi tiết và ghi chú. Đăng nhập nhưng không phải chủ đơn → **403**.
- Middleware `gioiHanTraCuuDonHang` giới hạn **30 lần / 10 phút** cho mỗi IP; vượt quá → **429**, chống dò mã đơn hàng loạt.

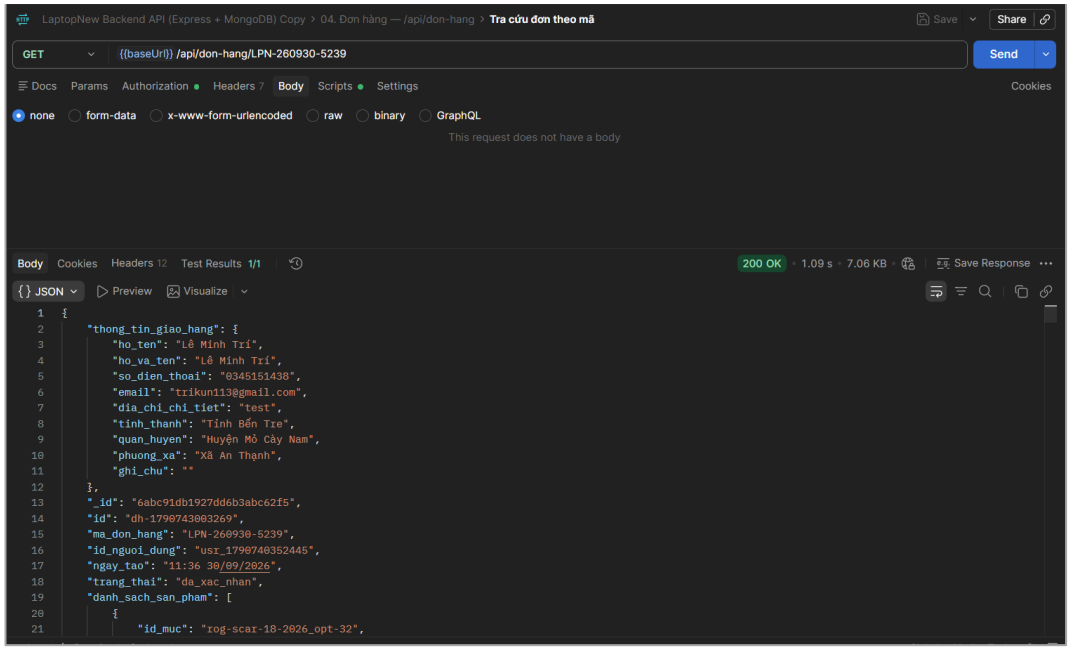
```
Phân quyền xem đơn và che dữ liệu khi tra cứu công khai
// controllers/don-hang.controller.js – layDonHangTheoIdHoacMa (trích)
if (laChuDonHang(req, donHang)) return res.status(200).json(donHang);   // Admin hoặc chủ đơn
if (req.user) return res.status(403).json({ thong_diep: 'Bạn chỉ được xem đơn hàng thuộc tài khoản của mình.' });
if (!maDonDuocNhap || !sdtXacMinh || sdtXacMinh !== sdtKhach)
    return res.status(403).json({ thong_diep: 'Vui lòng nhập đúng mã đơn hàng và số điện thoại...', yeu_cau_xac_minh_sdt: true });
return res.status(200).json(anThongTinDonHangCongKhai(donHang));        // Che họ tên, SĐT, email, địa chỉ
```



Hình 4.23. *Trang tra cứu tiến độ đơn hàng: khách nhập mã đơn và số điện thoại*



Hình 4.24. *Kiểm thử GET /api/don-hang/:id?sdt=... khi chưa đăng nhập: thông tin cá nhân đã được che*



Hình 4.25. *Kiểm thử GET /api/don-hang/:id với token chủ đơn: trả đầy đủ thông tin đơn hàng*

▣ CHÈN ẢNH CHỤP POSTMAN

Request: **04. Đơn hàng › Tra cứu đơn theo mã** — chọn **No Auth**, bỏ tham số `sdt` (hoặc nhập sai SĐT)

Kỳ vọng: **403** `"Vui lòng nhập đúng mã đơn hàng và số điện thoại..."`, `yeu_cau_xac_minh_sdt: true`

(Xóa khung này, dán ảnh vào vị trí này — giữ nguyên dòng chú thích bên dưới)

Hình 4.26. *Kiểm thử tra cứu đơn thiếu hoặc sai số điện thoại bị từ chối (403)*

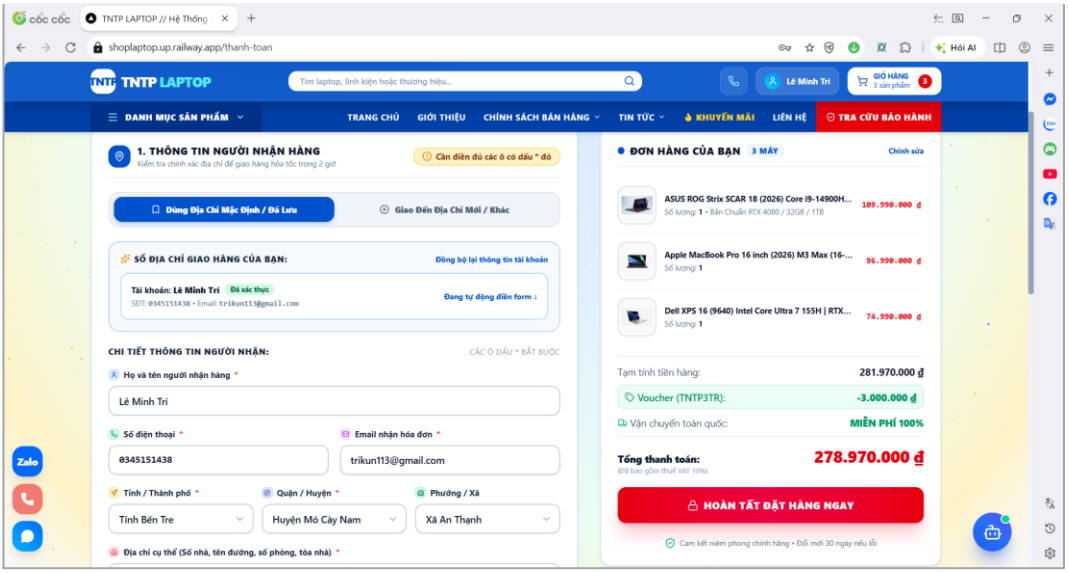
4.7.3. Trạng thái đơn và đồng bộ tồn kho



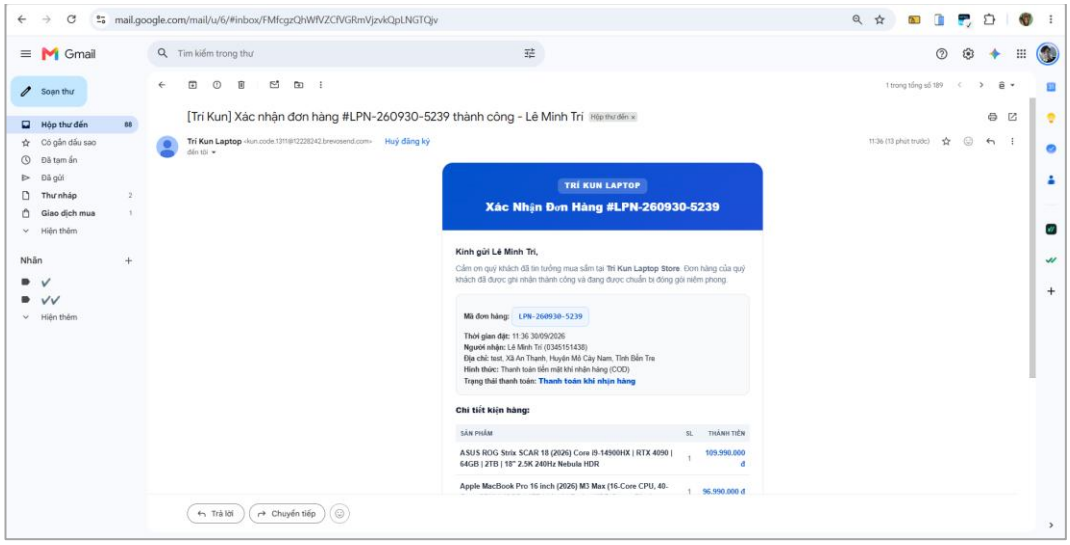
Hình 4.27. *Vòng đời trạng thái đơn hàng và quy tắc trừ/hoàn tồn kho*

Trừ và hoàn tồn kho hai chiều theo trạng thái đơn

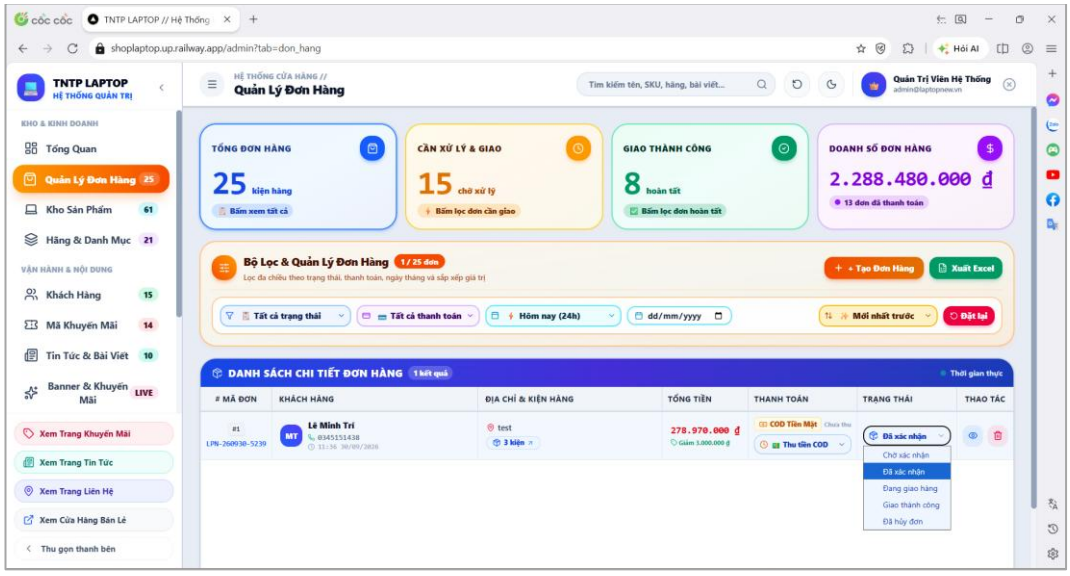
```
// controllers/don-hang.controller.js — capNhatTrangThaiDonHang (nhánh admin, trích)
if (trangThaiMoi === 'da_giao' && !donHangHienTai.da_tru_ton_kho) {
    await capNhatTonKhoTheoDonHang(danhSachSp, 'tru');     // giảm tồn kho, tăng đã bán
    duLieuCapNhat.da_tru_ton_kho = true;
    if (duLieuCapNhat.da_thanh_toan === undefined) duLieuCapNhat.da_thanh_toan = true;
} else if (trangThaiMoi !== 'da_giao' && donHangHienTai.da_tru_ton_kho) {
    await capNhatTonKhoTheoDonHang(danhSachSp, 'hoan');    // hoàn lại kho
    duLieuCapNhat.da_tru_ton_kho = false;
}
const donHang = await DonHang.findByIdAndUpdate(donHangHienTai._id,
    { $set: duLieuCapNhat }, { new: true, runValidators: true });
```



Hình 4.28. *Trang thanh toán: thông tin người nhận, địa chỉ hành chính, phương thức thanh toán*



Hình 4.29. *Email xác nhận đặt hàng thành công gửi tới khách hàng*



Hình 4.30. *Trang quản trị đơn hàng (Admin): lọc đơn và đổi trạng thái*

4.8. Chức năng thanh toán PayOS và VietQR

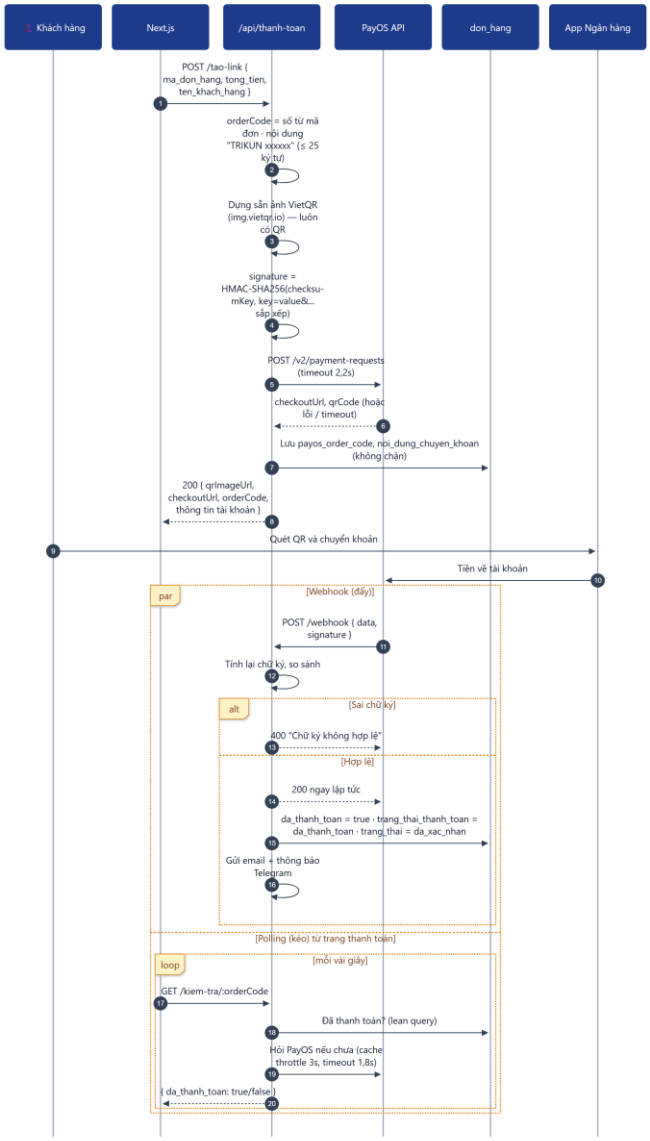
❖ Mục tiêu:

- Khách chuyển khoản bằng mã QR có sẵn số tiền và nội dung; hệ thống tự xác nhận khi tiền về, không cần nhân viên đối soát.
- Luôn hiển thị được QR ngay cả khi cổng PayOS phản hồi chậm.

❖ Luồng hoạt động:

- **Tạo yêu cầu:** sinh `orderCode` là số từ mã đơn, nội dung `TRIKUN xxxxxx` (≤ 25 ký tự theo quy định PayOS); dựng sẵn URL ảnh VietQR; ký payload bằng HMAC-SHA256 rồi gọi PayOS với timeout 2,2 giây.

- **Webhook:** PayOS gọi `POST /api/thanh-toan/webhook`. Server tính lại chữ ký trên `data`; sai → **400**. Đúng → trả **200 ngay** cho PayOS, sau đó cập nhật đơn (`da_thanh_toan`, `trang_thai_thanh_toan = da_thanh_toan`, `trang_thai = da_xac_nhan`), gửi email và Telegram.
- **Polling:** trang thanh toán gọi `GET /kiem-tra/:orderCode` định kỳ. Server đọc DB trước (lean query), rồi bộ nhớ đệm 3 giây, cuối cùng mới hỏi PayOS (timeout 1,8 giây) — tránh dồn request lên PayOS.

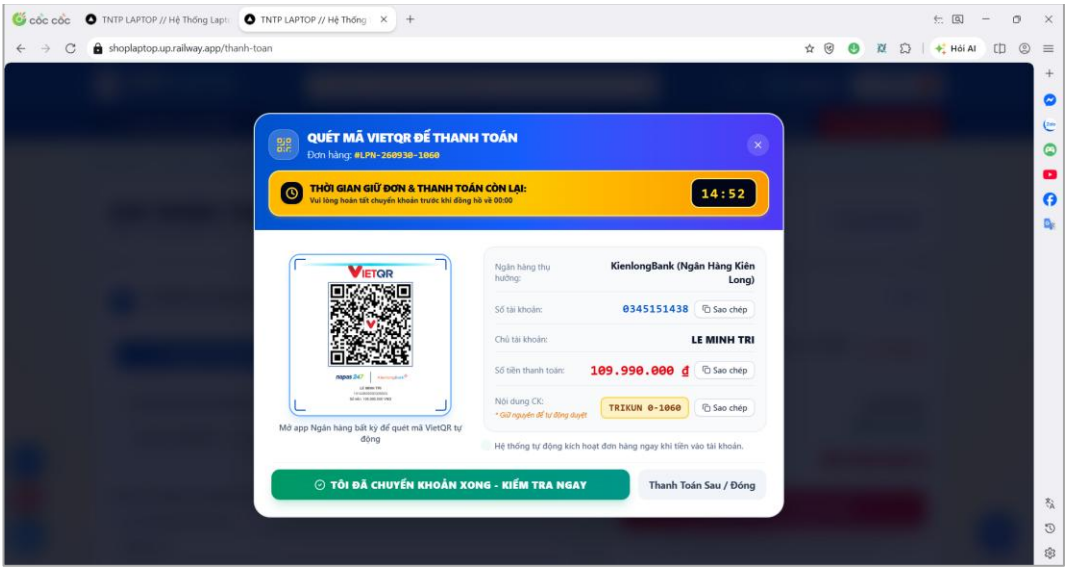


Hình 4.31. *Sơ đồ tuần tự thanh toán PayOS / VietQR với Webhook và polling*

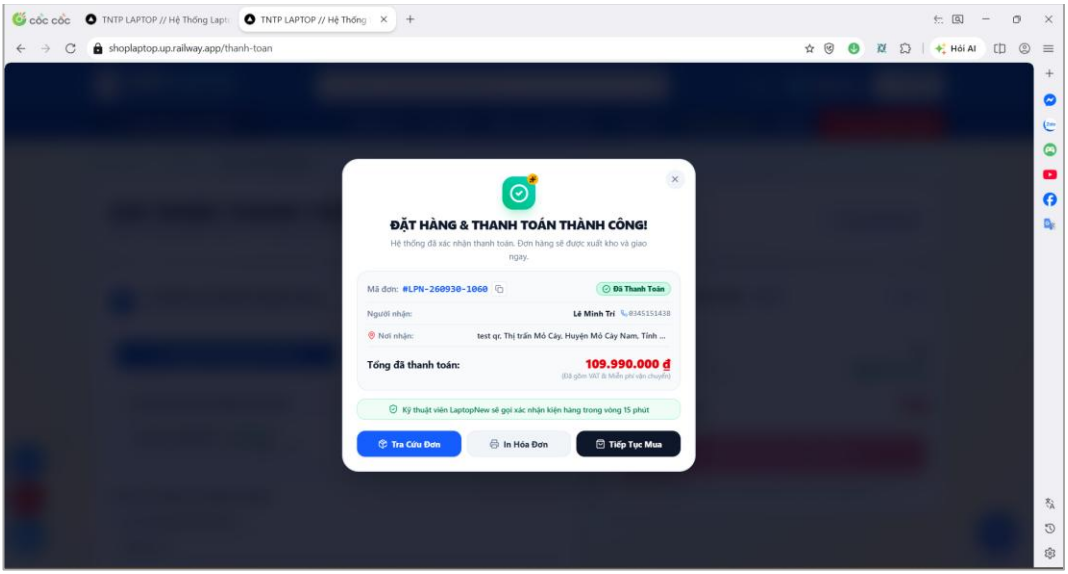
Ký và xác minh chữ ký HMAC-SHA256 của PayOS

```
// controllers/thanh-toan.controller.js
const taoChuKyPayOS = (data, checksumKey) => {
    const chuoiKy = Object.keys(data).sort()                              // sắp xếp khóa a→z
        .filter(k => data[k] !== undefined && data[k] !== null && typeof data[k] !==
'object')
        .map(k => `${k}=${data[k]}`).join('&');                    //
amount=...&cancelUrl=...
    return crypto.createHmac('sha256', checksumKey).update(chuoiKy).digest('hex');
};

const xuLyWebhookPayOS = async (req, res) => {
    const payload = req.body;
    if (!payload || !payload.data || !payload.signature)
        return res.status(400).json({ success: false, message: 'Thiếu dữ liệu
Webhook' });
    if (taoChuKyPayOS(payload.data, PAYOS_CHECKSUM_KEY) !== payload.signature)
        return res.status(400).json({ success: false, message: 'Chữ ký không hợp lệ' });
    res.status(200).json({ success: true });                              // phản hồi PayOS
ngay
    DonHang.findOneAndUpdate(
        { $or: [{ payos_order_code: Number(payload.data.orderCode) }, { ma_don_hang:
String(payload.data.orderCode) }] },
        { $set: { da_thanh_toan: true, trang_thai_thanh_toan: 'da_thanh_toan',
trang_thai: 'da_xac_nhan' } },
        { new: true }
    ).then(dh => { guiMailXacNhanDonHang(dh); guiThongBaoDonHangTelegram(dh); });
};
```



Hình 4.32. *Màn hình mã QR thanh toán PayOS / VietQR*



Hình 4.33. *Thanh toán thành công — đơn tự chuyển trạng thái*

4.9. Chức năng gửi email tự động

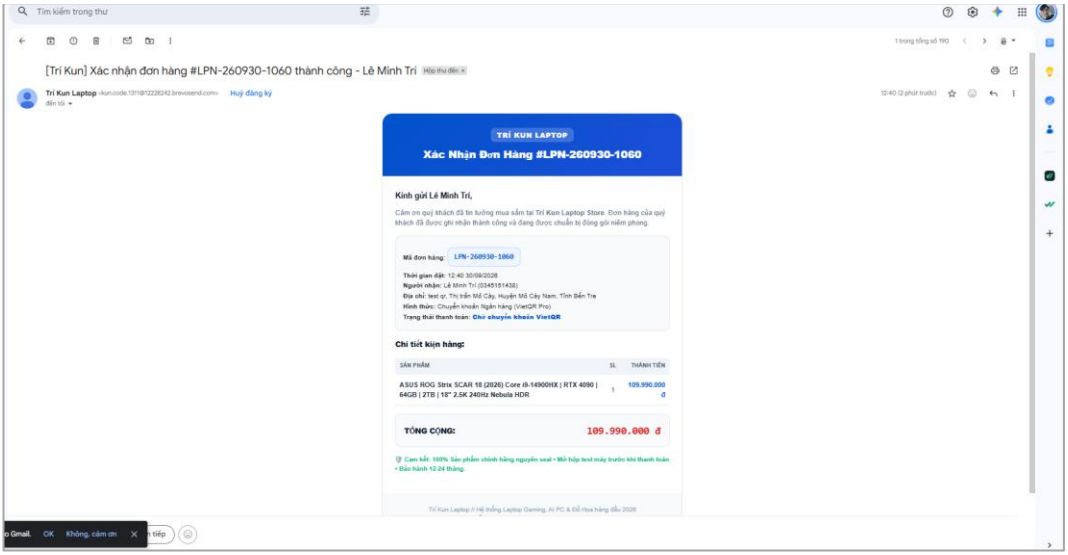
❖ Mục tiêu:

- Gửi email HTML cho OTP kích hoạt, OTP quên mật khẩu, OTP đổi mật khẩu và hóa đơn đơn hàng; hoạt động được cả ở máy cá nhân lẫn trên cloud.

❖ Luồng hoạt động:

- Hàm dùng chung `guiMailBangTransporter()` **ưu tiên Brevo REST API** (`POST https://api.brevo.com/v3/smtp/email` với header `api-key`).
- Nếu không có `BREVO_API_KEY`: khi chạy trên cloud (có biến `RAILWAY_ENVIRONMENT` / `VERCEL`) thì báo lỗi rõ ràng; khi chạy local thì dùng **Nodemailer + Gmail App Password**.
- Các hàm nghiệp vụ trả về `{ thanhCong, loi }` để controller quyết định trả **502** khi không gửi được OTP.

```
Gửi email hai tầng: Brevo API và Nodemailer
// services/email.service.js – guiMailBangTransporter (trích)
if (key) {                                              // Ưu tiên 1: Brevo API qua HTTPS
    const res = await fetch('https://api.brevo.com/v3/smtp/email', {
        method: 'POST',
        headers: { 'api-key': key, 'Content-Type': 'application/json' },
        body: JSON.stringify({ sender: { name: 'Trí Kun Laptop', email: sender },
            to: danhSachTo, subject: mailOptions.subject, htmlContent:
mailOptions.html })
    });
    if (!res.ok) throw new Error(`Brevo error ${res.status}`);
    return { messageId: (await res.json()).messageId };
}
if (isCloud) throw new Error('Railway chặn SMTP. Vui lòng cấu hình BREVO_API_KEY...');
const transporter = nodemailer.createTransport({ service: 'gmail', auth: { user:
emailUser, pass: emailPass } });
return transporter.sendMail(mailOptions);               // Dự phòng: Gmail SMTP khi chạy
local
```



Hình 4.34. *Email xác nhận đơn hàng (hóa đơn HTML)*

4.10. Chức năng quản trị đơn hàng qua Telegram Bot

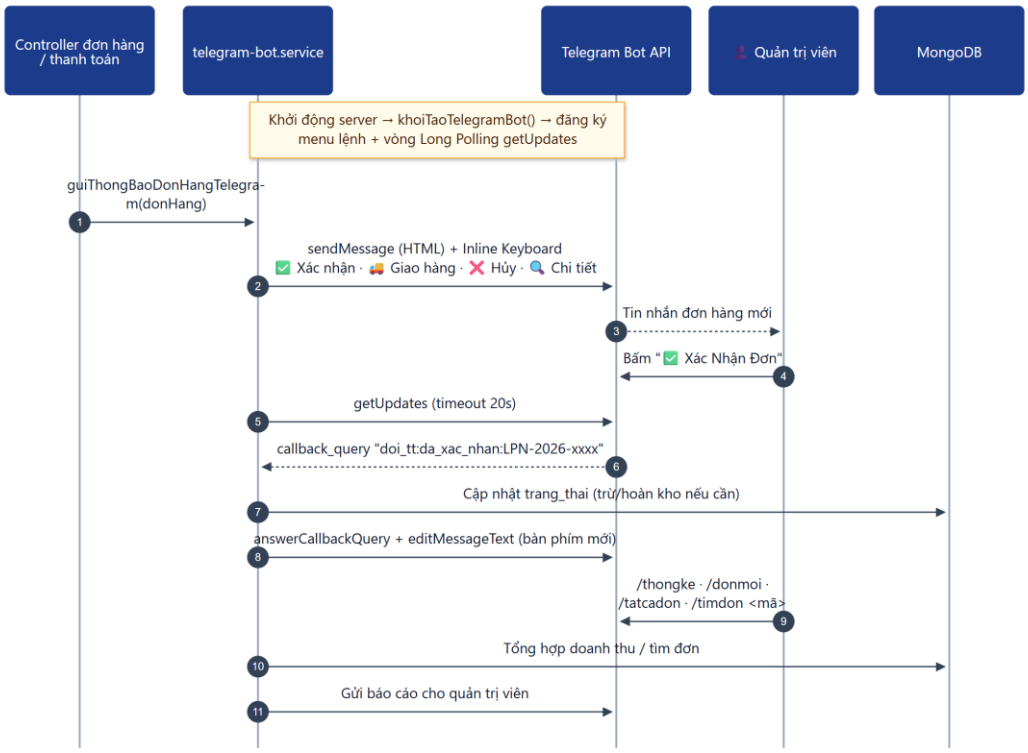
❖ Mục tiêu:

- Quản trị viên nhận thông báo đơn mới tức thì trên điện thoại và xử lý đơn chỉ bằng một lần bấm.

❖ Luồng hoạt động:

- Khi server khởi động, `khoiTaoTelegramBot()` đăng ký menu lệnh (`setMyCommands`) và chạy vòng **Long Polling** `getUpdates` (timeout 20 giây).
- Có đơn mới hoặc thanh toán thành công → gửi tin nhắn HTML kèm **Inline Keyboard**: Xác nhận, Giao hàng, Hủy, Chi tiết, Đổi trạng thái khác. Bàn phím thay đổi theo trạng thái hiện tại của đơn.

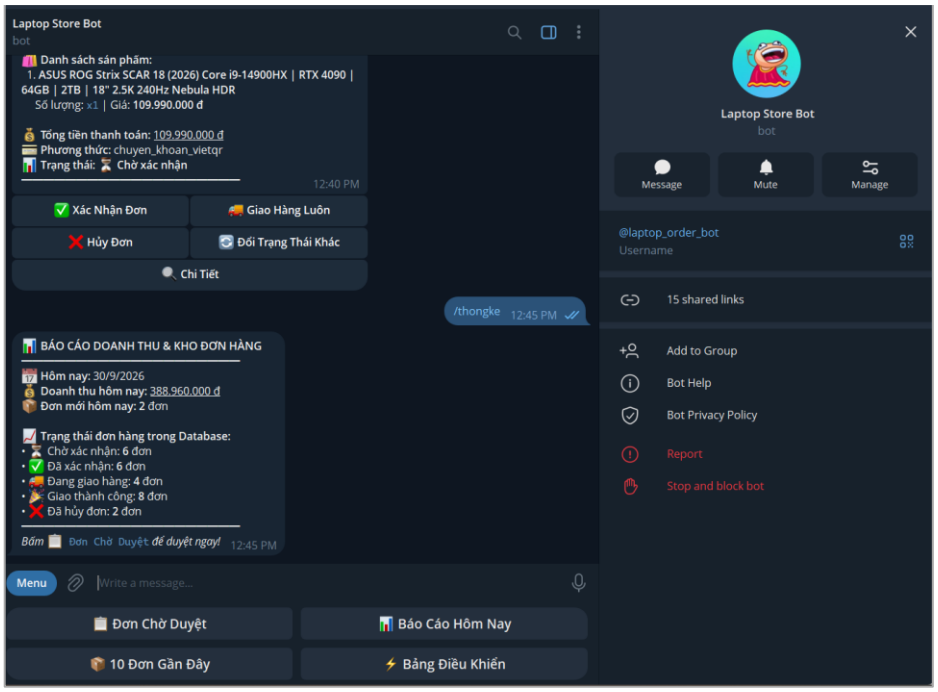
- Admin bấm nút → Telegram trả `callback_query` dạng `doi_tt:<trạng thái>:<mã đơn>` → bot cập nhật MongoDB (kèm trừ/hoàn kho), trả lời `answerCallbackQuery` và sửa lại tin nhắn.
- Lệnh văn bản: `/thongke` (báo cáo hôm nay), `/donmoi` (đơn chờ duyệt), `/tatcadon` (10 đơn gần đây), `/timdon <mã>`.



Hình 4.35. *Sơ đồ tuần tự thông báo và duyệt đơn qua Telegram Bot*



Hình 4.36. *Tin nhắn đơn hàng mới kèm nút duyệt trên Telegram*



Hình 4.37. *Kết quả lệnh /thongke trên Telegram*

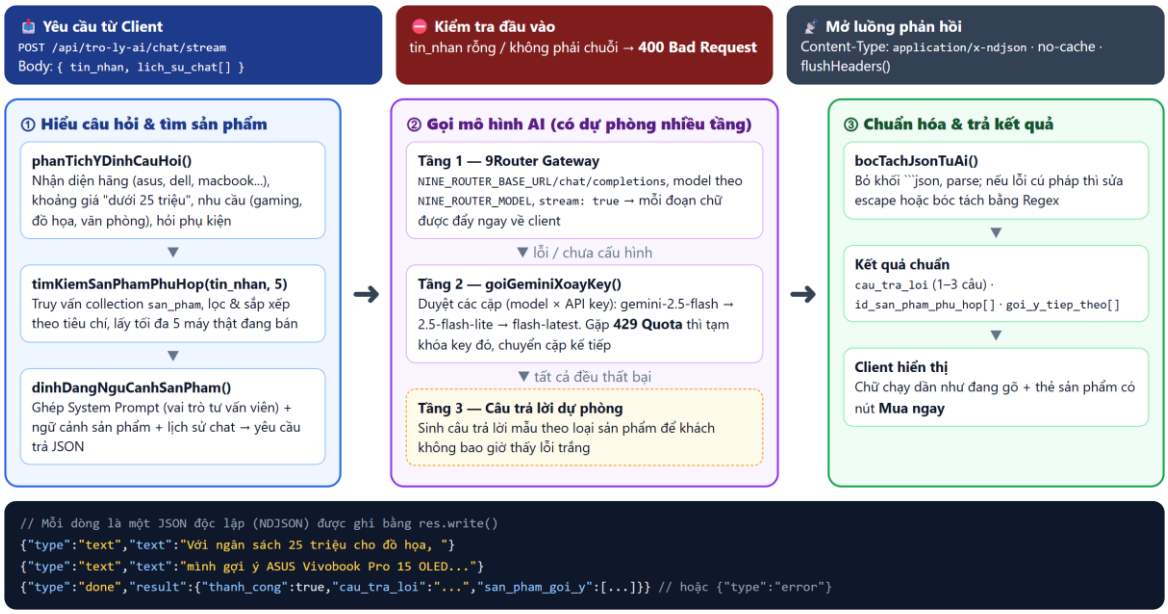
4.11. Chức năng Trợ lý AI tư vấn laptop

❖ Mục tiêu:

- Hiểu câu hỏi tiếng Việt tự nhiên (“laptop đồ họa dưới 25 triệu”), gợi ý **đúng các máy đang bán** thay vì trả lời chung chung.
- Hiển thị câu trả lời dần dần như đang gõ và không bao giờ “chết” khi một nhà cung cấp AI gặp sự cố.

❖ Luồng hoạt động:

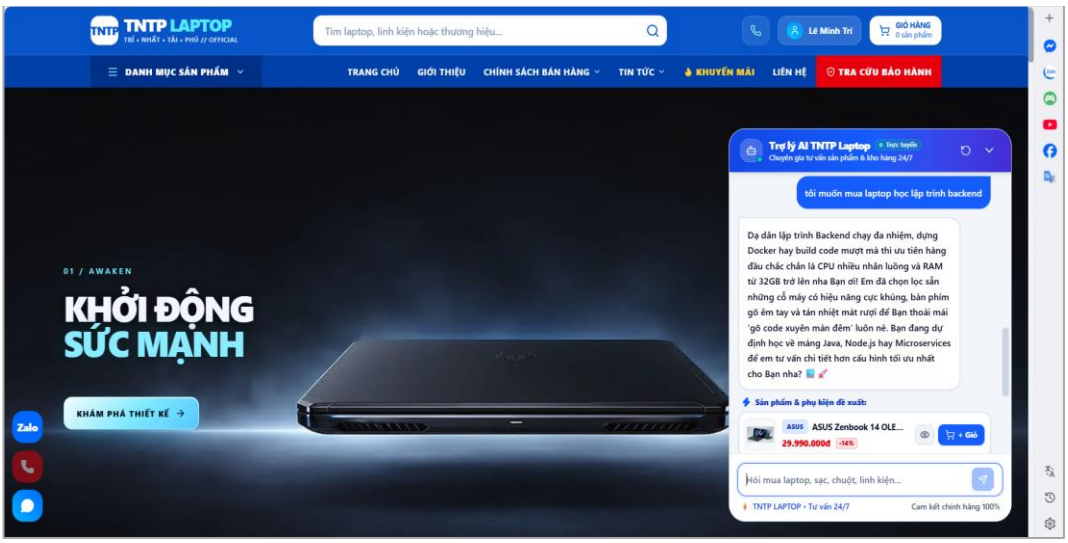
- Phân tích ý định câu hỏi (hãng, khoảng giá, nhu cầu, phụ kiện) → truy vấn tối đa 5 sản phẩm phù hợp trong `san_pham` → đưa vào prompt làm ngữ cảnh.
- Gọi mô hình theo 3 tầng dự phòng: **9Router** (stream) → **Gemini xoay vòng key** (tạm khóa key bị 429) → **câu trả lời mẫu**.
- Trả về dạng **NDJSON**: nhiều dòng `{"type":"text"}` rồi một dòng `{"type":"done","result":...}` chứa câu trả lời, sản phẩm gợi ý và câu hỏi gợi ý tiếp theo.



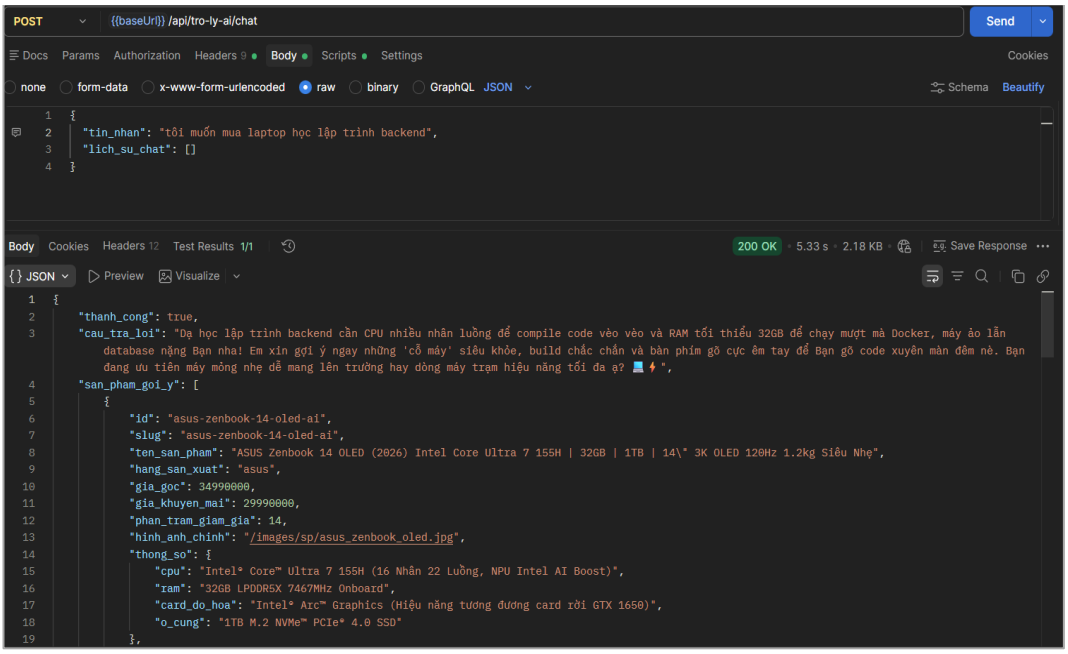
Hình 4.38. *Luồng xử lý của Trợ lý AI với cơ chế dự phòng nhiều tầng*

Truyền câu trả lời AI theo luồng NDJSON

```
// controllers/tro-ly-ai.controller.js – xuLyChatAIStream (trích)
res.setHeader('Content-Type', 'application/x-ndjson; charset=utf-8');
res.setHeader('Cache-Control', 'no-cache, no-transform');
res.flushHeaders();
try {
    const ketQua = await xuLyTroLyChat({
        tinNhan: tin_nhan.trim(), lichSuChat: lichSu,
        onText: text => res.write(JSON.stringify({ type: 'text', text }) + '\n')
    });
    res.write(JSON.stringify({ type: 'done', result: { thanh_cong: true, ...ketQua } }) +
'\n');
} catch (error) {
    res.write(JSON.stringify({ type: 'error', message: 'Không thể kết nối Trợ lý AI.' })
+ '\n');
} finally { res.end(); }
```



Hình 4.39. *Hộp chat Trợ lý AI tư vấn và gợi ý sản phẩm trên website*



Hình 4.40. *Kiểm thử POST /api/tro-ly-ai/chat bằng Postman: câu trả lời kèm sản phẩm gợi ý*

4.12. Chức năng tra cứu bảo hành điện tử

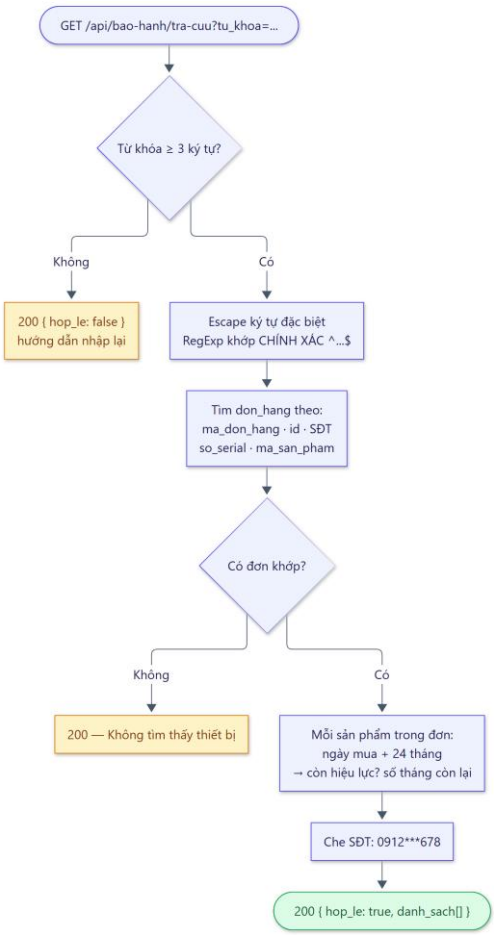
❖ Mục tiêu:

- Khách tra hạn bảo hành bằng mã đơn, số điện thoại, serial hoặc mã sản phẩm mà không cần phiếu giấy.

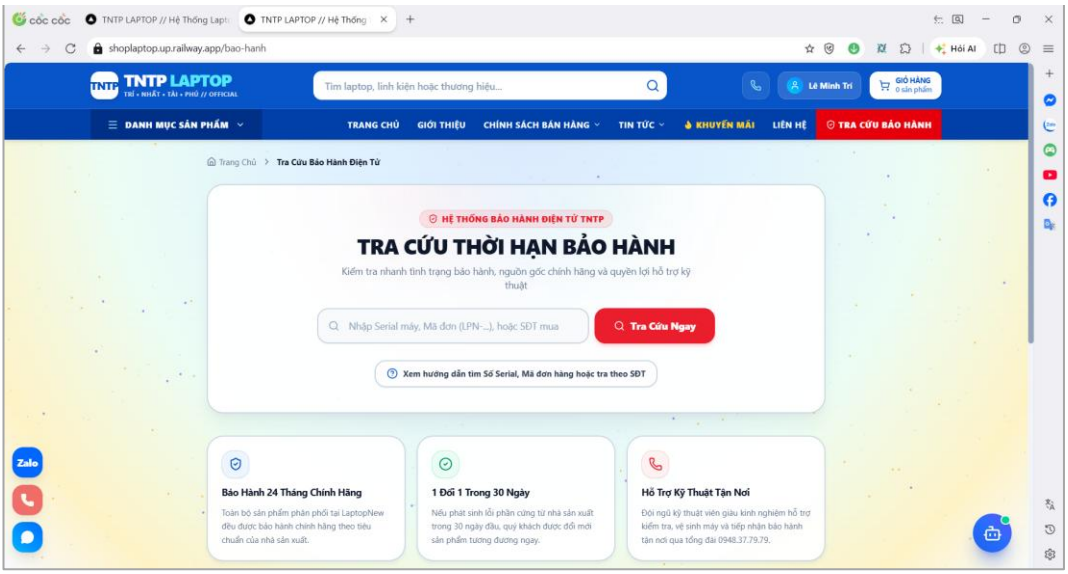
❖ Luồng hoạt động:

- Từ khóa phải ≥ 3 ký tự. Server **escape ký tự đặc biệt** và so khớp **chính xác toàn chuỗi** (`^...$`), tránh việc gõ vài số mà ra đơn của người khác.

- Với mỗi sản phẩm trong đơn khớp: ngày mua + 24 tháng → còn hiệu lực hay không, còn bao nhiêu tháng; số điện thoại được **che** dạng `0912***678`.



Hình 4.41. *Lưu đồ tra cứu bảo hành điện tử*



Hình 4.42. *Giao diện tra cứu bảo hành điện tử*

4.13. Các chức năng bổ trợ

Bảng 4.1. *Các chức năng bổ trợ khác*

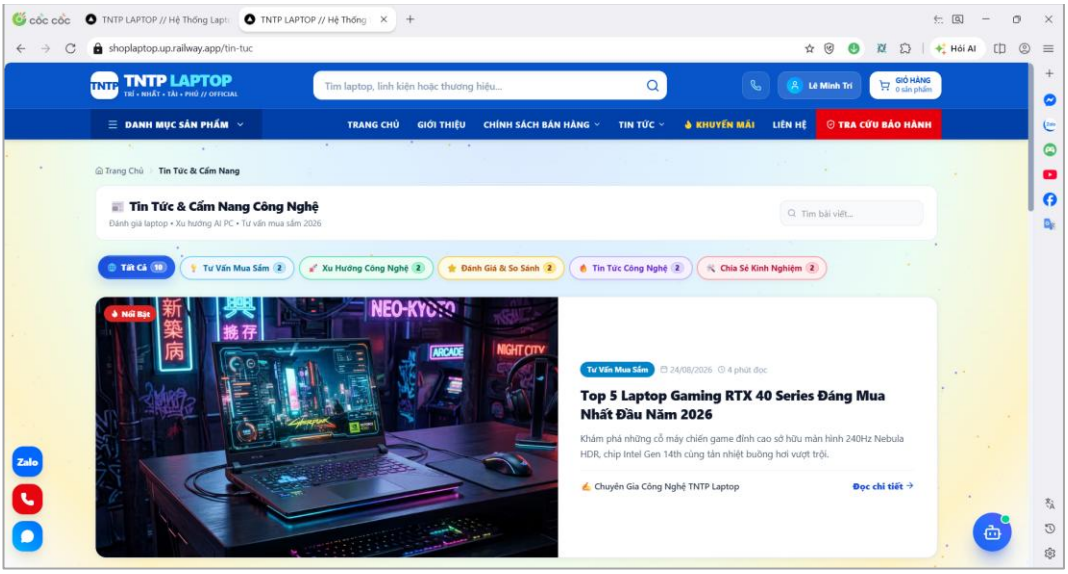
Chức năng	API	Điểm kỹ thuật
Đánh giá sản phẩm	`/api/danh-gia`	Sau khi thêm đánh giá, tính lại `diem_danh_gia_tb` và `so_luong_danh_gia` của sản phẩm; tăng lượt thích bằng `PATCH`
Liên hệ tư vấn	`/api/lien-he`	Kiểm tra SĐT Việt Nam, sinh mã phiếu duy nhất; danh sách có phân trang và thống kê theo trạng thái bằng `Promise.all` nhiều `countDocuments`
Tin tức	`/api/tin-tuc`	Tìm kiếm theo tiêu đề/tóm tắt/chuyên mục; xem chi tiết tự tăng `luot_xem` bằng `$inc`
Danh mục	`/api/danh-muc`	Slug tự sinh; sắp xếp theo `thu_tu_hien_thi`
Showroom	`/api/cai-dat/showroom`	Một document cấu hình duy nhất (`khoa = he_thong`), lưu tọa độ để hiển thị bản đồ

4.14. Một số giao diện kết quả

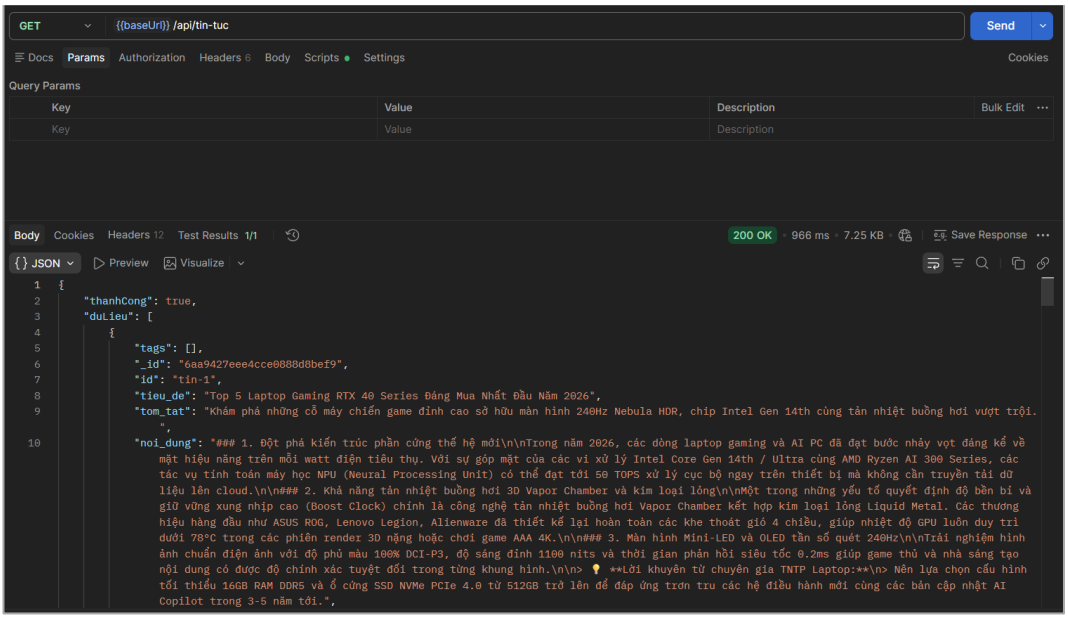
Các hình dưới đây là giao diện Front-End Next.js chạy tại `localhost:3000`, lấy dữ liệu thật từ API Express (`localhost:5000`) và MongoDB Atlas.



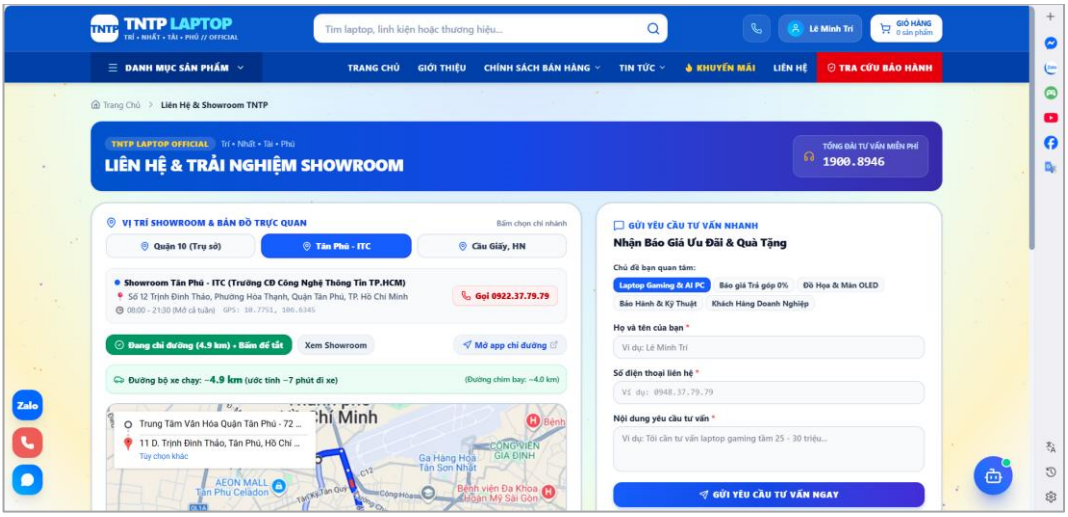
Hình 4.43. *Trang chủ website*



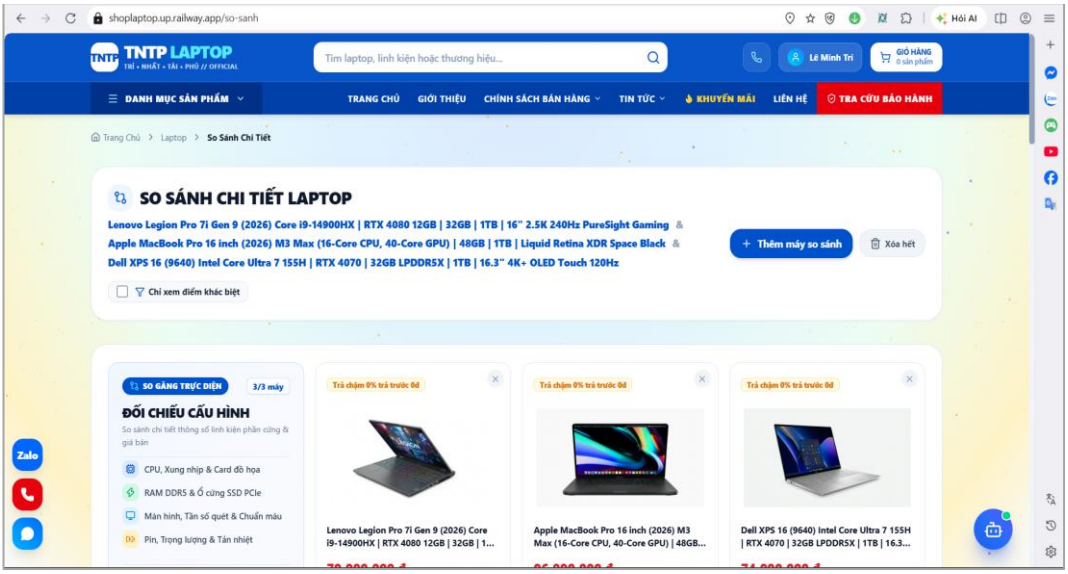
Hình 4.44. *Trang tin tức công nghệ*



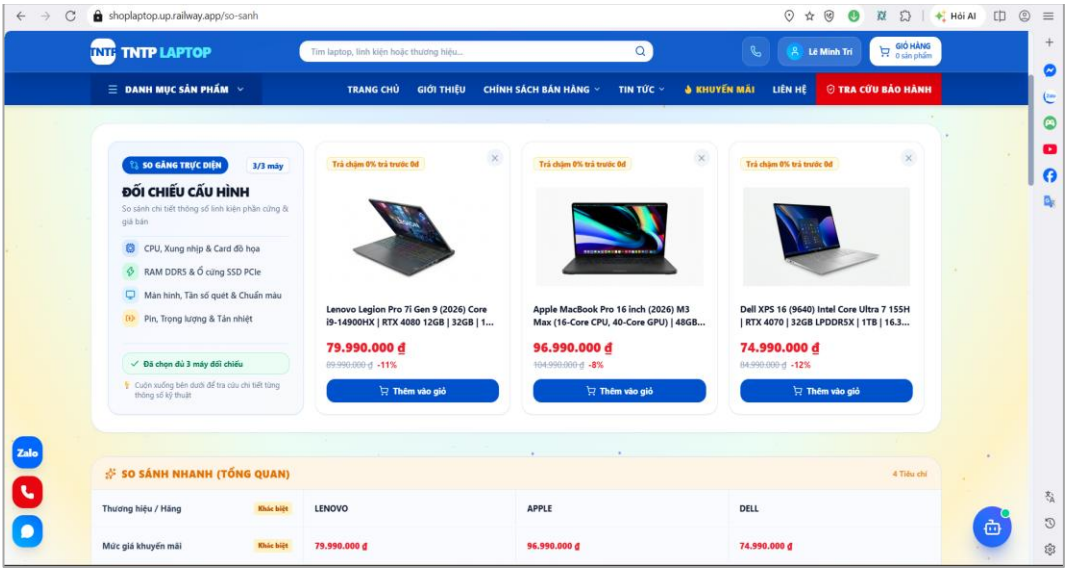
Hình 4.45. *Kiểm thử GET /api/tin-tuc bằng Postman: danh sách bài viết*



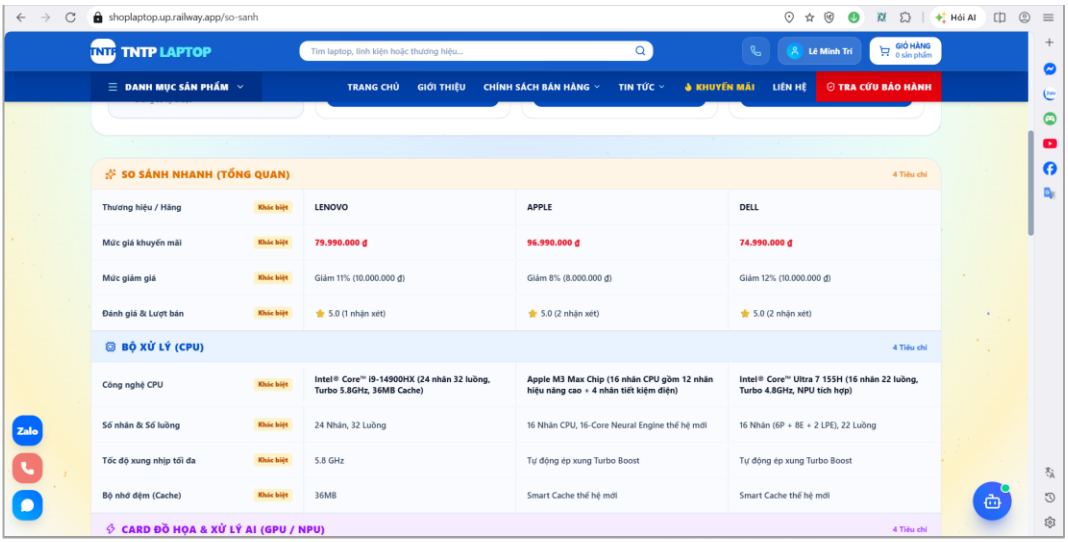
Hình 4.46. *Trang liên hệ và hệ thống showroom có bản đồ*



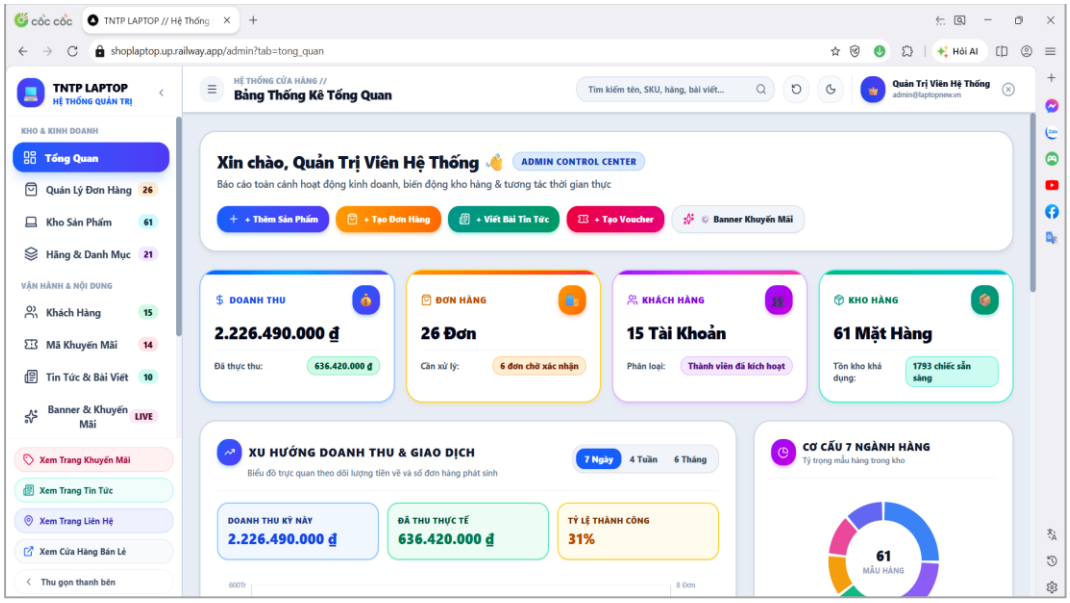
Hình 4.47. *Trang so sánh: danh sách laptop đã chọn để so sánh*



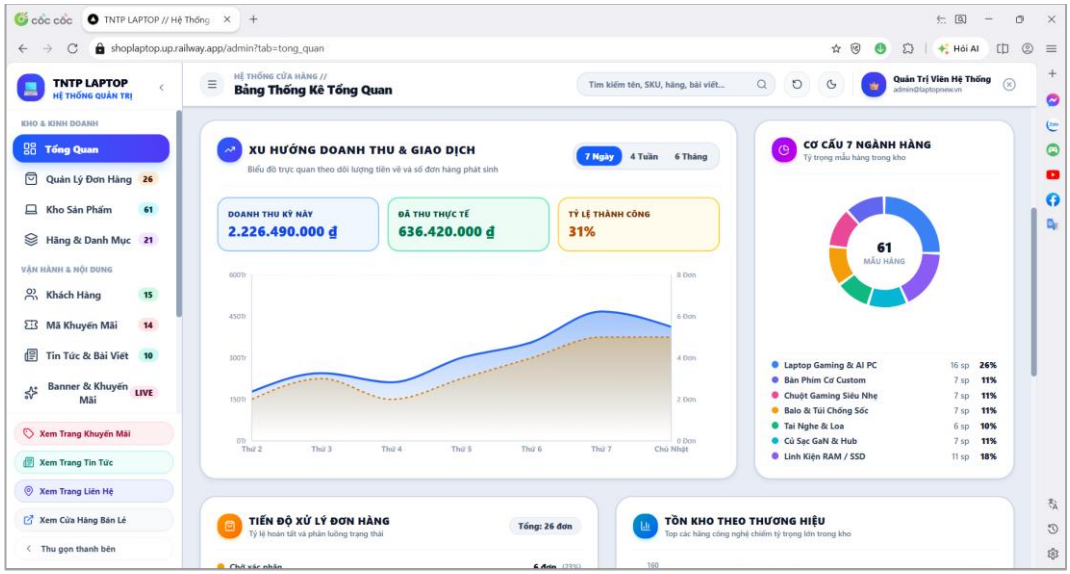
Hình 4.48. *Bảng đối chiếu nhanh thương hiệu, giá và đánh giá*



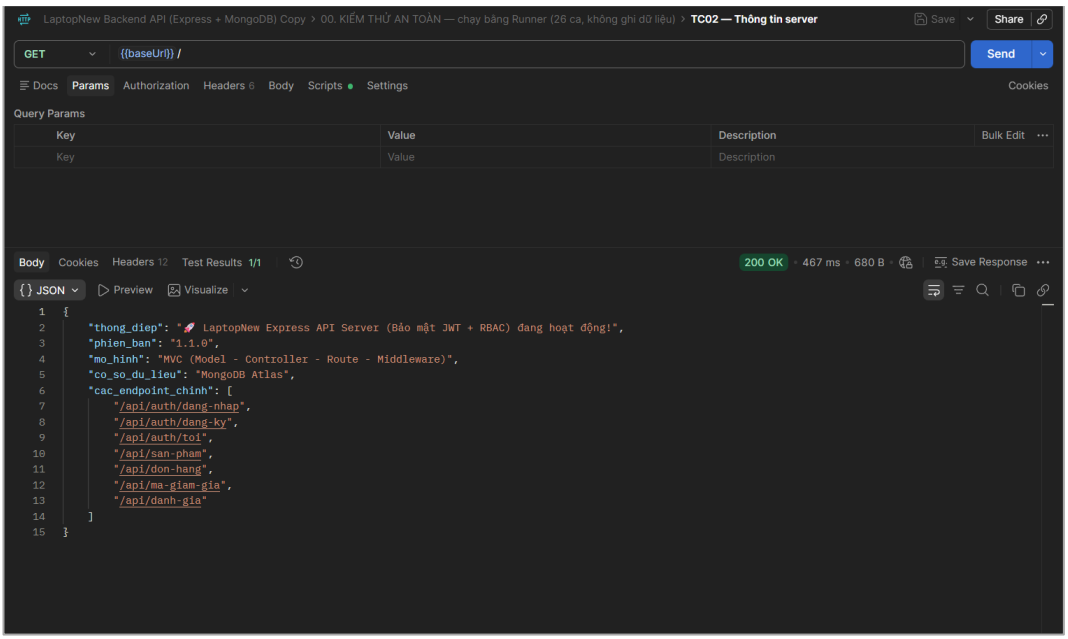
Hình 4.49. *So sánh chi tiết bộ xử lý (CPU) giữa các máy*



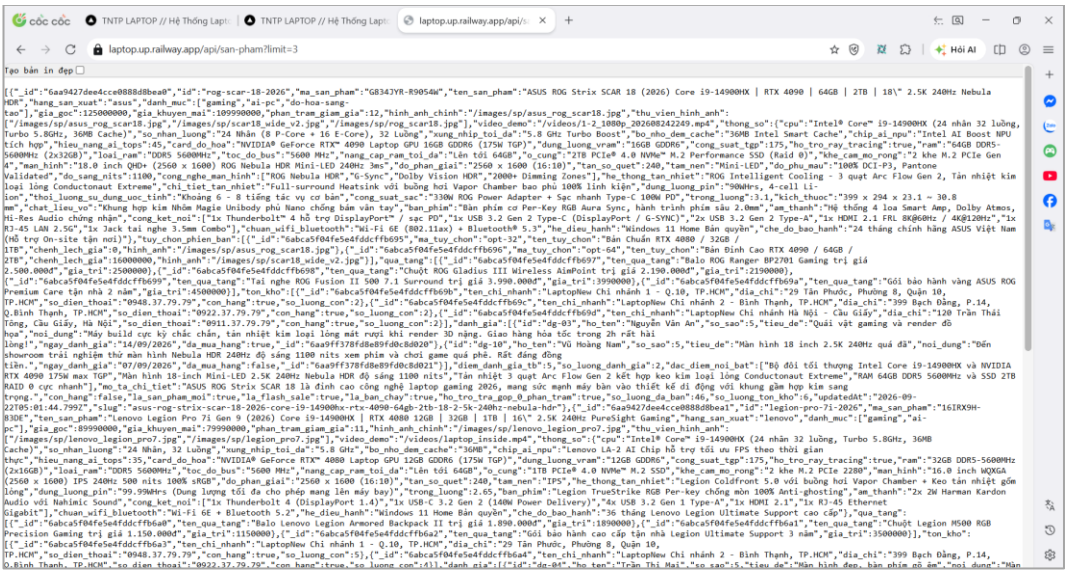
Hình 4.50. *Trang tổng quan quản trị: thẻ thống kê doanh thu, đơn hàng, khách hàng, kho*



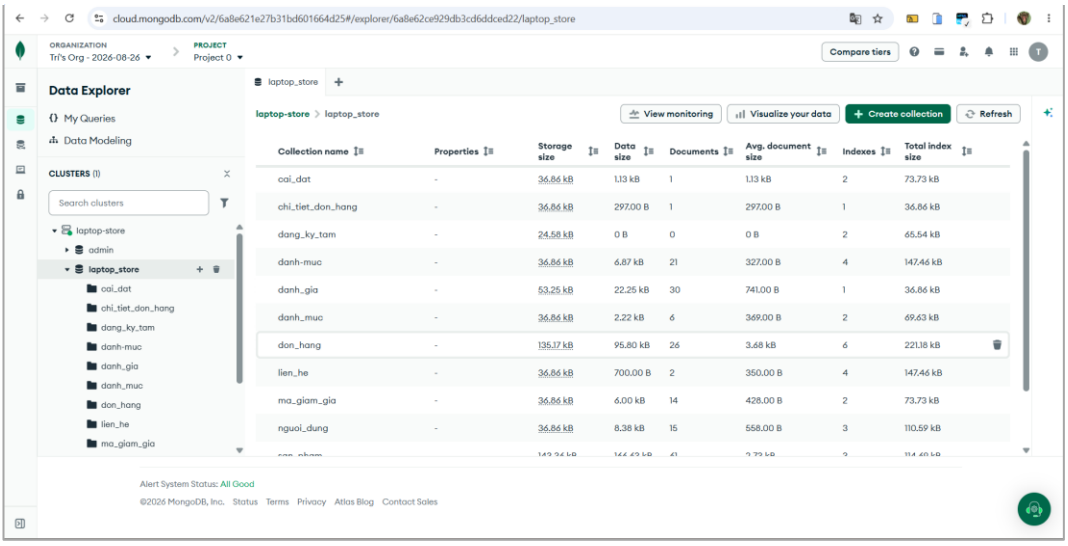
Hình 4.51. *Biểu đồ xu hướng doanh thu và cơ cấu ngành hàng*



Hình 4.52. *Kiểm thử GET / bằng Postman: thông tin server và danh sách endpoint chính*



Hình 4.53. *Dữ liệu JSON thực tế trả về từ GET /api/san-pham*



Hình 4.54. *Dữ liệu các collection trên MongoDB Atlas*

CHƯƠNG 5. XỬ LÝ LỖI VÀ BẢO MẬT

5.1. Chuẩn hóa mã trạng thái HTTP

Mọi controller đều bọc logic trong `try...catch` và trả về mã HTTP đúng ngữ nghĩa cùng thông điệp tiếng Việt dễ hiểu. Bảng sau tổng hợp các mã được dùng trong hệ thống.

Bảng 5.1. *Các mã trạng thái HTTP được sử dụng*

Mã	Tên	Tình huống thực tế trong hệ thống
200	OK	Đọc, cập nhật, xóa thành công; đăng nhập; kiểm tra voucher hợp lệ
201	Created	Tạo đơn hàng, sản phẩm, danh mục, voucher, đánh giá, phiếu liên hệ
400	Bad Request	Thiếu trường, sai định dạng email/SĐT, OTP sai/hết hạn, voucher chưa đủ điều kiện, chữ ký Webhook sai
401	Unauthorized	Thiếu/sai/hết hạn token; sai email hoặc mật khẩu; đặt hàng khi chưa đăng nhập
403	Forbidden	Không phải admin; tài khoản bị khóa; sửa đơn của người khác
404	Not Found	Không tìm thấy sản phẩm, đơn, voucher, người dùng; route không tồn tại
409	Conflict	Thêm sản phẩm hoặc danh mục trùng mã
410	Gone	API khôi phục showroom mặc định đã ngừng hỗ trợ
429	Too Many Requests	Tra cứu đơn hàng công khai quá 30 lần trong 10 phút từ cùng một IP
500	Internal Server Error	Lỗi truy vấn DB hoặc lỗi chưa lường trước — bắt bởi try/catch hoặc middleware lỗi cuối
502	Bad Gateway	Không gửi được email OTP do dịch vụ email bên ngoài lỗi

5.2. Xử lý từng loại lỗi

5.2.1. Lỗi 400 – Bad Request

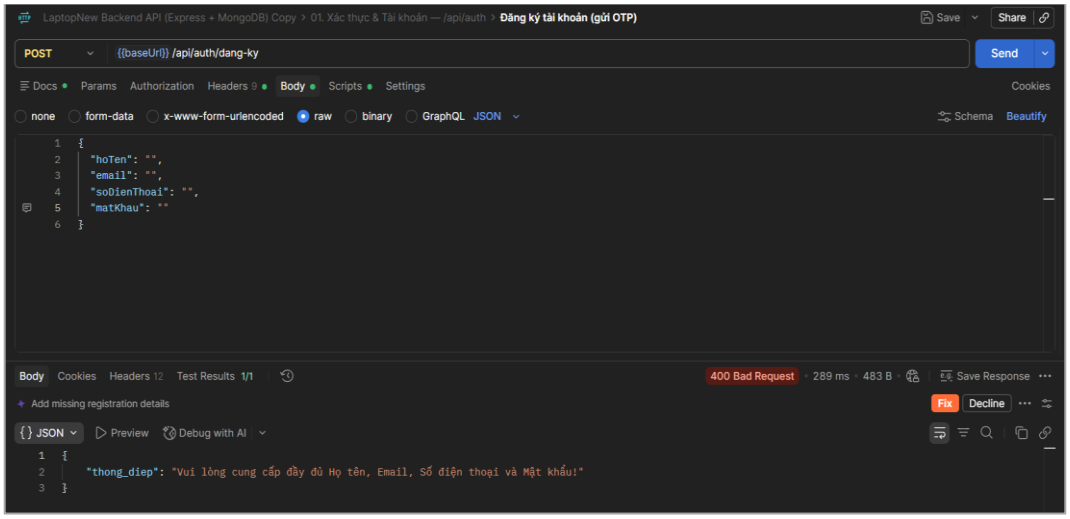
❖ **Nguyên nhân:**

- Dữ liệu client gửi lên thiếu hoặc sai định dạng (thiếu email, SĐT không đủ 10 số, mật khẩu mới dưới 6 ký tự).
- Mã OTP không đúng hoặc đã hết hạn; mã giảm giá chưa đủ giá trị đơn tối thiểu.

❖ **Cách xử lý:**

- Kiểm tra dữ liệu ngay đầu controller và `return res.status(400)` trước khi chạm vào cơ sở dữ liệu.
- Thông điệp nêu rõ cần sửa gì, ví dụ "Vui lòng cung cấp đầy đủ Họ tên, Email, Số điện thoại và Mật khẩu!".

```
POST /api/auth/dang-ky   { "hoTen": "A", "email": "x" }
HTTP/1.1 400 Bad Request
{ "thong_diep": "Vui lòng cung cấp đầy đủ Họ tên, Email, Số điện thoại và Mật khẩu!" }
```



Hình 5.1. *Kiểm thử lỗi 400 bằng Postman: đăng ký thiếu thông tin*

5.2.2. Lỗi 401 – Unauthorized

❖ **Nguyên nhân:**

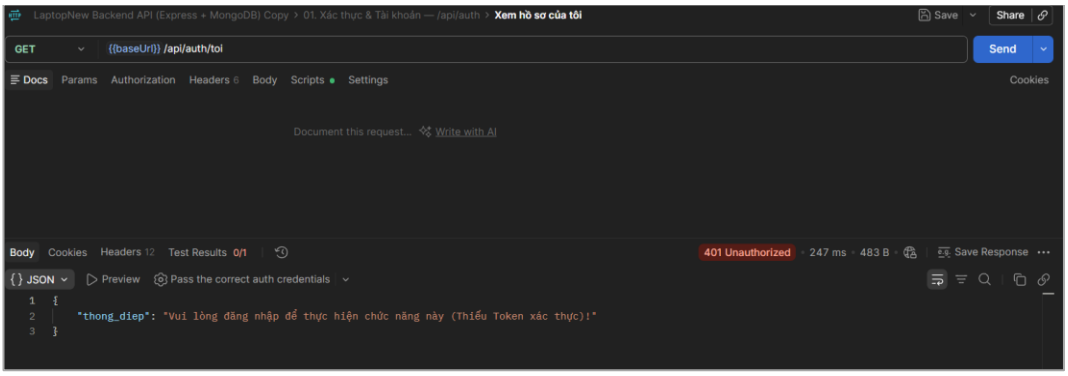
- Gọi API cần đăng nhập nhưng không gửi token, token sai chữ ký hoặc đã hết hạn 7 ngày.
- Đăng nhập sai email hoặc mật khẩu.

❖ **Cách xử lý:**

- Middleware `xacThucToken` phân biệt `TokenExpiredError` (yêu cầu đăng nhập lại) với token bị sửa đổi.
- Khi đăng nhập sai, trả cùng một thông điệp cho cả hai trường hợp để kẻ tấn công không dò được email nào tồn tại.

```
GET /api/auth/toi        (không gửi header Authorization)
HTTP/1.1 401 Unauthorized
{ "thong_diep": "Vui lòng đăng nhập để thực hiện chức năng này (Thiếu Token xác thực)!" }

GET /api/auth/toi        Authorization: Bearer abc.def.ghi
HTTP/1.1 401 Unauthorized
{ "thong_diep": "Token xác thực không hợp lệ hoặc đã bị thay đổi!" }
```



Hình 5.2. *Kiểm thử lỗi 401 bằng Postman: gọi /api/auth/toi không gửi token*

5.2.3. Lỗi 403 – Forbidden

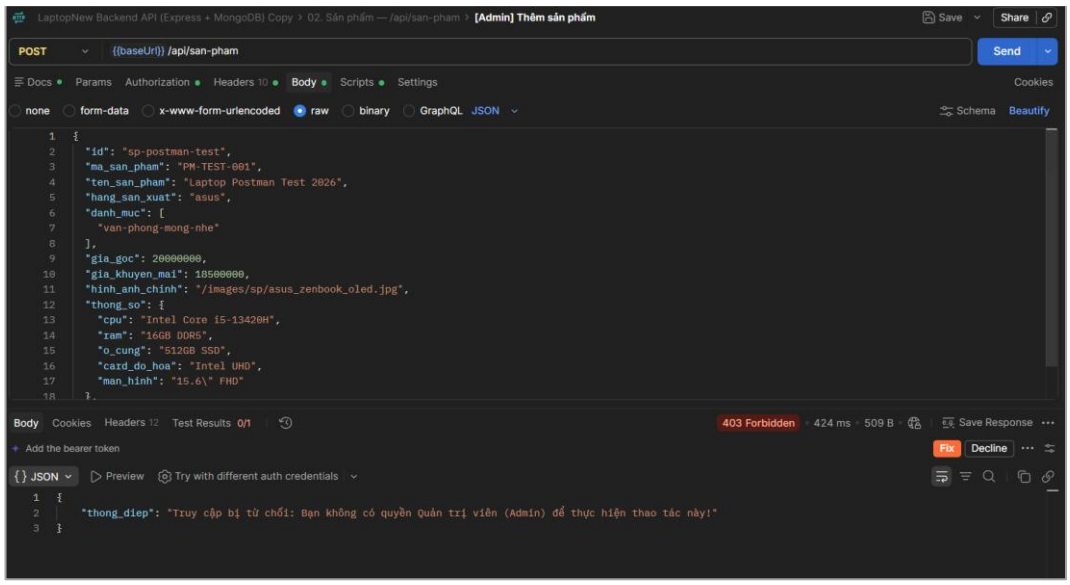
❖ **Nguyên nhân:**

- Tài khoản khách hàng gọi API dành cho admin (thêm/sửa/xóa sản phẩm, voucher, danh mục).
- Tài khoản bị khóa đăng nhập; khách cập nhật đơn không phải của mình.
- Khách tra cứu đơn hàng nhưng không nhập đúng số điện thoại nhận hàng, hoặc thành viên xem đơn của người khác.

❖ **Cách xử lý:**

- Middleware `chiAdmin` chặn trước khi vào controller; controller đơn hàng so sánh chủ đơn với `req.user`.

```
POST /api/san-pham       Authorization: Bearer <token của khach_hang>
HTTP/1.1 403 Forbidden
{ "thong_diep": "Truy cập bị từ chối: Bạn không có quyền Quản trị viên (Admin) để thực
hiện thao tác này!" }
```



Hình 5.3. *Kiểm thử lỗi 403 bằng Postman: tài khoản khách gọi API thêm sản phẩm*

5.2.4. Lỗi 404 – Not Found

❖ **Nguyên nhân:**

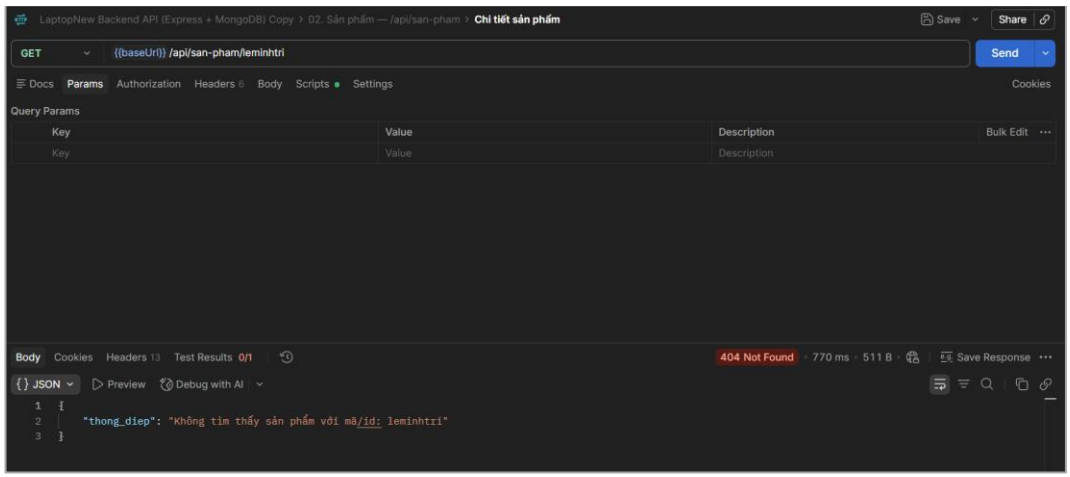
- Tìm tài nguyên theo id/slug/mã không tồn tại.
- Gọi sai đường dẫn API.

❖ **Cách xử lý:**

- Controller kiểm tra kết quả truy vấn `null` → trả 404 kèm mã đã tìm.
- Middleware cuối `server.js` bắt mọi route không khớp.

```
GET /api/san-pham/khong-ton-tai-xyz
HTTP/1.1 404 Not Found
{ "thong_diep": "Không tìm thấy sản phẩm với mã/id: khong-ton-tai-xyz" }

GET /api/khong-co-route
HTTP/1.1 404 Not Found
{ "thong_diep": "Đường dẫn không tồn tại: /api/khong-co-route" }
```



Hình 5.4. *Kiểm thử lỗi 404 bằng Postman: sản phẩm không tồn tại*

5.2.5. Lỗi 500 – Internal Server Error

❖ **Nguyên nhân:**

- Mất kết nối MongoDB, lỗi ép kiểu dữ liệu, lỗi logic chưa lường trước.

❖ **Cách xử lý:**

- Mỗi controller có `catch` trả 500 kèm `chi_tiet: loi.message` để dễ gỡ lỗi.
- Express 5 tự chuyển lỗi của hàm `async` tới middleware lỗi cuối cùng, nên server không bị sập.
- Ghi log `console.error(err.stack)` để lập trình viên truy vết.

```
// server.js – middleware xử lý lỗi tập trung
app.use((req, res) => {                                    // 404 cho route không tồn tại
    res.status(404).json({ thong_diep: `Đường dẫn không tồn tại: ${req.originalUrl}` });
});
app.use((err, req, res, next) => {                         // 500 cho lỗi chưa được bắt
    console.error('❌ Lỗi máy chủ chưa được bắt:', err.stack);
    res.status(500).json({ thong_diep: 'Đã xảy ra lỗi nội bộ trên máy chủ!', chi_tiet:
err.message });
});
```

5.2.6. Lỗi 429 – Too Many Requests

❖ **Nguyên nhân:**

- Một IP gọi `GET /api/don-hang/:id` quá **30 lần trong 10 phút** — dấu hiệu dò mã đơn hàng tự động.

❖ **Cách xử lý:**

- Middleware `gioiHanTraCuuDonHang` đếm số request theo IP trong bộ nhớ (`Map`), tự đặt lại bộ đếm sau mỗi cửa sổ 10 phút.

- Vượt giới hạn thì dừng ngay trước controller, không truy vấn cơ sở dữ liệu.

```
GET /api/don-hang/LPN-260930-5239?sdt=…   (lần thứ 31 trong 10 phút)
HTTP/1.1 429 Too Many Requests
{ "thong_diep": "Bạn tra cứu quá nhiều lần. Vui lòng thử lại sau ít phút." }
```

▣ CHÈN ẢNH CHỤP POSTMAN

Request: **04. Đơn hàng › Tra cứu đơn theo mã** — chạy bằng Runner với **31 iterations** (hoặc bấm Send liên tục)

Kỳ vọng: request thứ 31 trả **429 Too Many Requests**

(Xóa khung này, dán ảnh vào vị trí này — giữ nguyên dòng chú thích bên dưới)

Hình 5.5. *Kiểm thử lỗi 429 bằng Postman: tra cứu đơn vượt giới hạn*

5.3. Các biện pháp bảo mật đã áp dụng

Bảng 5.2. *Tổng hợp biện pháp bảo mật*

Biện pháp	Cách hiện thực	Chống lại
Băm mật khẩu	bcrypt salt 10 khi đăng ký, đặt lại, đổi mật khẩu; `toJSON()` loại `matKhau` khỏi mọi response	Lộ mật khẩu khi rò rỉ DB
Xác thực JWT	Ký HS256 bằng `JWT_SECRET`, hạn 7 ngày; middleware kiểm tra và tìm lại user trong DB	Giả mạo danh tính
Phân quyền RBAC	`chiAdmin` trên route quản trị sản phẩm, danh mục, voucher, người dùng (sửa/xóa), xóa đơn	Leo thang đặc quyền
Cách ly dữ liệu đơn hàng	Thành viên chỉ thấy đơn của mình; khách không điều kiện nhận `[]`	Lộ thông tin khách hàng
OTP một lần có hạn	OTP 6 số, hạn 10 phút, phân loại theo mục đích, xóa sau khi dùng; TTL index cho đăng ký tạm	Tài khoản ảo, chiếm tài khoản
Chữ ký Webhook	Tính lại HMAC-SHA256 bằng `PAYOS_CHECKSUM_KEY` và so sánh	Giả mạo thông báo đã thanh toán
CORS whitelist	Chỉ origin trong `CLIENT_URL` được gọi từ trình duyệt	Website lạ gọi API
Chống regex injection	Tra cứu bảo hành escape ký tự đặc biệt và khớp chính xác	Dò dữ liệu, ReDoS
Che dữ liệu nhạy cảm	Che SĐT khi tra bảo hành; che API key AI ở `/trang-thai`	Lộ thông tin cá nhân / khóa
Biến môi trường	Khóa bí mật đặt trong `.env`, tệp này nằm trong `.gitignore`	Lộ khóa trên GitHub
Tra cứu đơn hai yếu tố	Khách phải nhập đúng mã đơn **và** SĐT nhận hàng; chỉ admin / chủ đơn thấy đầy đủ	Dò đơn của người khác
Che dữ liệu đơn công khai	`anThongTinDonHangCongKhai()` che họ tên, SĐT, email; bỏ địa chỉ chi tiết, ghi chú	Lộ thông tin cá nhân
Giới hạn tần suất	`gioiHanTraCuuDonHang`: 30 request / 10 phút / IP, vượt quá trả 429	Dò mã đơn hàng loạt

5.4. Đánh giá rủi ro còn tồn tại và hướng khắc phục

Trong quá trình rà soát mã nguồn để viết báo cáo, nhóm phát hiện một số điểm chưa an toàn. Nhóm trình bày trung thực để làm cơ sở cho giai đoạn hoàn thiện tiếp theo.

Bảng 5.3. *Các rủi ro bảo mật phát hiện khi rà soát mã nguồn*

Mức	Vấn đề	Hướng khắc phục
Nghiêm trọng	`POST /api/auth/google` chấp nhận `email` do client gửi khi không có hoặc không xác minh được Google ID Token → có thể đăng nhập vào tài khoản bất kỳ	Chỉ chấp nhận `credential` đã `verifyIdToken` thành công; bỏ nhánh giải mã payload không kiểm tra chữ ký và nhánh nhận `email` trực tiếp
Nghiêm trọng	`POST /api/nguoi-dung` và `GET /api/nguoi-dung` công khai; POST lưu nguyên `req.body` (có thể tự gán `vaiTro: admin`, mật khẩu không được băm)	Gắn `xacThucToken, chiAdmin`; chỉ nhận các trường cho phép và băm mật khẩu
Cao	CRUD `/api/tin-tuc`, quản trị `/api/lien-he` và `PUT /api/cai-dat/showroom` chưa có middleware xác thực	Gắn `xacThucToken, chiAdmin` cho các thao tác ghi và xem danh sách liên hệ
Cao	Chủ đơn được tự cập nhật `da_thanh_toan`, `trang_thai_thanh_toan` qua `PUT /api/don-hang/:id`; `tao-link` lấy `tong_tien` từ client	Chỉ Webhook/polling PayOS được đổi trạng thái thanh toán; tính số tiền từ đơn trong DB
Trung bình	`JWT_SECRET` có giá trị dự phòng viết cứng trong mã; tự tạo tài khoản `admin/admin123` khi DB chưa có admin	Bắt buộc cấu hình `JWT_SECRET` (dừng server nếu thiếu); tạo admin bằng script seed với mật khẩu mạnh
Trung bình	Tìm kiếm sản phẩm/tin tức đưa trực tiếp từ khóa vào `$regex`; mới giới hạn tần suất cho tra cứu đơn, chưa áp dụng cho đăng nhập / gửi OTP	Escape từ khóa như API bảo hành; áp dụng giới hạn tần suất (`express-rate-limit`) cho `/api/auth/*`
Thấp	Khai báo index trùng `payos_order_code`, `ma_don_hang` (Mongoose cảnh báo khi khởi động)	Xóa khai báo `schema.index()` trùng với `index: true` / `unique: true`

CHƯƠNG 6. TRIỂN KHAI VÀ KIỂM THỬ

6.1. Yêu cầu môi trường

- Node.js 18 trở lên (nhóm dùng Node.js 24) và npm.
- Tài khoản MongoDB Atlas, mở Network Access cho IP máy chạy server.
- Tùy chọn: tài khoản PayOS, Telegram Bot (BotFather), Brevo, Google Cloud (OAuth Client ID), Google AI Studio (Gemini API key).

6.2. Cấu hình biến môi trường

Tạo tệp `BE/.env` với các biến sau (giá trị thật không đưa vào báo cáo vì là thông tin bí mật):

Bảng 6.1. *Danh sách biến môi trường của Back-End*

Biến	Bắt buộc	Ý nghĩa
`PORT`	Không	Cổng server, mặc định 5000
`MONGODB_URI`	Có	Chuỗi kết nối MongoDB Atlas
`CLIENT_URL`	Có	Danh sách origin Front-End được phép (cách nhau dấu phẩy)
`JWT_SECRET`, `JWT_EXPIRES_IN`	Có	Khóa ký JWT và thời hạn token (ví dụ `7d`)
`GOOGLE_CLIENT_ID`, `GOOGLE_CLIENT_SECRET`	Khi dùng Google	Xác minh đăng nhập Google
`EMAIL_USER`, `EMAIL_PASS`	Khi chạy local	Gmail và App Password cho Nodemailer
`BREVO_API_KEY`, `BREVO_SENDER_EMAIL`	Khi deploy cloud	Gửi email qua Brevo API
`PAYOS_CLIENT_ID`, `PAYOS_API_KEY`, `PAYOS_CHECKSUM_KEY`	Khi dùng PayOS	Khóa gọi API và ký/xác minh chữ ký
`PAYMENT_BANK_BIN`, `PAYMENT_BANK_ACCOUNT_NO`, `PAYMENT_BANK_ACCOUNT_NAME`	Khi dùng VietQR	Tài khoản nhận tiền để dựng ảnh QR
`TELEGRAM_BOT_TOKEN`, `TELEGRAM_CHAT_ID`	Khi dùng Bot	Token bot và chat ID của quản trị viên
`GEMINI_API_KEYS`	Khi dùng AI	Nhiều key Gemini cách nhau dấu phẩy để xoay vòng

Biến	Bắt buộc	Ý nghĩa
`NINE_ROUTER_API_KEY`, `NINE_ROUTER_BASE_URL`, `NINE_ROUTER_MODEL`	Tùy chọn	Cấu hình cổng AI 9Router

6.3. Cài đặt và khởi chạy

Các lệnh cài đặt và chạy dự án

```
# 1. Cài thư viện cho Back-End
cd LAPTOP_NEW/BE
npm install

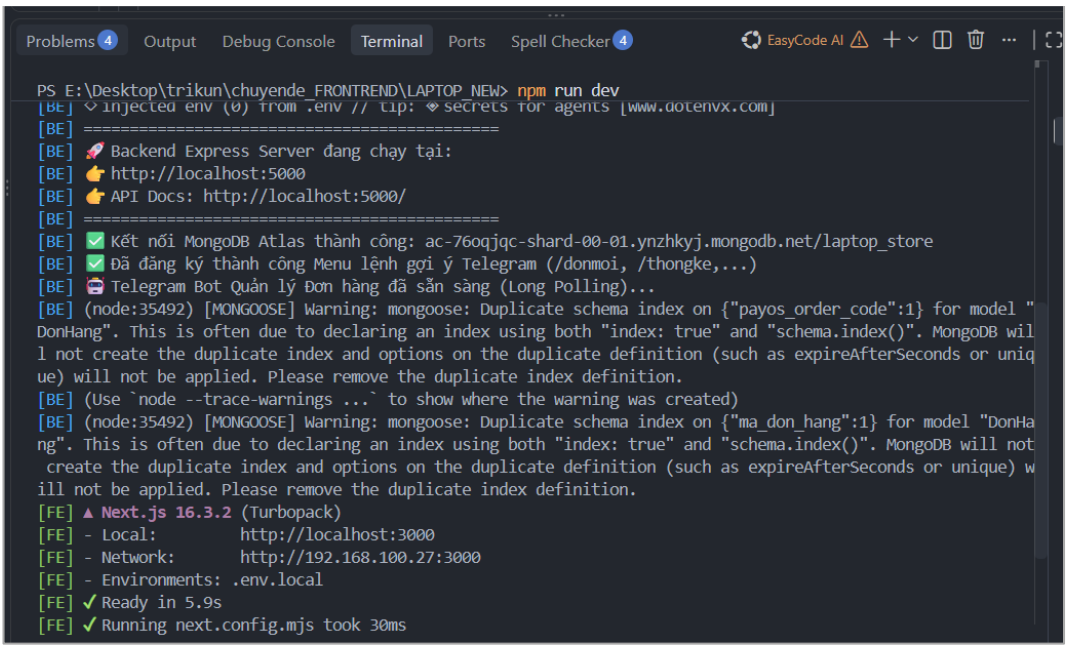
# 2. Chạy ở chế độ phát triển (tự khởi động lại khi sửa code)
npm run dev          # nodemon server.js
# hoặc chạy thường
npm start            # node server.js

# 3. (Tùy chọn) Chạy Front-End để thử giao diện
cd ../FE
npm install
npm run dev          # http://localhost:3000

# Hoặc chạy đồng thời cả hai bằng tệp chay.bat ở thư mục gốc (Windows)
```

Khi khởi chạy thành công, cửa sổ terminal hiển thị địa chỉ server và thông báo kết nối MongoDB Atlas:

```
==============================================
🚀 Backend Express Server đang chạy tại:
👉 http://localhost:5000
👉 API Docs: http://localhost:5000/
==============================================
☑ Kết nối MongoDB Atlas thành công: ac-76oqjqc-shard-00-
00.ynzhkyj.mongodb.net/laptop_store
```



Hình 6.1. *Terminal khởi chạy Back-End thành công*

6.4. Kiểm thử API

Nhóm kiểm thử API theo phương pháp **hộp đen**: gửi request với dữ liệu hợp lệ và không hợp lệ, so sánh mã HTTP và nội dung phản hồi với kết quả mong đợi. Ca TC01–TC26 được **chạy thực tế ngày 29/09/2026** trên server local kết nối MongoDB Atlas; ca TC27–TC30 kiểm tra tính năng tra cứu đơn an toàn, **chạy ngày 30/09/2026** trên server Railway. Chỉ dùng thao tác đọc và thao tác bị từ chối để không làm thay đổi dữ liệu thật.

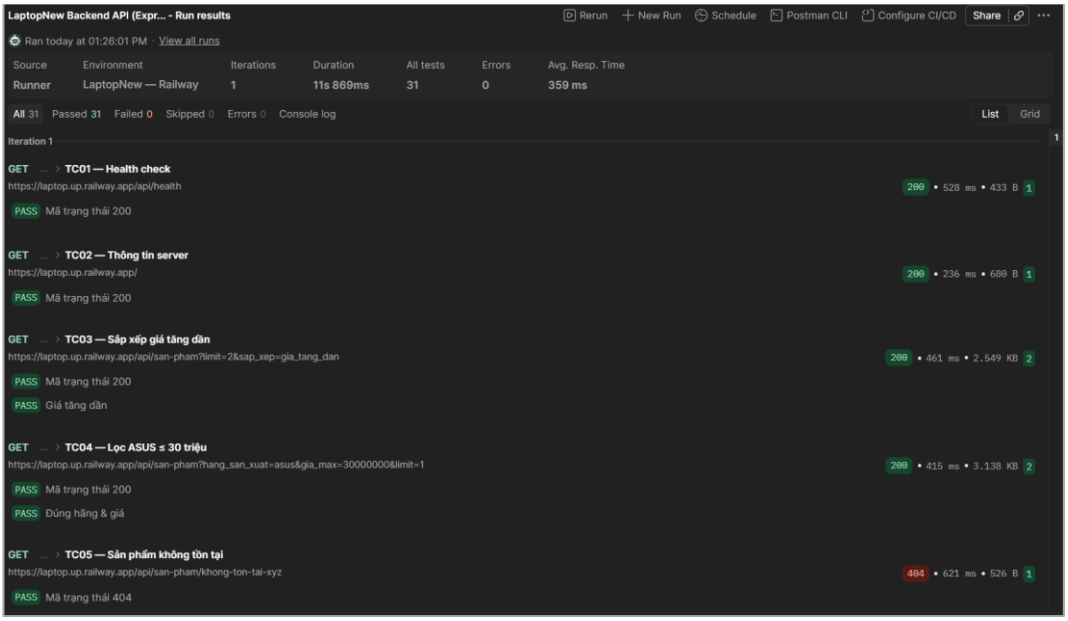
Bảng 6.2. *Kết quả kiểm thử API thực tế*

Mã	API	Dữ liệu vào	Mong đợi	Thực tế	KQ
Giám sát và sản phẩm					
TC01	GET /api/health	—	200, trang_thai OK	200 `{"trang_thai":"OK"}`	✓ Đạt
TC02	GET /	—	200, thông tin server	200, phien_ban 1.1.0	✓ Đạt
TC03	GET /api/san-pham	`limit=2&sap_xep=gia_tang_dan`	200, 2 sản phẩm giá tăng dần	200, 2 phần tử, rẻ nhất trước	✓ Đạt
TC04	GET /api/san-pham	`hang_san_xuat=asus&gia_max=30000000`	200, laptop ASUS ≤ 30 triệu	200, ASUS Zenbook 14 OLED 29.990.000đ	✓ Đạt
TC05	GET /api/san-pham/:id	`khong-ton-tai-xyz`	404	404 “Không tìm thấy sản phẩm...”	✓ Đạt
TC06	GET /api/khong-co-route	—	404 route	404 “Đường dẫn không tồn tại...”	✓ Đạt
Xác thực và phân quyền					
TC07	GET /api/auth/toi	Không có token	401	401 “Thiếu Token xác thực”	✓ Đạt

Mã	API	Dữ liệu vào	Mong đợi	Thực tế	KQ
TC08	GET /api/auth/toi	Token giả `abc.def.ghi`	401	401 “Token... không hợp lệ”	✓ Đạt
TC09	POST /api/san-pham	Không có token	401 (route admin)	401	✓ Đạt
TC10	POST /api/auth/dang-nhap	Email không tồn tại	401	401 “Tài khoản hoặc Mật khẩu không chính xác!”	✓ Đạt
TC11	POST /api/auth/dang-nhap	`email` rỗng	400	400 “Vui lòng nhập đầy đủ...”	✓ Đạt
TC12	POST /api/auth/dang-ky	Thiếu SĐT, mật khẩu	400	400 “Vui lòng cung cấp đầy đủ...”	✓ Đạt
Nghiệp vụ					
TC13	POST /api/ma-giam-gia/kiem-tra	Mã `KHONGCO999`	404, hop_le false	404, so_tien_giam 0	✓ Đạt
TC14	POST /api/ma-giam-gia/kiem-tra	Mã rỗng	400	400 “Vui lòng nhập mã giảm giá.”	✓ Đạt
TC15	GET /api/ma-giam-gia	`kich_hoat=true`	200, danh sách voucher	200, 14 voucher	✓ Đạt
TC16	GET /api/bao-hanh/tra-cuu	`tu_khoa=ab`	Từ chối từ khóa < 3 ký tự	200, hop_le false, thông báo quá ngắn	✓ Đạt
TC17	GET /api/bao-hanh/tra-cuu	Mã đơn không tồn tại	Không tìm thấy	200, hop_le false	✓ Đạt
TC18	GET /api/don-hang	Khách, không điều kiện	Không lộ đơn	200 `[]`	✓ Đạt
TC19	POST /api/don-hang	Chưa đăng nhập	401, yêu cầu đăng nhập	401, yeu_cau_dang_nhap true	✓ Đạt
TC20	GET /api/danh-muc	—	200	200, 21 danh mục	✓ Đạt
TC21	GET /api/tin-tuc	—	200	200, thanhCong true	✓ Đạt
TC22	GET /api/cai-dat/showroom	—	200	200, 3 showroom	✓ Đạt
Thanh toán và AI					
TC23	POST /api/thanh-toan/webhook	Chữ ký sai	400	400 “Chữ ký không hợp lệ”	✓ Đạt
TC24	GET /api/thanh-toan/kiem-tra/123	orderCode chưa thanh toán	200, chưa thanh toán	200, da_thanh_toan false, PENDING	✓ Đạt
TC25	POST /api/tro-ly-ai/chat/stream	`tin_nhan` rỗng	400	400 “Vui lòng cung cấp nội dung...”	✓ Đạt
TC26	GET /api/tro-ly-ai/trang-thai	—	200, trạng thái AI	200, 9Router đã cấu hình, 5 key Gemini	✓ Đạt
Bảo mật tra cứu đơn hàng					
TC27	GET /api/don-hang/:ma	Khách, không gửi `sdt`	403, yêu cầu xác minh SĐT	403, `yeu_cau_xac_minh_sdt` true	✓ Đạt

Mã	API	Dữ liệu vào	Mong đợi	Thực tế	KQ
TC28	GET /api/don-hang/:ma	Khách, `sdt` sai	403	403 "Vui lòng nhập đúng mã đơn hàng và số điện thoại..."	✓ Đạt
TC29	GET /api/don-hang/:ma	Khách, `sdt` đúng	200, dữ liệu cá nhân đã che	200, họ tên `L* M*** T**`, SĐT `034****438`	✓ Đạt
TC30	GET /api/don-hang/:ma	Token của chủ đơn	200, đầy đủ thông tin	200, đầy đủ `thong_tin_giao_hang`	✓ Đạt

> **Kết quả:** 30/30 ca kiểm thử đạt. Các chức năng có ghi dữ liệu (đăng ký thật, đặt hàng, thanh toán, duyệt đơn Telegram) được kiểm thử thủ công qua giao diện và minh họa bằng ảnh chụp ở Chương 4.



Hình 6.2. *Bộ kiểm thử chạy bằng Postman Runner*

6.5. Minh chứng kiểm thử API bằng Postman

Toàn bộ API được gom trong collection **LaptopNew Backend API** (tệp `_tai_lieu_bao_cao/postman/`), chia 16 thư mục theo nhóm chức năng, dùng chung biến môi trường `baseUrl`, `token`, `adminToken`, `productId`, `maDonHang`, `orderCode`. Request đăng nhập tự lưu token vào biến môi trường nên các request sau chỉ cần bấm **Send**. Bảng dưới đây tổng hợp minh chứng Postman cho từng nhóm API; các hình còn trống ở mục 6.5.1 – 6.5.5 là vị trí chèn ảnh chụp.

Bảng 6.3. *Tổng hợp minh chứng kiểm thử Postman theo nhóm API*

STT	Nhóm API	Request tiêu biểu	Hình minh chứng
1	Xác thực & tài khoản	Đăng ký, kích hoạt OTP, đăng nhập, xác nhận OTP, xem hồ sơ, đổi mật khẩu	Hình 4.7, 4.4, 4.15, 6.3, 6.4, 6.5

STT	Nhóm API	Request tiêu biểu	Hình minh chứng
2	Sản phẩm & danh mục	Danh sách, lọc nhiều điều kiện, thêm sản phẩm (Admin), lọc thương hiệu	Hình 3.1, 6.6, 6.7, 6.8
3	Đơn hàng	Tạo đơn, tra cứu (chủ đơn / khách có SĐT / thiếu SĐT), cập nhật trạng thái	Hình 6.9, 4.25, 4.24, 4.26, 6.10
4	Thanh toán	Tạo QR, kiểm tra trạng thái, Webhook sai chữ ký	Hình 6.11, 6.12, 6.13
5	Mã giảm giá	Kiểm tra mã, lưu vào ví, xem ví	Hình 4.21, 6.14
6	Đánh giá, bảo hành, liên hệ	Gửi đánh giá, tra cứu bảo hành, gửi phiếu tư vấn	Hình 6.15, 6.16, 6.17
7	Tin tức, AI, showroom	Danh sách tin, chat AI, trạng thái AI, showroom	Hình 4.45, 4.40, 6.18, 6.19
8	Giám sát máy chủ	Thông tin server, health check	Hình 4.52, 6.20
9	Kịch bản lỗi	400, 401, 403, 404, 429	Hình 5.1, 5.2, 5.3, 5.4, 5.5
10	Chạy tự động	Runner 26 ca kiểm thử an toàn	Hình 6.2

6.5.1. Nhóm Xác thực và Tài khoản

▣ CHÈN ẢNH CHỤP POSTMAN

Request: **01. Xác thực & Tài khoản › Đăng ký tài khoản (gửi OTP)**

Kỳ vọng: **200**, thông điệp đã gửi mã OTP tới email

(Xóa khung này, dán ảnh vào vị trí này — giữ nguyên dòng chú thích bên dưới)

Hình 6.3. *Kiểm thử POST /api/auth/dang-ky bằng Postman: đăng ký hợp lệ, gửi OTP*

▣ CHÈN ẢNH CHỤP POSTMAN

Request: **01. Xác thực & Tài khoản › Xem hồ sơ của tôi** (Bearer `{{token}}`)

Kỳ vọng: **200**, trả thông tin người dùng, không có trường `matKhau`

(Xóa khung này, dán ảnh vào vị trí này — giữ nguyên dòng chú thích bên dưới)

Hình 6.4. *Kiểm thử GET /api/auth/toi với token hợp lệ*

▣ **CHÈN ẢNH CHỤP POSTMAN**

Request: **01. Xác thực & Tài khoản › Đổi mật khẩu — bước 2: xác nhận**

Kỳ vọng: **200**, đổi mật khẩu thành công khi OTP đúng loại `doi_mat_khau`

(Xóa khung này, dán ảnh vào vị trí này — giữ nguyên dòng chú thích bên dưới)

Hình 6.5. *Kiểm thử PUT /api/auth/doi-mat-khau bằng Postman*

6.5.2. Nhóm Sản phẩm và Danh mục

▣ **CHÈN ẢNH CHỤP POSTMAN**

Request: **02. Sản phẩm › Lọc nhiều hãng + danh mục gaming**

Kỳ vọng: **200**, chỉ gồm ASUS / MSI / Lenovo thuộc danh mục gaming, giá giảm dần

(Xóa khung này, dán ảnh vào vị trí này — giữ nguyên dòng chú thích bên dưới)

Hình 6.6. *Kiểm thử lọc sản phẩm theo nhiều hãng và danh mục*

▣ **CHÈN ẢNH CHỤP POSTMAN**

Request: **02. Sản phẩm › [Admin] Thêm sản phẩm** (Bearer `{{adminToken}}`)

Kỳ vọng: **201**, trả sản phẩm `sp-postman-test` vừa tạo (chạy thêm Cập nhật / Xóa để dọn dữ liệu)

(Xóa khung này, dán ảnh vào vị trí này — giữ nguyên dòng chú thích bên dưới)

Hình 6.7. *Kiểm thử POST /api/san-pham bằng tài khoản Admin (201 Created)*

▣ **CHÈN ẢNH CHỤP POSTMAN**

Request: **03. Danh mục › Lọc thương hiệu**

Kỳ vọng: **200**, danh sách danh mục có `loai = thuong_hieu`

(Xóa khung này, dán ảnh vào vị trí này — giữ nguyên dòng chú thích bên dưới)

Hình 6.8. *Kiểm thử GET /api/danh-muc?loai=thuong_hieu*

6.5.3. Nhóm Đơn hàng và Thanh toán

▣ **CHÈN ẢNH CHỤP POSTMAN**

Request: **04. Đơn hàng › Tạo đơn hàng (khách đã đăng nhập)**

Kỳ vọng: **201**, trả `ma_don_hang` dạng LPN-2026-xxxx, email hóa đơn và tin Telegram được gửi

(Xóa khung này, dán ảnh vào vị trí này — giữ nguyên dòng chú thích bên dưới)

Hình 6.9. *Kiểm thử POST /api/don-hang: tạo đơn thành công (201 Created)*

▣ **CHÈN ẢNH CHỤP POSTMAN**

Request: **04. Đơn hàng › [Admin] Đã giao (tự trừ kho)**

Kỳ vọng: **200**, `trang_thai = da_giao`, `da_tru_ton_kho = true`

(Xóa khung này, dán ảnh vào vị trí này — giữ nguyên dòng chú thích bên dưới)

Hình 6.10. *Kiểm thử PUT /api/don-hang/:id: Admin cập nhật trạng thái đơn*

▣ **CHÈN ẢNH CHỤP POSTMAN**

Request: **05. Thanh toán › Tạo link / QR thanh toán**

Kỳ vọng: **200**, có `orderCode`, `qrImageUrl`, `checkoutUrl`, nội dung chuyển khoản TRIKUN xxxxxx

(Xóa khung này, dán ảnh vào vị trí này — giữ nguyên dòng chú thích bên dưới)

Hình 6.11. *Kiểm thử POST /api/thanh-toan/tao-link: tạo mã QR thanh toán*

▣ **CHÈN ẢNH CHỤP POSTMAN**

Request: **05. Thanh toán › Kiểm tra trạng thái thanh toán**

Kỳ vọng: **200**, `da_thanh_toan` true/false theo đơn thực tế

(Xóa khung này, dán ảnh vào vị trí này — giữ nguyên dòng chú thích bên dưới)

Hình 6.12. *Kiểm thử GET /api/thanh-toan/kiem-tra/:orderCode*

▣ **CHÈN ẢNH CHỤP POSTMAN**

Request: **05. Thanh toán › Webhook với chữ ký sai (phải bị từ chối)**

Kỳ vọng: **400** `"Chữ ký không hợp lệ"`

(Xóa khung này, dán ảnh vào vị trí này — giữ nguyên dòng chú thích bên dưới)

Hình 6.13. *Kiểm thử Webhook PayOS với chữ ký giả mạo bị từ chối (400)*

6.5.4. Nhóm Mã giảm giá, Đánh giá, Bảo hành và Liên hệ

▣ **CHÈN ẢNH CHỤP POSTMAN**

Request: **06. Mã giảm giá › Lưu voucher vào ví**, sau đó **Ví voucher của tôi**

Kỳ vọng: **200**, ví có mã vừa lưu

(Xóa khung này, dán ảnh vào vị trí này — giữ nguyên dòng chú thích bên dưới)

Hình 6.14. *Kiểm thử lưu voucher và xem ví voucher của tôi*

▣ CHÈN ẢNH CHỤP POSTMAN

Request: **07. Đánh giá › Gửi đánh giá mới**

Kỳ vọng: **201**, điểm trung bình sản phẩm được tính lại

(Xóa khung này, dán ảnh vào vị trí này — giữ nguyên dòng chú thích bên dưới)

Hình 6.15. *Kiểm thử POST /api/danh-gia: gửi đánh giá sản phẩm*

▣ CHÈN ẢNH CHỤP POSTMAN

Request: **10. Bảo hành › Tra cứu theo mã đơn**

Kỳ vọng: **200**, `hop_le: true`, hạn bảo hành 24 tháng, SĐT đã che

(Xóa khung này, dán ảnh vào vị trí này — giữ nguyên dòng chú thích bên dưới)

Hình 6.16. *Kiểm thử GET /api/bao-hanh/tra-cuu: tra cứu bảo hành điện tử*

▣ CHÈN ẢNH CHỤP POSTMAN

Request: **11. Liên hệ CSKH › Gửi yêu cầu tư vấn**

Kỳ vọng: **201**, trả `ma_yeu_cau` của phiếu mới

(Xóa khung này, dán ảnh vào vị trí này — giữ nguyên dòng chú thích bên dưới)

Hình 6.17. *Kiểm thử POST /api/lien-he: gửi phiếu yêu cầu tư vấn*

6.5.5. Nhóm Trợ lý AI, Cài đặt và Giám sát máy chủ

▣ CHÈN ẢNH CHỤP POSTMAN

Request: **13. Trợ lý AI › Trạng thái AI (9Router + Gemini keys)**

Kỳ vọng: **200**, trạng thái 9Router, số key Gemini (đã che)

(Xóa khung này, dán ảnh vào vị trí này — giữ nguyên dòng chú thích bên dưới)

Hình 6.18. *Kiểm thử GET /api/tro-ly-ai/trang-thai*

▣ CHÈN ẢNH CHỤP POSTMAN

Request: **12. Cài đặt showroom › Danh sách showroom**

Kỳ vọng: **200**, 3 showroom kèm tọa độ `vi_do`, `kinh_do`

(Xóa khung này, dán ảnh vào vị trí này — giữ nguyên dòng chú thích bên dưới)

Hình 6.19. *Kiểm thử GET /api/cai-dat/showroom*

▣ **CHÈN ẢNH CHỤP POSTMAN**

Request: **14. Giám sát máy chủ › Health check**

Kỳ vọng: **200** `{ "trang_thai": "OK", "thoi_gian": ... }`

(Xóa khung này, dán ảnh vào vị trí này — giữ nguyên dòng chú thích bên dưới)

Hình 6.20. *Kiểm thử GET /api/health: kiểm tra sức khỏe máy chủ*

CHƯƠNG 7. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

7.1. Kết luận

❖ Kết quả đạt được

- Xây dựng hoàn chỉnh Back-End cho website bán laptop với **13 nhóm tài nguyên, 69 route**, tổ chức rõ ràng theo các tầng Route – Middleware – Controller – Service – Model.
- Thiết kế **9 collection** trên MongoDB Atlas, khai thác tốt ưu điểm của mô hình document (nhúng thông số, phiên bản, tồn kho) và dùng TTL index cho dữ liệu tạm.
- Hoàn thiện luồng **đăng ký có OTP, đăng nhập JWT, Google, quên/đổi mật khẩu** và phân quyền khách hàng / quản trị viên.
- Hiện thực nghiệp vụ **đơn hàng có đồng bộ tồn kho hai chiều**, mã giảm giá, bảo hành điện tử.
- Tích hợp thành công các dịch vụ thực tế: **PayOS/VietQR** (Webhook có chữ ký), **Brevo/Nodemailer**, **Telegram Bot** tương tác hai chiều và **Trợ lý AI** có dự phòng nhiều tầng.
- Kiểm thử 30 ca API thực tế đều đạt; hệ thống chạy ổn định cùng Front-End Next.js.

❖ Kiến thức và kỹ năng thu được

- Hiểu rõ quy trình xây dựng một hệ thống Back-End từ thiết kế kiến trúc, cơ sở dữ liệu đến đặc tả và kiểm thử API.
- Thành thạo Express (router, middleware, xử lý lỗi), Mongoose (schema, index, hook), JWT và bcrypt.
- Biết cách tích hợp API bên thứ ba an toàn: ký và xác minh HMAC, timeout, bộ nhớ đệm, cơ chế dự phòng khi dịch vụ lỗi.
- Rèn luyện kỹ năng đọc tài liệu kỹ thuật, làm việc nhóm với Git và tự rà soát bảo mật mã nguồn.

❖ **Hạn chế**

- Một số route quản trị chưa được bảo vệ và luồng đăng nhập Google còn nhánh dự phòng không an toàn (mục 5.4).
- Chưa có bộ kiểm thử tự động (unit test / integration test) và chưa giới hạn tần suất request.
- Chưa dùng transaction khi trừ tồn kho nhiều sản phẩm; tìm kiếm dựa trên `$regex` chưa tối ưu cho dữ liệu lớn.
- Đăng xuất mới xóa token ở client; server chưa có cơ chế thu hồi token.

7.2. Hướng phát triển

- **Bảo mật:** khắc phục toàn bộ mục 5.4; thêm rate limit, Helmet, kiểm tra dữ liệu bằng thư viện schema (Joi / Zod); dùng access token ngắn hạn kèm refresh token.
- **Chất lượng:** viết test tự động bằng Jest + Supertest, chạy CI trên GitHub Actions; sinh tài liệu API bằng Swagger/OpenAPI.
- **Hiệu năng:** thêm Redis cache cho danh sách sản phẩm; dùng Atlas Search / text index cho tìm kiếm; phân trang cho mọi API danh sách.
- **Dữ liệu:** dùng MongoDB transaction khi tạo đơn và trừ kho; chuẩn hóa định dạng ngày về kiểu `Date`.
- **Tính năng:** WebSocket để cập nhật trạng thái thanh toán tức thì thay cho polling; tích hợp đơn vị vận chuyển (GHN, GHTK) để tính phí và theo dõi đơn; AI gợi ý sản phẩm theo lịch sử xem.
- **Triển khai:** đóng gói Docker, triển khai Railway/Render có tên miền và HTTPS, giám sát bằng health check.

TÀI LIỆU THAM KHẢO

[1] Node.js Documentation — https://nodejs.org/docs

[2] Express 5.x API Reference — https://expressjs.com/en/5x/api.html

[3] MongoDB Manual — https://www.mongodb.com/docs/manual/

[4] Mongoose Documentation — https://mongoosejs.com/docs/

[5] RFC 7519 — JSON Web Token (JWT) — https://datatracker.ietf.org/doc/html/rfc7519

[6] jsonwebtoken (npm) — https://github.com/auth0/node-jsonwebtoken

[7] bcrypt.js — https://github.com/dcodeIO/bcrypt.js

[8] Nodemailer Documentation — https://nodemailer.com/

[9] Brevo API v3 Reference — https://developers.brevo.com/reference

[10] PayOS Developer Documentation — https://payos.vn/docs/

[11] VietQR Quick Link — https://www.vietqr.io/danh-sach-api/link-tao-ma-nhanh/

[12] Telegram Bot API — https://core.telegram.org/bots/api

[13] Google Identity — Verify the Google ID token — https://developers.google.com/identity/gsi/web/guides/verify-google-id-token

[14] Google Gemini API — https://ai.google.dev/gemini-api/docs

[15] OWASP API Security Top 10 — https://owasp.org/API-Security/

[16] MDN Web Docs — HTTP response status codes — https://developer.mozilla.org/en-US/docs/Web/HTTP/Status